

软件实践说明书

会议室预约与时间冲突检查系统

学 号： 2413042129 姓 名： 李宗熹

学 号： 2413042128 姓 名： 翟浩森

学 号： 2413042121 姓 名： 闫佳馥

学 号： 2413042116 姓 名： 张陈浩

学 号： 2413041630 姓 名： 霍汶洵

学 院： 软件学院

专 业： 软件工程

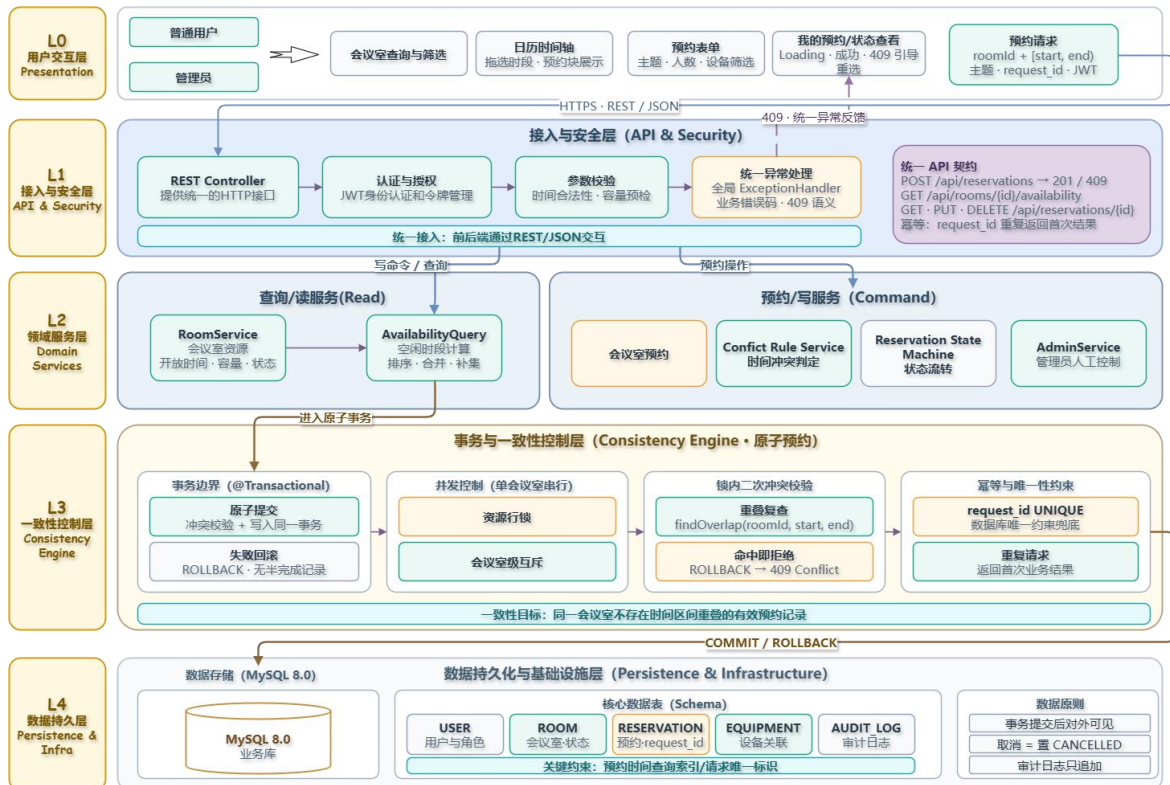
方向名称： 软件开发与测试

指导教师： 张淑娟 王亚恒

2026 年 9 月

摘要

会议室是高校与企业日常协作的重要公共资源，传统的人工登记或简单的电子表格管理方式存在时段冲突、审批滞后、通知不及时等问题。本实践以“会议室预约系统（TimeSlot）”为题，采用 Spring Boot 3 与 Vue 3 前后端分离技术栈，设计并实现了一套支持在线预约、自动冲突检测、分类审批、跨模块通知与运营统计的软件系统。



在架构上，系统采用模块化单体与六边形架构思想，将业务划分为身份、资源、预约、会议、审批、公共六大模块，并通过三对端口与适配器约束跨模块协作。在核心逻辑上，系统实现了预约创建全流程管控、基于半开区间的时间冲突检测、由会议室分类决定是否需要审批的状态机，以及“悲观行锁 + 幂等唯一键 + 乐观锁”的并发保护；同时通过业务时钟、全局预约窗口与分类提前天数约束可预约时段，并实现信用分预约资格判定、统一异常处理和操作审计。

经过功能测试、接口测试和并发场景验证，系统能够处理预约创建、审批流转、冲突拒绝、预约变更与取消等主要业务场景，并对非法参数、越权访问和业务冲突给出明确反馈。本文按照可行性研究、需求分析、概要设计、详细设计、系统实现和系统测试的逻辑，对系统设计与实现过程进行说明。

关键词：会议室预约；时间冲突检测；并发控制；模块化单体；Spring Boot

目 录

1 引言	1
1.1 项目背景与意义	1
1.2 同类系统现状与不足	2
1.3 建设目标与拟解决的关键问题	3
1.4 主要工作与个人分工	4
1.5 系统范围与约束	8
1.6 论文组织结构	9
2 可行性研究	11
2.1 技术可行性	11
2.2 经济可行性	12
2.3 操作可行性	12
2.4 运行与维护可行性	13
2.5 可行性结论	13
3 系统需求分析	14
3.1 用户与角色分析	14
3.2 系统总体业务流程	15
3.3 功能需求分析	18
3.3.1 用户与组织管理	18
3.3.2 会议室资源管理	18
3.3.3 预约申请与审批	18
3.3.4 会议执行与消息通知	19
3.3.5 运营统计与看板	19
3.3.6 信用与违约管理	19
3.4 核心业务规则	19
3.5 非功能性需求	20
3.6 系统用例分析	21
3.7 数据需求分析	22
4 系统概要设计	23
4.1 设计目标与原则	23

4.2	系统总体架构	24
4.3	功能模块划分与职责	26
4.4	模块间通信设计	27
4.5	数据库总体设计	29
4.6	接口总体设计	29
4.7	系统部署设计	30
5	系统详细设计	32
5.1	预约创建流程设计	32
5.2	预约状态机设计	34
5.3	时间冲突检测设计	35
5.4	并发与幂等设计	36
5.5	时间与预约窗口约束设计	38
5.6	审批与通知设计	39
5.7	信用与预约资格设计	40
5.8	数据库详细设计	41
5.9	预约状态迁移汇总	43
5.10	异常处理与错误码设计	44
5.11	关键设计权衡	45
5.12	审批超时自动失效设计	46
5.13	周期性预约批量创建设计	47
5.14	运营统计指标口径设计	48
5.15	会议实际使用记录设计	49
6	系统实现	51
6.1	开发环境与工具	51
6.2	后端分模块实现	51
6.3	前端实现	53
6.4	数据库实现与迁移	54
6.5	安全与鉴权实现	56
6.6	系统部署与启动	57
6.7	关键页面与业务操作说明	58

6.7.1 功能展示路线	58
6.7.2 用户登录与角色功能入口	59
6.7.3 会议室台账与资源信息展示	60
6.7.4 会议室条件筛选与指定时段可用查询	61
6.7.5 预约申请与参数校验	62
6.7.6 时间冲突检测	64
6.7.7 我的预约与预约详情	66
6.7.8 预约改期与会议室变更	67
6.7.9 预约取消与时段释放	68
6.7.10 会议实际使用登记	69
6.7.11 会议室资源新增、编辑与状态维护	70
6.7.12 会议室删除保护	72
6.7.13 统一异常与业务反馈	72
6.7.14 会议室使用统计	73
6.7.15 周期性会议批量预约	74
6.7.16 参会人名单维护	75
6.7.17 会议通知	75
6.7.18 关键页面业务闭环总结	76
6.7.19 截图覆盖检查表	76
7 系统测试及结果	78
7.1 测试目标与环境	78
7.2 功能测试	78
7.3 接口测试	79
7.4 并发与边界测试	80
7.5 异常与安全测试	81
7.6 性能测试	81
7.7 缺陷记录与修复	82
7.8 新增能力补充测试	83
7.9 测试结果分析	84
8 结论	86

8.1 工作总结	86
8.2 系统特点与设计亮点	86
8.3 不足与展望	87
8.4 实践体会	88
参考文献	90
致 谢	91
附录说明	92
附录 A 主要数据库表结构	93
A.1 department	93
A.2 sys_user	94
A.3 user_violation	94
A.4 room_category	95
A.5 meeting_room	95
A.6 room_facility	96
A.7 room_open_rule	96
A.8 room_maintenance	96
A.9 facility_repair_ticket	97
A.10 reservation	97
A.11 approval_record	98
A.12 meeting_execution	98
A.13 reservation_attendee	99
A.14 notification	99
A.15 operation_log	100
A.16 system_time_config	100
A.17 booking_window_config	100
附录 B REST API 接口清单	101
附录 C 核心业务规则与状态转换	103
C.1 预约持久化状态	103
C.2 展示状态推导	103
C.3 时间冲突与资源占用规则	103
C.4 审批与取消状态转换	104
C.5 其他业务状态	104

附录 D 错误码与异常响应说明	105
D.1 统一响应示例	105
附录 E 测试用例与执行记录	106
E.1 自动化与集成测试执行汇总	108
附录 F 系统运行与部署说明	109
F.1 环境要求	109
F.2 数据库初始化	109
F.3 本地环境配置	109
F.4 启动方式	110
F.5 验证命令	110
F.6 部署与安全注意事项	110

1 引言

1.1 项目背景与意义

在高校与企业的日常运行中，会议室作为共享资源常常面临“谁在用、何时用、是否冲突”的信息不对称问题。传统的纸质登记或微信群接龙方式，不仅容易漏记、错记，更无法在预约发生的瞬间发现时段重叠，导致“双占”或临时找不到空闲房间。本实践的目的在于利用软件工程的方法，设计并实现一个网页端的会议室预约系统，使预约方能够直观地查看空闲时段并发起预约，管理员能够统一审批与监管，系统自动完成冲突校验与状态流转。

本系统在实用性上解决了三个痛点：一是把“人找空闲房间”变为“系统展示空闲”，降低沟通成本；二是把“事后发现冲突”前移到“提交即校验”，避免资源浪费；三是把审批、通知、统计沉淀为系统能力，便于管理闭环。从工程训练角度看，本项目涵盖了需求分析、架构设计、并发控制、数据库设计与测试等软件工程核心环节，具有较高的综合训练价值。

从使用者的角度看，会议室管理的困难并不在于“能不能登记”，而在于三条关键信息能否保持实时与一致：这间会议室此刻是否被占用、某个时段是否还空着、这次变更有没有通知到相关人。人工登记或群内接龙的方式把这三点都交给了人的记忆与自觉，一旦使用频率上升，问题就会集中暴露：同一时段被两次承诺、临时占用无人知晓、事后无法追溯是谁在什么时候改了安排。

本系统把这些信息交由系统统一维护：会议室台账提供房间的位置、容量、设备与可用状态；日历周视图按周展示每个房间的占用与空闲；预约提交时即时完成冲突判定并给出冲突明细；状态变化由系统主动生成通知。表 1.1 从六个维度对比了传统管理方式与系统化管理方式的差别。

表 1.1 传统管理方式与本系统的对比

对比维度	传统人工登记 / 群内接龙	本系统
空闲信息获取	需逐个询问或翻查记录，信息滞后	日历周视图集中展示各会议室的占用与空闲
冲突发现时机	往往在会议当天现场才发现重复占用	提交预约时即时校验并拒绝重叠时段

软件学院软件实践说明书

对比维度	传统人工登记 / 群内接龙	本系统
审批与留痕	口头或聊天记录，难以追溯	审批动作、理由与操作人结构化保存
状态变更通知	依赖人工逐个转告，容易遗漏	状态变化自动生成站内通知并去重
资源使用统计	无法统计，只能凭印象判断	按房间与时段聚合使用时长、次数与高峰分布
历史可追溯	记录被直接划掉，历史随之丢失	存在预约或工时禁止删除，须改为停用

对本实践而言，把上述差异落地为可验证的系统能力，比单纯完成一个“能提交预约”的页面更有价值：它要求把使用中被反复讨论的规则（如首尾相接算不算冲突、已开始的会议能否取消、有预约的会议室能否删除）固化为明确的判定条件，并让所有入口对这些条件保持一致的解释。本说明书后续各章正是围绕这一目标展开。

1.2 同类系统现状与不足

业界成熟的会议预约产品（如 Outlook 会议室预订、飞书/钉钉会议室）多以企业办公套件为载体，功能完备但体量庞大、定制成本高；开源社区也存在若干轻量级预约系统，但往往在并发正确性与审批流转上较为薄弱。本实践不追求功能广度，而是聚焦于“预约正确性”这一核心难点，着重打磨冲突检测、审批流转与并发安全。作为课程设计，本项目更强调对软件工程方法论的掌握与落地。

同类产品可以大致分为四类：企业办公套件内置的会议室预订、SaaS 协作平台的会议室插件、开源轻量预约系统，以及仍然普遍存在的表格与群接龙方式。它们的差异并不只在功能多少，更在于部署形态、数据归属与规则可控性，如表 1.2 所示。

表 1.2 同类方案对比

方案类型	典型代表	主要优势	主要不足
办公套件内置预订	Outlook / Exchange 会议室预订	与邮件、日历深度集成，组织内推广容易	依赖 Exchange 环境，部署重，业务规则难以按需定制
协作平台插件	飞书 / 钉钉会议室	移动端体验好，消息触达渠道多	绑定第三方平台，数据存放在外部，二次开发受限
开源轻量预约系统	社区开源项目	部署简单，源码可自行修改	并发与冲突处理多不严谨，普遍缺少审批、审计与统计
表格与群接龙	Excel、群内接龙	零成本，无需部署	无校验、无留痕、无统计，规模稍大即失效

软件学院软件实践说明书

方案类型	典型代表	主要优势	主要不足
本系统	TimeSlot	冲突判定严谨、审批与审计完整、模块边界清晰	功能范围聚焦于单组织内部，未接入外部消息渠道

因此本项目的定位不是复刻一个功能大而全的办公套件，而是面向单一组织内部，把“预约时段不被重复占用”这一核心正确性做扎实，并在此基础上补齐审批、通知与统计等配套能力。这一定位也决定了系统的取舍：宁可把范围收窄，也要保证核心规则在新建、改期、审批、删除等所有入口上保持一致。

1.3 建设目标与拟解决的关键问题

本系统的建设目标是：面向单一组织内部，提供一套可用的会议室在线预约与管理系统，做到“提交即校验、审批可追溯、状态可靠流转”，并在中等规模使用下保证预约数据的正确性。围绕该目标，需要重点解决以下关键问题：

- 如何在提交预约时即时、准确地判断同一会议室的时间冲突；
- 如何根据不同的会议室分类，自动决定“免审通过”或“需审批”；
- 如何在高并发下避免两个请求同时抢走同一时段（双重占用）；
- 如何让预约状态的变化（创建/审批/取消）可靠地通知到相关人；
- 如何将系统划分为边界清晰的模块，避免相互耦合。

围绕“提交即校验、审批可追溯、状态可靠流转”这一目标，需要解决的五个关键问题及其解决思路如表 1.3 所示。其中的共同特点是：它们都不是靠增加页面或字段就能解决的，而需要在数据模型、判定语义与模块协作方式上做出明确设计。

表 1.3 关键问题与解决思路

关键问题	解决思路	对应章节
冲突的即时与准确判定	以半开区间语义定义重叠判定式，只统计待审批与已确认两种状态	3.3.3、5.3
审批与免审的分流	把“是否需要审批”作为会议室分类的规则，创建时据此决定初始状态	3.3.2、5.2
高并发下避免双重占用	行锁串行化并发创建、幂等键消除重复提交、版本号防止并发修改覆盖	4.4

关键问题	解决思路	对应章节
状态变化可靠通知相关人	预约模块声明通知端口，由会议模块实现，避免模块间循环依赖	4.4、5.6
模块边界清晰、依赖无环	模块化单体 + 端口与适配器，跨模块调用只走公开接口	4.2、4.4

1.4 主要工作与个人分工

本项目在团队分工时，没有采用“一个人只做前端、一个人只做后端、一个人只负责数据库”的横向技术分层方式，而是采用**面向业务域的垂直分工方式**。其基本思想是以相对完整的业务能力作为成员的责任边界，使每名成员都能够经历需求分析、数据设计、后端实现、前端交互、接口联调和功能测试等完整的软件开发过程，而不是只完成某一技术层上的局部代码。

这种分工方式主要基于以下几个方面的考虑。

第一，**以业务高内聚作为分工依据**。系统虽然在技术上可以分为前端、后端和数据库，但真正发生变化的通常是业务规则。例如预约规则发生变化时，往往会同时影响预约表结构、后端校验逻辑、接口返回以及前端交互。如果这些内容分别由不同成员负责，一次较小的业务变更也可能需要多人协调。项目因此优先按照用户、资源、预约、会议执行和审批运营等业务能力划分开发线路，使与同一业务对象相关的数据、规则和操作尽量集中在同一责任域内，从而减少跨成员修改。

第二，**以完整业务闭环作为个人交付单位**。每名成员负责的不是若干孤立页面或接口，而是一条从用户操作到数据持久化的完整链路。例如预约线路不仅包括“创建预约”接口，还包括前端预约入口、参数校验、预约状态流转、冲突检测、数据库写入以及对应测试。这样可以使每条开发线路都具有明确的输入、处理过程和最终输出，也便于在阶段验收时独立验证。

第三，**通过明确的模块所有权降低多人并行开发冲突**。团队为主要业务域设置相对稳定的责任人，一个模块内部的核心规则原则上由该模块负责人维护，其他成员不直接修改其内部实现。跨模块使用某项能力时，优先通过公开 Service、接口或端口进行调用，而不是直接访问其他模块的数据表或内部类。这样能够避免多人同时修改同一批文件，也能够防止为了快速实现功能而形成无序的跨模块依赖。

第四，**共享能力不单独形成孤立开发线路，而由统一规范约束**。系统中的统一响应、错误码、鉴权、异常处理、业务时钟等公共能力会被多个模块使用，如果完全分配给某一个独立成员，容易形成“公共模块成为所有人的依赖中心”。因此，本项目将公共模块视为基础能力，由主要负责核心集成的成员维护，同时通过统一接口规范、错误码规范和安全规范约束其他模块的使用方式。公共能力只提供基础设施，不承载具体业务规则。

第五，**对跨域业务设置明确的协作边界**。实际业务不会完全局限于某一个模块，例如创建预约需要读取用户资格与会议室信息，审批需要改变预约状态，会议执行又需要读取预约记录。对于这类跨域场景，团队首先确定“谁拥有最终裁决权”，再决定调用关系。例如预约状态由预约模块负责最终修改，审批模块只负责提出审批动作；会议室属性由资源模块维护，预约模块只读取其业务属性。通过这种方式，可以避免同一条业务规则在两个模块中各实现一份。

第六，**在模块独立性的基础上平衡工作量和技術难度**。分工并不是简单按照模块数量平均分配。预约模块涉及时间冲突、状态机、事务、并发和幂等，技术复杂度较高，因此由一名成员集中承担核心逻辑，并同时承担部分系统集成工作；资源、身份、会议执行和审批运营虽然单点算法复杂度较低，但分别包含较多管理页面、接口、数据表和业务流程，以保证各成员总体工作量相对均衡。这种分配方式兼顾了任务数量、业务复杂度和技术难度，而不是单纯追求每个人接口数量完全一致。

第七，**采用“独立开发—接口联调—系统集成”的协作流程**。各成员首先在本业务域内完成需求细化和核心功能，实现到能够单独测试的状态；随后根据统一的数据结构和接口约定进行模块间联调；最后再进行完整业务链路测试。对于公共数据结构、接口参数、状态值和错误码等可能影响其他成员的内容，需要先形成统一约定后再修改，从而降低后期集成阶段的大规模返工风险。

基于上述思想，本项目最终形成了五条主要开发线路。五条线路分别围绕预约调度与冲突控制、会议资源与设备管理、用户组织与权限治理、会议执行与个人工作台、审批审计与运营分析展开。系统架构上的公共模块作为共享基础能力存在，因此**软件模块数量与人员数量并不要求一一对应**。这种分工方式既保持了各成员任务的相对独

立性，又通过统一接口形成必要协作，体现了高内聚、低耦合和面向业务能力组织代码的软件工程思想。

（1）李宗熹：预约调度与时间冲突控制核心域。

主要负责系统预约核心业务及关键技术问题的设计与实现，包括预约创建、预约改期与取消、指定时段空闲会议室查询、时间冲突检测、预约状态机、并发控制、请求幂等以及周期性预约等功能。在系统整体开发过程中，同时负责预约核心规则与其他业务模块之间的协调，参与公共业务规则、关键接口联调、数据库结构设计以及核心缺陷排查。该部分是系统保证“同一会议室同一时段不得被重复占用”的主要实现基础，也是整个系统业务链路的核心。

（2）翟浩森：会议资源与设备管理域。

主要负责会议室资源完整生命周期相关功能，包括会议室信息维护、会议室分类及容量管理、会议室状态管理、开放规则、设施设备管理、维护计划、设备报修以及资源可用性判断等功能。该模块向预约业务提供会议室基本信息及可预约性基础数据，并负责会议室新增、修改、停用和资源维护等管理操作，使预约模块能够在统一的资源规则基础上进行容量、状态和可用性校验。

（3）张陈浩：用户、组织与权限治理域。

主要负责用户身份及组织管理相关功能，包括用户账号、部门信息、角色权限、个人信息、账号状态以及用户信用和违规记录等功能，并参与登录鉴权与用户预约资格判断。该模块负责维护系统中的身份与权限边界，为预约、审批和资源管理等模块提供可靠的用户身份、角色以及资格信息，保证普通用户与管理员在不同业务场景下只能执行其权限范围内的操作。

（4）闫佳馥：会议执行与个人工作台域。

主要负责预约成功后的会议执行阶段及用户侧主要交互功能，包括日历式会议查看、我的预约、预约详情、参会人管理、签到签退、缺席判定、历史记录以及个人通知等功能。该模块重点解决会议从“预约成功”进入“实际使用”之后的数据记录问题，通过参会人和出勤信息维护，使系统不仅能够记录计划预约，还能够反映会议的实际执行情况，同时形成面向普通用户的完整个人工作台。

(5) 霍汶泫：审批、审计与运营分析域。

主要负责管理端业务闭环，包括特殊会议室预约审批、审批意见与审批历史、异常预约处理、操作审计、会议室利用情况统计、高峰时段分析以及运营数据展示等功能。该模块面向管理员集中提供待审批任务处理、管理操作留痕和系统运行情况分析能力，使预约业务中的管理决策具有明确记录，并通过统计数据辅助会议室资源的后续管理和优化。

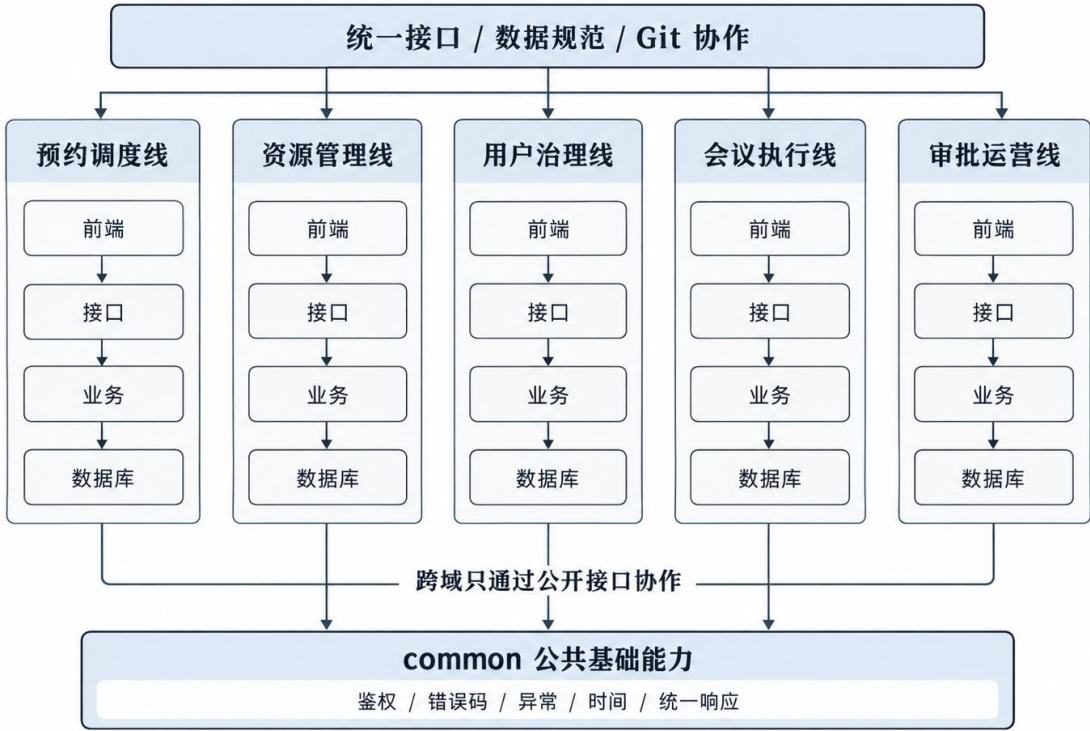


图 1.1 团队分工与业务垂直划分

项目开发过程中，五条业务线路并非完全独立。预约模块需要读取身份模块提供的用户资格信息和资源模块提供的会议室数据；审批模块需要通过预约业务的统一状态流转入口完成审批和强制取消；会议执行模块则需要基于预约记录维护参会人与通知信息。分工结果如图 1.1 所示。团队在开发过程中统一接口约定、状态语义、错误返回格式和数据库约束，通过模块间公开能力完成协作，原则上避免一个模块直接修改另一模块的内部业务状态。

表 1.4 团队成员主要工作与产出

软件学院软件实践说明书

成员	负责业务域	主要工作	主要产出
李宗熹	预约调度与冲突控制	预约创建、改期与取消、空闲查询、冲突检测、状态机、并发与幂等、周期性预约	预约核心服务、冲突检测机制、状态流转与并发控制
翟浩森	会议资源与设备管理	会议室及分类、容量规则、开放规则、设施、维护计划、报修、资源状态管理	会议室资源管理、设备与维护功能、资源可用性能力
张陈浩	用户、组织与权限治理	用户、部门、角色权限、账号状态、信用与违规记录、预约资格	身份认证与用户管理、权限控制、用户资格管理
闫佳徐	会议执行与个人工作台	我的预约、预约详情、参会人、签到签退、缺席判定、通知与历史记录	用户侧工作台、会议执行与出勤管理、个人通知
霍汶洙	审批、审计与运营分析	预约审批、审批历史、强制取消、操作审计、使用统计与运营分析	管理工作台、审批与审计功能、运营统计能力

这种划分方式使每名成员负责的内容都形成了相对完整的业务闭环，而不是只承担某一层的零散编码任务。通过五条垂直开发线路并行推进，团队能够分别完成各自业务域的前端、后端和数据相关工作，再通过统一接口完成系统级联调，体现了高内聚、低耦合的软件工程设计思想。

1.5 系统范围与约束

本系统的范围界定为：面向单一组织内部的会议室预约与管理，涵盖预约、审批、通知与基础统计；不涉及跨组织的资源调度，也不包含门禁、视频会议硬件等外部系统的深度集成。系统运行于通用开发环境，采用单节点部署，适合教学演示与中小规模使用。在实现上，系统以“正确性优先”为约束，重点保证冲突检测与并发控制的可靠性，而对界面美观度与超大规模并发等非核心指标不作过度追求。

为避免需求蔓延，本系统的功能范围在开始实现前即做了明确界定，表 1.5 列出了范围内的能力与明确排除在外的内容。

表 1.5 系统范围界定

类别	范围内	范围外
资源范围	单一组织内部的会议室及其分类、开放规则与设施	跨组织资源调度、外部场地的预约
业务流程	预约提交、审批、改期、取消、会议执行与出勤记录	门禁联动、视频会议硬件控制
通知渠道	站内通知（含已读状态）	短信、邮件及第三方即时通讯推送

类别	范围内	范围外
统计范围	按会议室与时段的使用时长、次数与高峰分布	成本核算、部门预算与费用结算
运行环境	单节点部署，支持开发与演示环境	多节点集群、高可用容灾与灰度发布

在约束方面，系统运行于 Java 21 与 Spring Boot 3 环境，数据持久化使用 MySQL 8，前端为 Vue3 单页应用；部署上采用单节点方式，开发阶段前端通过开发服务器代理后端接口以避免跨域。这些约束既是课程实践环境决定的，也直接影响了技术选型与架构风格：正因为运行环境是单节点、交互要求强一致，系统选择了模块化单体与端口适配器，而非分布式方案。

1.6 论文组织结构

本文围绕会议室预约与时间冲突检查系统的分析、设计、实现与验证展开，整体按照“可行性研究—需求分析—系统设计—系统实现—测试验证—总结改进”的技术路线组织，各章节内容逐步递进。

第 1 章为**引言**，介绍项目背景、建设目标、实践内容、团队分工以及本文的研究与实践范围，为后续系统设计与实现奠定基础。

第 2 章为**可行性研究**，从技术可行性、经济可行性、操作可行性和维护可行性等方面分析系统建设条件，并结合课程实践规模说明技术方案选择的合理性。

第 3 章为**系统需求分析**，明确系统用户角色、核心业务流程、功能需求、业务规则及非功能需求，重点界定会议室预约、时间冲突检测、审批与会议执行等业务边界，为系统设计提供需求依据。

第 4 章为**系统概要设计**，在需求分析基础上给出系统总体架构、业务模块划分、模块协作关系、接口组织方式及部署结构，明确系统各组成部分的职责及整体协作机制。

第 5 章为**系统详细设计**，针对系统核心业务展开进一步设计，重点说明预约状态机、时间冲突检测、事务与并发控制、请求幂等、跨模块协作以及数据库结构等关键机制。

第 6 章为**系统实现**，结合实际工程代码说明前端页面、后端业务服务、数据持久化、安全认证、公共基础能力及系统部署等内容，展示设计方案向可运行系统的具体转化过程。

第 7 章为**系统测试及结果分析**，围绕核心业务功能、接口行为、异常场景、并发一致性及系统运行情况开展测试，并结合测试结果分析系统实现是否满足既定需求和关键业务约束。

第 8 章为**结论**，总结本次软件实践的主要工作与系统实现成果，分析当前系统仍存在的不足，并提出后续可进一步完善的方向。

2 可行性研究

可行性研究用于在进入详细需求与设计之前，判断本系统在现有课程实践条件下是否具备可实现、可运行和可维护的基础。本项目规模定位为单组织内部的会议室预约管理系统，重点关注预约正确性、审批流转和数据一致性，因此不需要引入分布式服务、商业中间件或专用硬件。

2.1 技术可行性

系统采用 Java 21、Spring Boot 3、MyBatis、MySQL 8 与 Vue 3 等成熟技术构建。后端框架能够提供依赖注入、Web 接口和事务管理能力；MySQL 支持事务与行级锁，可用于实现同一会议室并发预约的串行化；MyBatis 允许直接控制冲突检测、聚合统计等关键 SQL；Vue 3 与 TypeScript 适合构建日历、表单和管理后台等交互页面。从现有开发环境和已完成的功能验证看，该技术路线能够覆盖本系统的主要需求。

- 后端：Spring Boot 3、Java 21、MyBatis、MySQL 8；
- 安全：JWT（HS256）令牌 + 过滤器 + 方法级角色注解；
- 前端：Vue3、Vite、TypeScript、Pinia、Element Plus；
- 设计理念：领域驱动的分层 + 端口适配器解耦。

技术选型围绕三个考虑因素：是否服务于核心正确性需求、是否便于在单节点环境部署验证、是否存在成熟稳定的社区支持。主要选型及其理由如表 2.1 所示。

表 2.1 主要技术选型与理由

技术	版本	主要用途	选择理由
Java	21	后端开发语言	长期支持版本，记录类等特性便于书写不可变值对象
Spring Boot	3.3.5	应用框架与依赖注入	生态成熟、内嵌容器便于单节点启动，分层结构便于按模块组织代码
MyBatis	3.x	数据访问	SQL 可控，冲突检测与聚合统计需要精确控制语句与索引

软件学院软件实践说明书

技术	版本	主要用途	选择理由
MySQL	8.0	关系型数据库	支持事务与行级锁，可满足并发抢占同一时段时的串行化要求
Vue 3 + Vite	3.x	前端框架与构建工具	组合式 API 便于按功能组织逻辑，开发服务器启动快并内置接口代理
Element Plus	—	界面组件库	表格、表单、弹层等组件齐全，与业务页面的交互需求匹配
Pinia	—	前端状态管理	统一维护登录用户与角色，避免权限判断散落在各页面
JWT (HS256)	—	无状态鉴权	前后端分离下无需服务端会话，便于在过滤器中统一完成鉴权

其中 MyBatis 与 MySQL 的组合是出于正确性考虑而非习惯：冲突检测需要在一条语句中完成“房间 + 状态 + 时间段”的交叠判断，删除保护需要按条件计数，运营统计需要分组聚合，这些场景使用可控 SQL 配合索引比通用 ORM 更直观，也更容易在评审时说明其判定过程。

2.2 经济可行性

本项目使用的主要开发框架、数据库与前端组件均可在课程实践环境中直接使用，不需要额外采购商业授权或专用服务器。系统采用单节点部署，普通开发计算机即可完成前后端启动、数据库运行和并发测试；团队协作依赖 Git 等现有开发工具，因此额外成本主要来自开发与测试时间，而非软硬件采购。对于课程设计和中小规模内部使用场景，该成本结构具有可接受性。

2.3 操作可行性

系统的核心操作围绕“查询会议室—选择时段—提交预约—查看状态”展开。普通用户不需要理解底层冲突算法，只需在会议室列表或日历中选择资源与时间，系统负责完成资格、容量、时间窗口与冲突校验；管理员的主要操作集中在会议室维护、待审批预约处理、用户信用管理和运营看板。前端对成功、冲突、越权和参数错误均提

供明确反馈，并通过状态颜色、通知横幅和内置说明页面降低学习成本，因此具备较好的操作可行性。

2.4 运行与维护可行性

系统采用前后端分离的模块化单体结构，后端按身份、资源、预约、会议、审批和公共模块组织代码，跨模块调用通过公开接口与端口完成；数据库使用版本化脚本初始化和迁移，核心业务规则具有自动化测试覆盖。部署时只需依次启动 MySQL、Spring Boot 后端和 Vue 前端，关键配置在启动阶段进行校验。该结构避免了分布式部署带来的额外运维复杂度，也使功能定位、缺陷排查和后续修改具有明确边界。

2.5 可行性结论

综合技术、经济、操作和维护四个方面，本项目不存在超出课程实践条件的关键技术障碍，所选技术栈与系统规模匹配，开发和运行成本可控，主要业务流程具备清晰的操作入口与维护边界。因此，本系统具备继续开展需求分析、系统设计、编码实现和测试验证的可行性。

3 系统需求分析

本章从用户角色、业务流程、功能需求、业务规则、非功能需求和数据需求六个方面定义系统应当实现的能力。需求分析只描述系统“需要做什么”和“什么情况下不允许做”，具体的算法、事务和代码实现放在后续设计与实现章节中说明。

3.1 用户与角色分析

系统存在两类主要角色：普通用户与管理员，二者权限边界清晰。普通用户以“发起与管理自己的预约”为中心，能查看会议室、发起与取消本人预约、查看个人通知；管理员以“资源配置与审批监管”为中心，能维护会议室与分类、审批预约、查看运营统计与操作审计。管理类接口仅对管理员开放，普通用户访问时返回无权限提示，权限判定在后端完成，不依赖前端隐藏按钮。两类角色的功能需求如表 3.1 所示。



图 3.1 系统功能与角色业务图

表 3.1 角色与功能需求

角色	功能需求
普通用户	注册与登录、查看会议室与空闲时段、发起预约、查看与取消本人预约、维护本人会议的参会人并签到签退、查看个人通知与预约资格
管理员	会议室与分类维护、开放规则与报修管理、预约审批与驳回、强制取消、操作审计、运营看板统计、系统时间与预约窗口配置、用户信用与状态管理

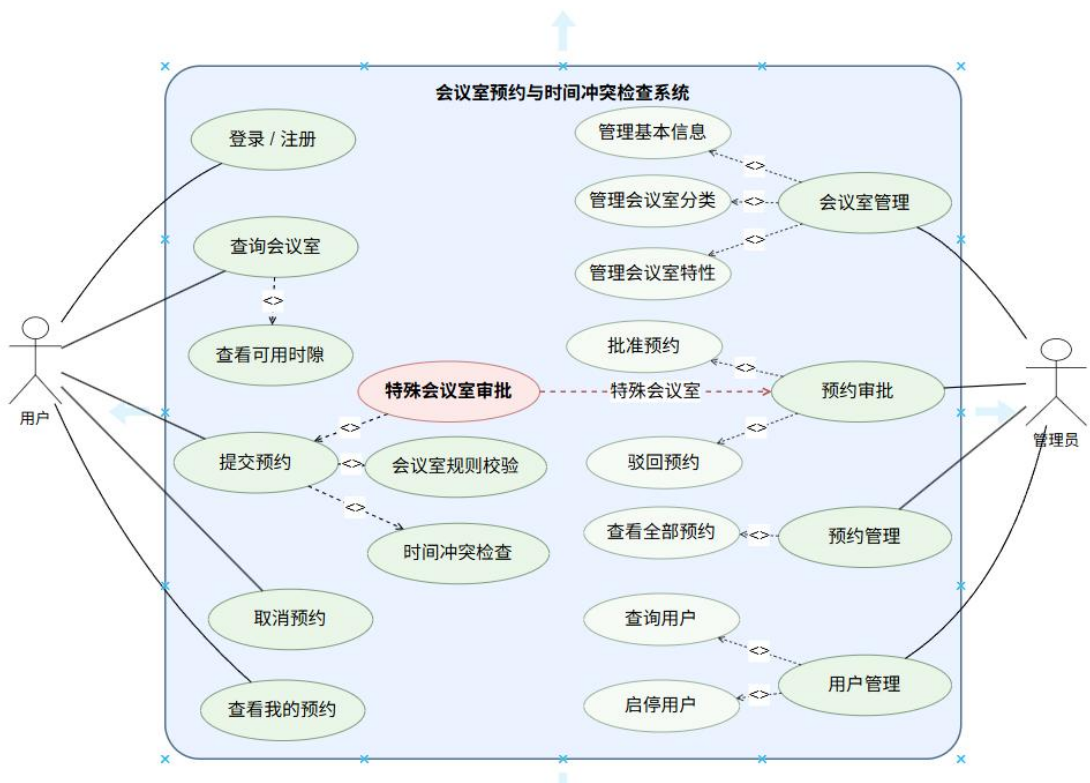


图 3.2 系统用例图

3.2 系统总体业务流程

系统的核心业务流程以预约为中心。用户登录后首先查询会议室台账或指定时段
的空闲资源，选择会议室与起止时间并填写预约信息；系统完成预约资格、容量、时
间窗口和冲突校验后，根据会议室分类决定直接确认或进入待审批状态。后续可继续
发生审批、改期、取消、通知、参会人维护、签到签退和实际使用记录登记等操作。
功能分类为图 3.3 的样式，总体核心业务流程如图 3.4 所示。

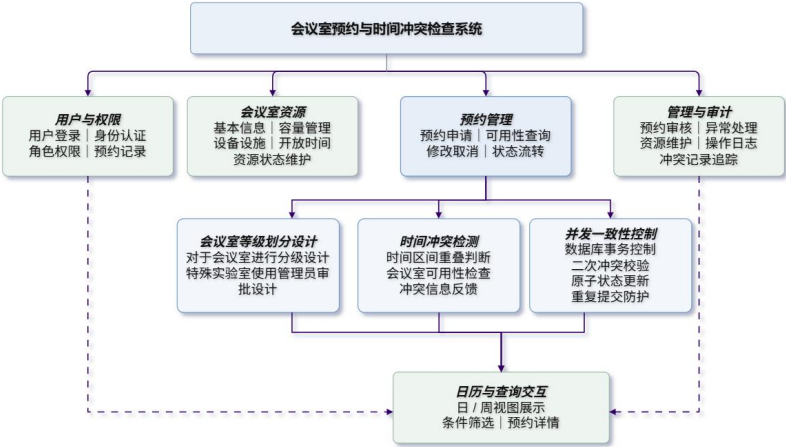


图 3.3 系统总体业务流程

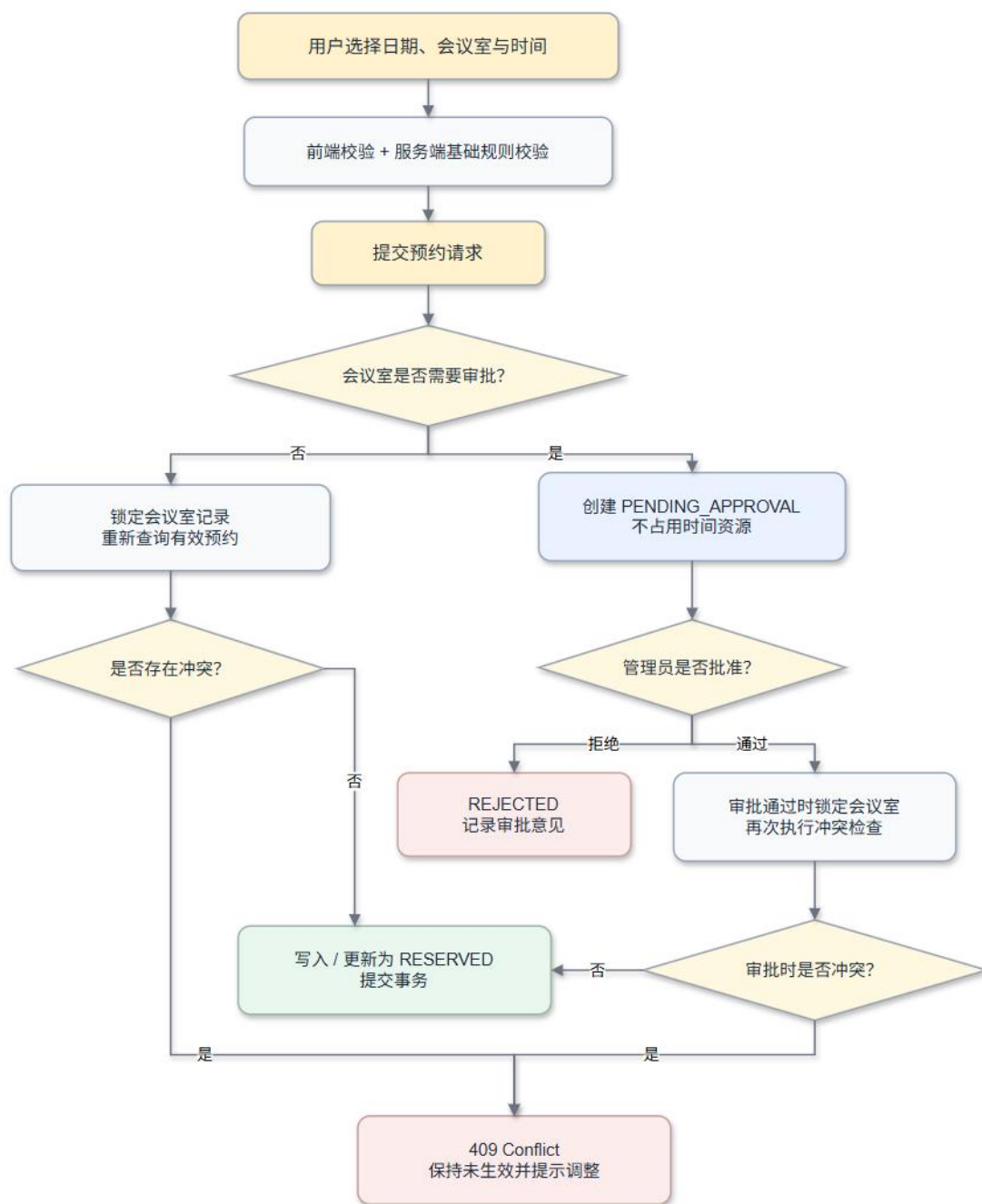


图 3.4 系统总体核心业务流程

3.3 功能需求分析

3.3.1 用户与组织管理

支持用户登录，登录成功后签发访问令牌，后续请求凭令牌识别身份与角色；支持按部门组织用户，维护用户的账号信息与角色。管理员可对用户执行启用、禁用等状态管理，并可在需要时调整用户的信用分与限制期。

用户信息中的“账号状态、信用分、限制期”三项共同构成预约资格判定的三要素，供预约环节统一校验：三项中任何一项不满足，都不允许发起预约。把资格判定集中到一处，可以避免预约、审批等环节各自判断、口径不一致。

3.3.2 会议室资源管理

支持会议室分类的维护。每个分类可配置四类规则：是否需要审批、容量区间（最小与最大人数）、单次最长预约时长、可提前预约的天数。把规则挂在分类上而不是逐间会议室单独设置，可以在管理上“成组管理”——例如所有小型讨论室共用一套规则，改一次即可全部生效。

支持会议室的新增、修改与停用，维护其所属分类、名称、位置、容量与状态，其中会议室容量必须落在所属分类规定的容量区间之内。支持按星期配置会议室开放时段，登记会议室及其设施的报修工单，并支持维护计划管理。当前版本已完成按星期开放规则的配置与维护，但该规则尚未接入预约创建与改期的强制校验；这一实现边界在第 8.3 节中列为后续完善项。

3.3.3 预约申请与审批

支持用户在日历视图中查看各会议室的空闲与占用并发起预约。提交时系统自动完成预约资格校验、必填校验、时间窗口校验与时间冲突检测，并根据所选会议室分类是否需要审批，决定该预约直接确认还是进入待审批。

管理员可对待审批的预约执行“通过”或“驳回”，也可对异常预约执行“强制取消”。预约状态在待审批、已确认、已驳回、已取消之间按规则流转：已驳回与已取消是终态，不可再迁移。所有审批动作都会写入审批记录与操作审计，保留操作人、时间与备注，便于事后追溯。

3.3.4 会议执行与消息通知

支持为预约登记参会人名单，并在会议开始后进行签到、签退与缺席登记；支持按出勤状态统计应到、已签到、已签退与缺席人数，使会议的实际执行情况可被记录与查看。

支持在预约生命周期的关键变化（创建成功、进入待审批、审批通过、被驳回、被取消）以及会议执行阶段（被加入会议、被移出会议、会议即将开始、缺席判定）向相关用户推送站内通知。通知要求具备去重能力，避免同一事件因重复调度而产生多条重复消息。

3.3.5 运营统计与看板

支持管理员查看预约数量、会议室使用时长与次数、高峰时段分布等运营统计数据，用于了解资源使用情况并优化排班。看板数据以真实预约记录为依据，通过数据库聚合查询得出，不依赖前端二次计算，从而保证统计口径统一、结果可核对。此外，管理员可查看操作审计日志，了解管理动作的发生时间、操作人与来源地址。

3.3.6 信用与违约管理

支持对用户信用分的管理。正常履约维持信用分，违规行为（如预约后不履约、长期占用不释放等）会被记录并可能扣减信用分，同时保留违规记录便于审计与追溯。

系统设定两个信用门槛：低于预约门槛的用户不得发起预约；低于自动拉黑阈值的用户会被系统自动限制一段时间，恢复达标后自动解除限制。系统自动产生的信用变动与人工加黑在记录中通过原因前缀区分，使自动处理与人工处理可分辨；人工加黑须由人工解除，不会被自动流程覆盖。

3.4 核心业务规则

业务规则是系统需要在任何入口（新建、改期、审批、删除等）都保持一致的关键约束。系统的核心业务规则如下：

- 审批规则：不同分类的会议室可配置是否需要审批。需审批的预约在获得批准之前不得视为最终占用，但仍占用时段（其他人在此时段预约会被判为冲突），以避免同一时段被重复承诺。
- 容量规则：包含两层——会议室的容量必须落在其分类规定的容量区间之内；单次预约的参会人数不得超过该会议室的容量。前者约束资源本身，后者约束使用行为。
- 时间规则：预约的开始时间必须晚于当前时间；整个预约时段必须落在管理员设置的全局可预约时段之内；必须满足分类规定的提前预约天数限制；单次预约时长不得超过分类规定的最长时长。四条同时成立才允许预约。
- 唯一规则：同一会议室在同一时段内的有效预约不得重叠，重叠判定采用半开区间语义，首尾相接不算重叠。
- 状态规则：预约在待审批、已确认、已驳回、已取消之间按状态机流转；已驳回与已取消为终态；会议开始之后不允许修改或取消。
- 信用规则：信用分低于预约门槛或处于限制期的用户不得发起预约；涉及预约资格的三要素（账号状态、信用分、限制期）中任一不满足即拒绝。
- 引用规则：存在未来预约的会议室不允许删除；存在历史预约或报修工单的会议室同样不允许物理删除，建议改为“停用”。
- 幂等规则：同一请求标识的重复提交只生成一条预约记录，重复提交返回首次处理结果，避免因网络重试产生重复预约。

3.5 非功能性需求

除功能需求外，系统还需满足下列非功能性要求：

- 正确性：同一会议室同一时段不得被重复占用，这是系统的首要指标。冲突检测与并发控制必须共同保证这一点，且在“查得到空闲”与“约得上”之间保持口径一致。
- 并发与一致性：多用户同时抢约同一时段时，只允许一个请求成功。系统采用行锁、请求幂等与版本号乐观锁三重手段，分别应对“并发抢占”“重复提交”与“并发修改覆盖”三类问题。

- 安全性：采用登录鉴权与基于角色的访问控制，管理类接口仅对管理员开放；密码以加盐摘要方式存储；错误响应不向前端暴露异常堆栈与数据库细节。
- 易用性：以日历周视图直观展示空闲与占用，使选时段这一核心操作所见即所得；操作反馈清晰明确，失败时给出可直接据以调整操作的具体原因。
- 可维护性：模块边界清晰，域与域之间通过接口协作；核心状态由统一的状态机裁决，避免状态判断散落在各处；关键业务规则有自动化用例覆盖，便于修改与回归。
- 性能：常用查询（会议室列表、我的预约、日历占用）均为单表或少量关联查询，并建有相应索引；统计类查询使用数据库聚合，避免把大量明细数据传到前端再计算。
- 可部署性与兼容性：系统以单节点方式即可在普通开发机上部署运行，开发环境通过前端开发服务器代理后端接口，部署时只需调整数据库连接与密钥配置。

在运行环境方面，后端运行于 Java 21 与 Spring Boot 3 环境，数据持久化使用 MySQL 8；前端为 Vue3 单页应用。前后端分离部署，通过 HTTP 接口通信。

3.6 系统用例分析

系统的主要用例汇总如表 3.2 所示，覆盖普通用户与管理的全部主要操作。

表 3.2 主要用例汇总

用例	参与者	简述
用户登录	普通用户 / 管理员	通过账号密码获取访问令牌，据此识别身份与角色
查看会议室	普通用户	查看会议室台账及其位置、容量与设备，可按条件筛选
查看空闲	普通用户	在日历视图中查看各会议室空闲时段，或按时段查询空闲会议室
发起预约	普通用户	选择房间与时段提交预约，系统校验资格、时间规则与冲突
改期预约	普通用户	修改尚未开始预约的时段或会议室，重新执行冲突检测
取消预约	普通用户	取消本人尚未开始的预约并释放时段
维护参会人	普通用户 / 管理员	维护预约的参会人名单，登记签到、签退与出勤

用例	参与者	简述
审批预约	管理员	对待审批预约执行通过或驳回
强制取消	管理员	对异常预约执行强制取消
维护资源	管理员	管理会议室、分类、开放规则、设施与报修工单
查看看板	管理员	查看预约统计、会议室使用情况与高峰时段
接收通知	全部用户	接收与预约、会议相关的站内通知并标记已读

3.7 数据需求分析

系统需要持久化的数据可分为五类：

- 组织与用户数据：用户账号、角色、账号状态、信用分、限制期与所属部门，以及用户的违规与信用变动记录。
- 资源数据：会议室分类及其业务规则（是否审批、容量区间、最长时长、提前天数）、会议室本体（名称、位置、容量、状态）、按星期的开放规则、设施清单与报修工单。
- 预约数据：预约记录（包含请求标识、预约编号、预约人、会议室、起止时间、参会人数、状态与版本号）、参会人名单与出勤记录（含签到与签退时间、出勤状态），以及会议级实际使用记录（实际起止时间、实际参会人数与登记人，与预约一对一）。
- 过程数据：审批记录（审批人、动作、备注）与操作审计日志（操作人、操作类型、业务对象、来源地址），用于回溯管理动作。
- 通知与配置数据：站内通知（含去重键与已读标记）、业务时钟配置与全局可预约时段配置。

这些数据以“用户—分类—会议室—预约”为主线彼此关联：用户发起预约，预约指向会议室，会议室隶属于分类，分类决定了审批与时间规则；预约向下延伸出参会人与出勤记录、审批记录与通知。数据之间的关联关系需要以约束（唯一键与外键）保证一致性，例如预约的请求标识需唯一以保证幂等、分类名称需唯一以避免重复分类等。数据的详细设计见第 5 章。

4 系统概要设计

4.1 设计目标与原则

概要设计以实现“正确、清晰、可维护”为目标，遵循以下原则：一是正确性优先，把冲突检测与并发控制作为设计的核心，宁可牺牲部分灵活性也要保证同一时段不被重复占用；二是边界清晰，按业务能力划分模块，模块对外只暴露接口，禁止跨模块直接访问内部实现；三是依赖单向，通过端口与适配器控制依赖方向，从设计上杜绝循环依赖；四是以数据为准，系统状态由后端统一裁决，前端只做展示，避免前后端状态分叉。

正确性优先是全部设计决策的第一顺位。会议室预约的业务规则本身并不复杂，真正的风险集中在两处：同一时段被重复承诺，以及并发操作导致数据被静默覆盖。为此，系统在关键路径上并不追求代码最短，而是显式引入行锁、请求幂等与版本号三重保护，并让冲突判定只依赖一个统一的判定式，避免“查得到空闲”与“约得上”出现口径不一致。

边界清晰指模块之间只通过约定的接口协作，任何一方都不得读取对方的内部实现。系统按业务能力切分为六个模块，模块内部再分接口层、业务层与数据层；跨模块调用只能走对方公开的服务接口或端口接口。这条原则在实现中最直接的收益是：修改某个模块的内部结构不会波及调用方，回归测试的范围可以被准确界定。

关键业务规则必须只有一处权威实现。预约状态的迁移矩阵集中在状态机中，创建预约的初始状态、审批、驳回、取消与改期都调用同一个迁移入口；时间冲突的判定集中在预约模块，其他模块若需要判断“某时段是否被占用”，必须通过接口询问，而不是自行拼接一条查询语句。规则集中之后，规则变更只需改一处，也不会出现两个入口判断结果相互矛盾的情况。

系统还需要对使用者与管理者的疑问给出可解释的答案，因此把错误处理与过程记录一并纳入设计范围：错误响应区分“参数不对”“权限不足”“业务不允许”“系统异常”四类并给出可读原因；审批与强制取消等管理动作写入审计日志，记录

操作人、动作、业务对象与来源地址；站内通知带有业务去重键，重复调度不会产生重复记录。

4.2 系统总体架构

系统采用“模块化单体 + 六边形（端口与适配器）”的架构风格：按业务能力划分为六大模块，模块内部遵循接口层、业务层、数据层、对外端口的分层；模块对外只暴露“端口（接口）”，由其他模块以“适配器”实现，依赖方向始终单向、无环。这样既保持了模块边界的清晰，又为未来可能的服务化拆分预留了空间。

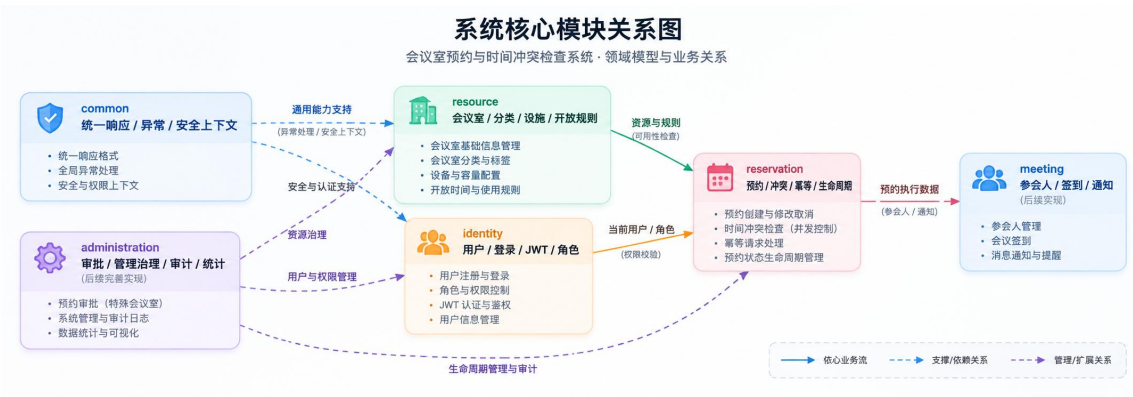


图 4.1 系统核心模块关系图

软件学院软件实践说明书

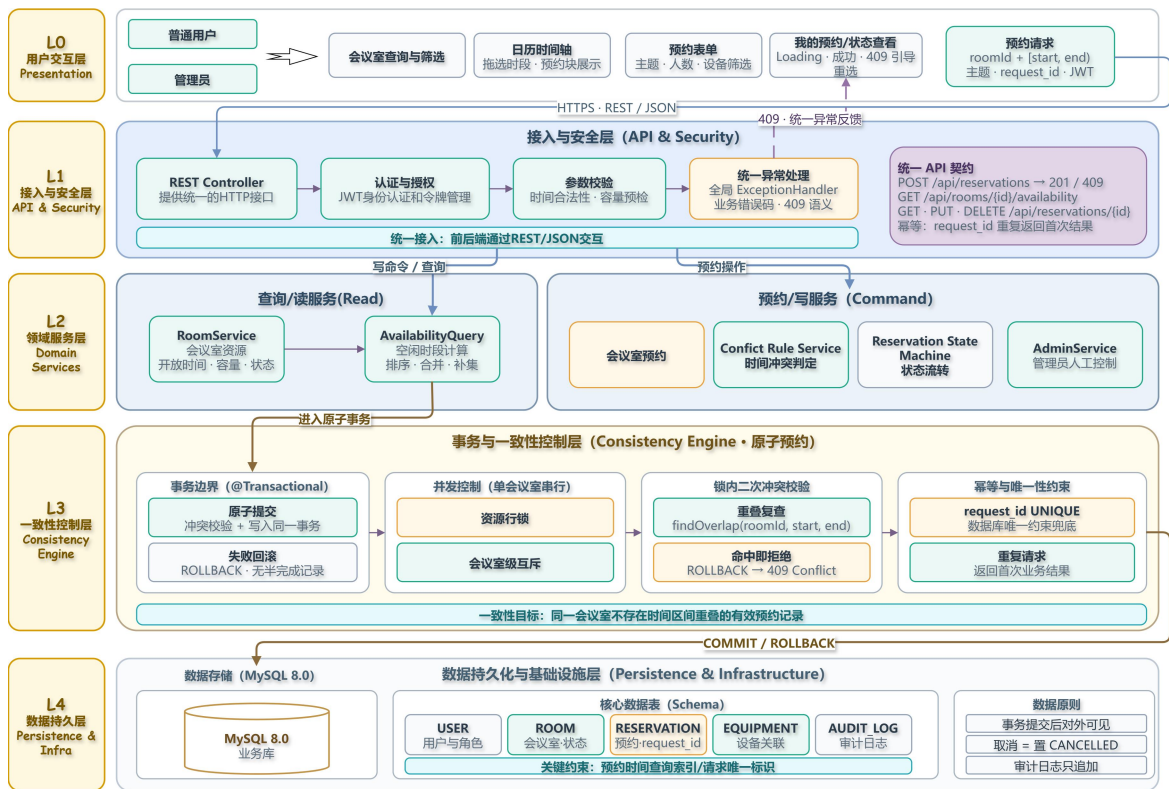


图 4.2 系统分层架构与模块分布

从应用代码职责上看，系统采用表现层、接口层、业务层和数据访问层四层结构；此外，模块之间通过端口层表达跨模块依赖，底层由 MySQL 提供持久化存储，如图 4.2 所示。端口层属于模块协作边界，存储层属于基础设施，不与应用四层并列理解。应用层次遵循单向依赖原则：上层可以调用下层，下层不得反向依赖上层。

各层的职责边界是明确的。表现层只负责呈现与交互，不承载业务判断；接口层负责参数解析、鉴权与统一响应封装，不写业务分支；业务层承载全部业务规则与事务边界，是“什么情况下不允许做”的唯一判断场所；数据层只做数据存取，不包含业务语义。这样划分之后，业务规则的位置是确定的，评审与排错都可以沿着这条线索定位。

在架构风格上，系统采用模块化单体，而不是微服务。原因是本项目的运行环境是单节点，且预约、审批、通知之间的交互频繁并要求强一致——例如审批通过必须与预约状态变更在同一事务内完成。若拆分为独立服务，就需要引入分布式事务或最终一致性方案，复杂度与运维成本都会显著上升，而收益在本场景下并不明显。模块

化的收益则可以在单体内部获得：边界清晰、依赖单向；若未来某个模块确有独立部署的需要，也可以沿既有的接口边界平滑拆分。

4.3 功能模块划分与职责

系统按业务能力划分为六大模块，各模块的核心职责如表 4.1 所示。

表 4.1 六大功能模块及职责

模块	核心职责
身份模块	用户、部门、登录鉴权、信用分、违规与预约资格判定
资源模块	会议室、分类（含容量区间）、开放规则、设施、报修与维护
预约模块	预约的创建、状态流转、时间冲突检测、幂等与并发控制
会议模块	参会人管理、签到签退与缺席判定、会议实际使用记录登记、通知的存储与读取
审批模块	审批、强制取消、审批记录、操作审计与运营统计
公共模块	统一响应、错误码、全局异常、安全、业务时钟与预约窗口

模块划分的依据是业务能力而不是技术分层：凡是对同一种业务对象负责、且规则变化时应当一起改动的代码，归入同一个模块。各模块对外暴露的能力与依赖方向如表 4.2 所示。

表 4.2 各模块对外能力与依赖方向

模块	对外暴露的能力	依赖方向
身份 identity	用户与角色查询、预约资格校验、信用分与违规记录维护	被预约、审批模块只读调用
资源 resource	会议室与分类查询、按时段的可约会议室查询、开放规则与工单	被预约模块只读调用；删除保护经端口向预约模块查询
预约 reservation	创建与改期、生命周期入口（审批/驳回/取消）、冲突检测、预约查询	依赖身份与资源的只读能力；实现资源的引用检查端口
会议 meeting	参会人维护、签到签退与出勤统计、通知存储与读取	实现预约模块的通知端口；只读预约信息
审批 administration	审批、驳回、强制取消、审批记录、操作审计与运营统计	经生命周期端口调用预约模块，不直接改状态
公共 common	统一响应、错误码、异常映射、鉴权、业务时钟与预约窗口	被各模块共用，不依赖任何业务模块

模块之间的依赖方向经过刻意设计：身份与资源是被依赖方，预约是依赖方，审批与会议则通过端口反向协作。这一安排使得业务核心（预约）位于依赖链的中段，既不依赖上层模块，也不被上层模块直接改动内部状态，为后续把规则变化集中在预约模块提供了结构上的保障。

4.4 模块间通信设计

为避免模块间循环依赖，系统围绕三类跨模块协作定义端口：预约通知、预约生命周期以及会议室预约引用检查。消费方只依赖端口契约，提供方通过适配器完成具体实现，从而把“需要什么能力”与“由谁实现”分离。预约模块对身份与资源仅执行只读查询；会议模块在需要读取预约信息时使用预约模块公开能力；审批动作则统一经预约生命周期端口进入预约模块的权威状态迁移逻辑；会议室删除保护通过引用检查端口获取预约引用信息，不直接跨模块访问数据表。

预约模块 ——只读查询——> 身份模块
预约模块 ——只读查询——> 资源模块
预约模块 ——通知端口契约——> 会议模块（通知适配器）
审批模块 ——生命周期端口——> 预约模块（权威状态迁移入口）

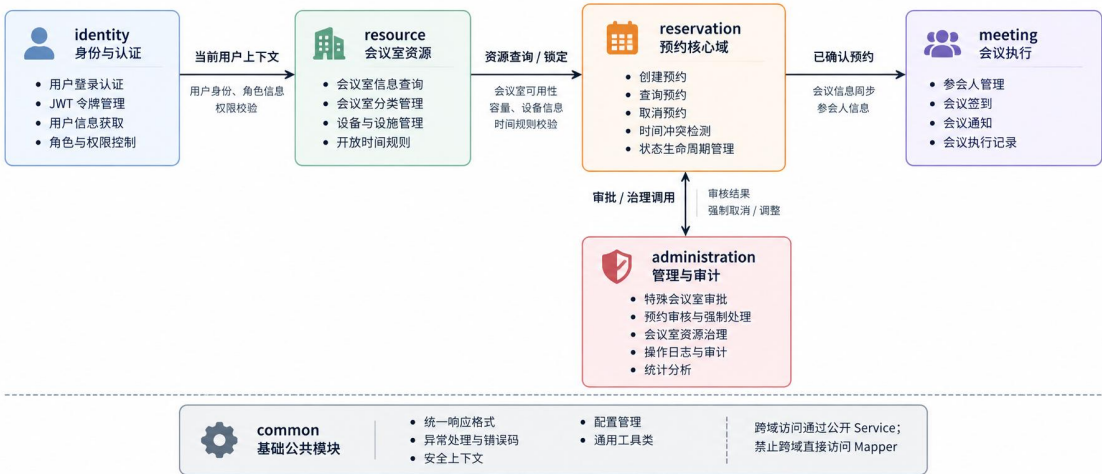


图 3-1 系统模块依赖关系图

图 4.3 系统模块依赖关系图

为避免模块之间形成递归依赖，系统用“端口 + 适配器”表达跨模块协作：由消费方定义接口（只声明需要什么能力），由提供方实现该接口。目前系统共有三对端口，如表 4.3 所示。

表 4.3 模块间端口与适配器

端口（定义方）	声明的能力	实现方（适配器）	解决的问题
ReservationNotificationPort （预约模块）	预约创建、待审批、通过、驳回、取消时的通知	ReservationNotificationAdapter （会议模块）	预约模块不直接依赖通知实现，避免与会议模块形成循环依赖
ReservationLifecyclePort （审批模块）	审批通过、驳回与强制取消的状态变更	ReservationLifecycleAdapter （审批模块）	审批动作统一委托预约模块的权威入口，不绕过状态机与校验
RoomReservationGuardPort （资源模块）	按会议室统计未来预约与历史预约数量	RoomReservationGuardAdapter （预约模块）	资源模块做删除保护时可获得预约引用信息而不反向依赖预约模块

第一对端口解决的是最直接的循环依赖风险：会议模块出于参会人与出勤统计的需要，本就要读取预约信息，已经单向依赖预约模块；若预约模块再直接注入会议模块的通知服务，两个模块就会互相依赖，编译期即成环。改为由预约模块定义只含接口的通知端口、由会议模块提供实现后，依赖重新变成单向。

第三对端口体现了同一条思路的延伸。会议室的删除保护需要知道该房间是否还存在未来预约或历史预约，而这属于预约模块的数据；若由资源模块直接查询预约表，就会出现“资源 → 预约”与既有的“预约 → 资源”并存的反向依赖。因此改由资源模块定义只读查询端口，预约模块实现，资源模块只依赖自己定义的接口，依赖方向得以保持单向。

在实现上，端口接口只包含方法签名，不出现任何来自实现方的类型，这一点是端口模式能够真正解耦的前提。审批模块的适配器只做参数转换与委托，把状态变更的全部判断（是否处于待审批、是否已开始、是否命中状态机）保留在预约模块内部，从而保证管理动作与用户动作走同一套规则。

除上表所列职责外，本版还在两个模块上补充了新的内部职责：预约模块增加了“审批超时清扫”的定时任务，把时段已结束仍未审批的预约迁为终态并通知预约

人；资源模块增加了独立的只读查询入口，把台账筛选与时段空闲判定与写路径分离，使写入逻辑不受查询语句变化的影响。两处调整都没有新增模块之间的依赖关系：清扫任务通过预约模块已有的生命周期入口与通知端口完成，只读查询入口只依赖资源模块自身的表。

4.5 数据库总体设计

系统数据模型围绕“用户—会议室分类—会议室—预约”主线组织。用户发起预约，预约关联会议室，会议室隶属于分类；分类承载是否需要审批、容量区间、最长预约时长和提前预约天数等规则。预约向下关联审批记录、参会人与出勤、站内通知和会议实际使用记录；操作审计日志相对独立，通过业务对象类型与标识记录管理动作。概要设计阶段只确定主要实体及关系，字段、索引和约束在第 5.8 节数据库详细设计中展开。

4.6 接口总体设计

系统接口遵循统一约定，所有响应都采用同一种结构：状态码、提示信息与数据体三部分，其中状态码为业务语义上的成功或错误标识，提示信息为可直接展示给用户的文字，数据体承载业务数据。前端因此可以用一套逻辑处理全部接口的成败分支，不需要为每个接口单独解析。

错误码按业务域划分，便于快速定位问题来源；需要鉴权的接口统一在请求头中携带访问令牌，由鉴权过滤器解析并写入安全上下文；分页与筛选参数采用可选形式，留空即表示不限制。表 4.4 给出了错误码的分段规则及典型场景。

表 4.4 错误码分段与典型场景

分段	典型错误码	典型场景
通用	VALIDATION_ERROR	请求参数缺失或格式非法， 响应附带出错字段
鉴权	UNAUTHORIZED / FORBIDDEN	未携带或令牌失效、已登录 但无权限执行管理动作
资源 resource	ROOM_UNAVAILABLE / ROOM_CAPACITY_EXCEEDED / ROOM_DELETE_BLOCKED	会议室不可约、参会人数超 出容量、存在引用不可删除

分段	典型错误码	典型场景
预约 reservation	RESERVATION_TIME_CONFLICT / RESERVATION_INVALID_STATE /DUPLICATE_REQUEST	时段冲突、状态不允许该动作、同一请求重复提交
资源缺失	RESOURCE_NOT_FOUND	按标识查询的房间、用户或预约不存在
内部	INTERNAL_ERROR	未预期异常的兜底错误，对外只提示“服务器内部错误”

需要强调的是，业务规则导致的拒绝不会使用“服务器内部错误”。其原因是二者对使用者的含义完全不同：前者是可以通过调整操作解决的正常结果，应当返回请求错误或冲突状态并附带可读原因；后者意味着系统存在缺陷，需要查看日志排查。把两者分开，既让用户知道下一步该怎么做，也让排障时能够快速判断问题是否属于业务范畴。

4.7 系统部署设计



图 4.4 开发环境与生产环境访问链路

系统采用前后端分离的部署结构：前端为静态单页应用，后端提供接口服务，数据库独立部署并被后端访问。开发阶段前端通过开发服务器代理后端接口，避免跨域问题；生产阶段可由反向代理承担静态资源与接口转发的职责。整体访问链路如图 4.5 所示，部署要素与启动顺序如表 4.5 所示。

表 4.5 部署要素与关键配置

组件	技术栈	默认端口	关键配置
前端	Vue 3 + Vite	5173	接口代理指向后端；是否启用模拟数据由环境变量控制
后端	Spring Boot 3 + Java 21	8080	数据库连接、监听端口与访问令牌密钥，其中密钥长度不足时启动即失败
数据库	MySQL 8	3306	库名与账号；表结构由版本化脚本初始化

启动顺序为：先启动数据库，再启动后端服务，最后启动前端开发服务器。后端启动时会对数据库连接与访问令牌密钥做前置校验，密钥长度不足或数据库不可用都会导致启动失败并给出明确提示，而不是在首次请求时才暴露问题。这样做可以减少“服务已启动但接口不可用”的排查时间。

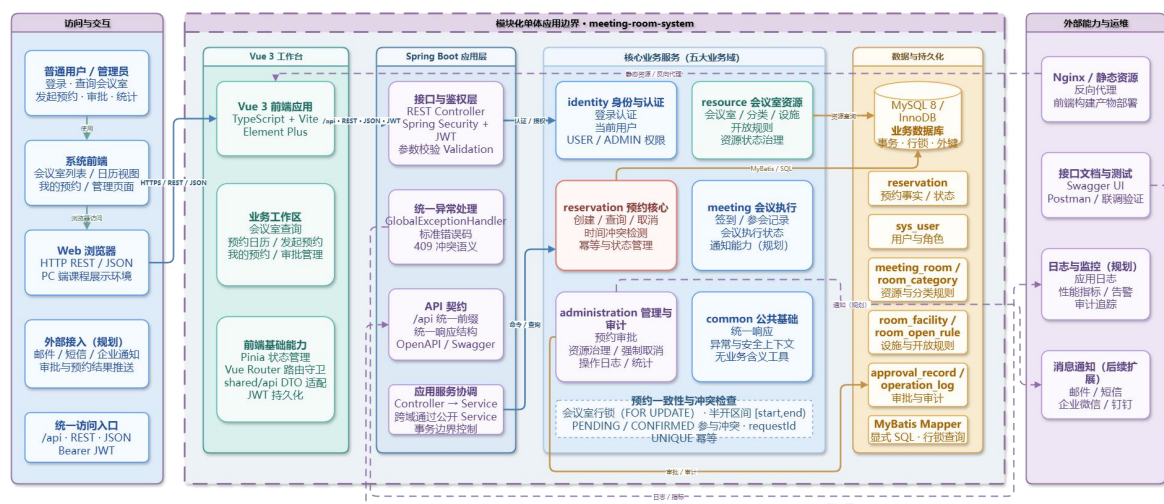


图 4.5 系统总体部署架构

5 系统详细设计

5.1 预约创建流程设计

一次预约的创建遵循如下顺序逻辑：首先取得当前登录用户并做“用户 + 请求标识”的幂等检查；其次跨模块校验预约资格（账号状态、限制期、信用分）；随后解析时段并加锁该会议室的数据行，在锁内再次校验幂等与时间冲突；接着综合校验会议室可用、时长与人数上限、全局时段与提前天数约束；最后按分类决定初始状态并落库，触发跨模块通知。先加锁后查冲突，是从根本上防止并发双重占用的关键顺序。

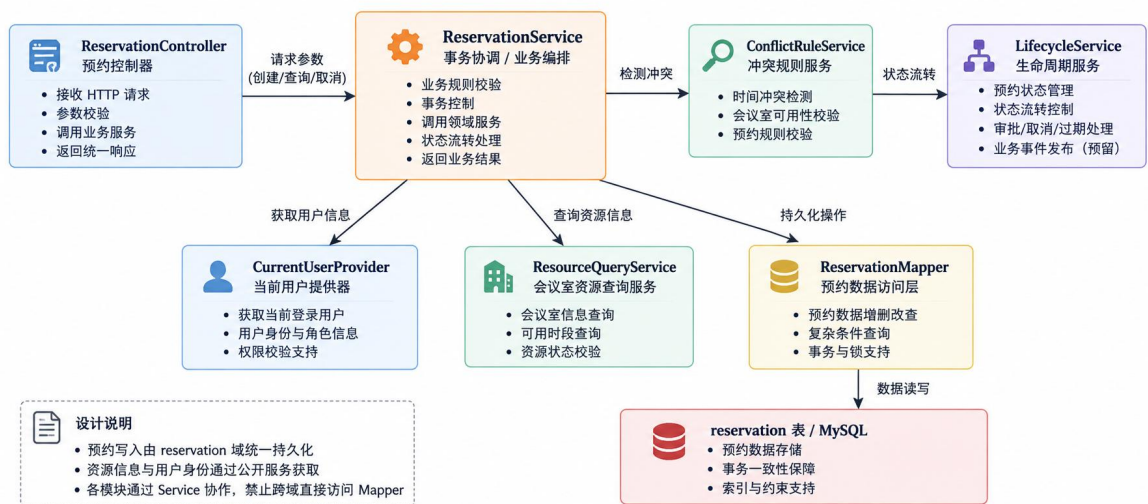


图 3-3 reservation 模块内部结构与调用关系图

图 5.1 预约模块内部结构与调用关系图



图 5.2 预约创建主流程与失败分支

创建流程的步骤顺序遵循两条要求：一是校验按成本由低到高排列，先完成仅依赖内存与单表查询的判断，再做需要加锁与跨表扫描的判断，避免为一个必然失败的请求提前付出加锁代价；二是把状态写入放在全部校验通过之后，使任何一步失败都不会留下中间态数据。完整步骤与失败分支如图 5.2 所示。

第②步的幂等检查以“用户 + 请求标识”为键：请求标识由前端在每次提交时生成，网络重试会沿用同一个标识。若该键已存在，则不再重复创建，直接返回首次处理的结果，用户看到的是同一条预约，而不是两条内容相同的记录。这一处理针对的是最常见的“网络抖动后用户重复点击提交”场景。

第⑤步对会议室的数据行加锁是整个流程中唯一可能阻塞的位置。加锁之后必须重新执行冲突检测，而不能复用加锁之前的检测结果：两个并发请求可能在加锁前都通过了检测，只有串行进入临界区后再查一次，才能保证后到者看到前者的写入并被拒绝。这一“加锁 + 复查”的组合，是并发下不出现双重占用的关键。

改期流程沿用同一套校验，唯一的差别是冲突检测需要排除被修改的预约自身。如果不做排除，该预约会与自己原来的时段判定为冲突，导致任何改期请求都失败。

这一细节在实现中通过给冲突检测传入“需排除的预约标识”解决，而不是为改期单独写一套判定逻辑。

5.2 预约状态机设计

预约状态共有四种：待审批、已确认、已驳回、已取消。初始状态由分类的“是否需要审批”决定——需要审批则进入待审批（先占时段待批），免审则直接已确认。待审批可经审批通过变为已确认，或经驳回变为已驳回；待审批与已确认均可被取消；已驳回与已取消为终态。所有状态迁移都由统一的状态机裁决，避免状态被随意改写。

四种状态各自的含义、是否占用时段以及可迁移性如表 5.1 所示。其中“是否占用时段”直接决定冲突检测的统计范围：待审批虽然尚未获得批准，但它已经是一次明确的占用承诺，因此在冲突检测中被计入，避免同一时段被重复承诺给两个人。

表 5.1 预约状态及其含义

状态	含义	是否占用时段	可迁移性
待审批 PENDING	已提交且需要审批，尚未获得批准	占用	可通过审批、驳回或取消迁移
已确认 CONFIRMED	已生效的预约	占用	可取消
已驳回 REJECTED	审批未通过，为终态	不占用	不可再迁移
已取消 CANCELLED	用户取消或管理员强制取消，为终态	不占用	不可再迁移

状态迁移的全部规则集中在状态机中实现，包含两类入口：一是根据会议室分类计算初始状态，需要审批的分类进入待审批，免审批的分类直接确认；二是根据事件与当前状态计算目标状态。系统不为创建预约伪造一条“空状态到某状态”的迁移，而是把初始状态的计算单独表达，使状态机只负责“已存在的预约如何变化”这一件事，语义更清晰。

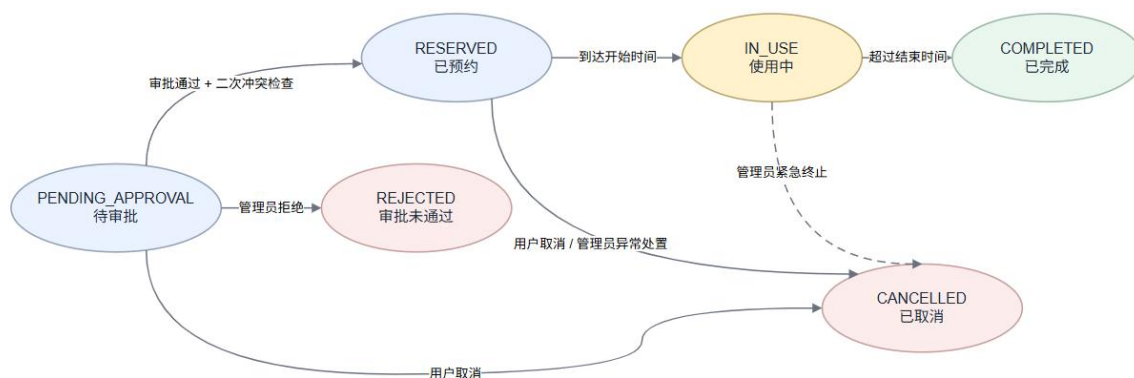


图 5.3 预约状态机与迁移条件

状态机还处理了一个容易被忽略的情形：改期后的目标状态可能发生变化。若改期后的会议室属于需要审批的分类，改期结果应重新回到待审批，而不是保持原来的已确认；反之则直接确认。因此状态机在改期事件上额外接收“目标会议室是否需要审批”这一参数，由它统一决定目标状态，避免把这段判断散落在业务代码里。

5.3 时间冲突检测设计

系统把时间段抽象为区间值对象。设新预约时段为 $[s1, e1)$ ，已有预约时段为 $[s2, e2)$ ，当且仅当 $s1 < e2$ 且 $e1 > s2$ 时判定两者发生冲突。该判定采用“含起点、不含终点”的半开区间语义，因此首尾相接（如 10:00—11:00 与 11:00—12:00）不构成冲突。典型情形如图 5.4 所示。



图 5.4 时间冲突判定的典型情形

判定式在数据访问层以一条带条件的计数查询实现，只统计处于待审批与已确认两种状态的记录，已取消与已驳回的记录不参与统计。这样处理之后，“取消预约”这一操作会自动释放时段，不需要额外写一段“清理占用”的逻辑，也不会出现“预约已取消但仍无法被他人预约”的问题。

需要特别说明的是，冲突检测不能只依靠数据库唯一约束来完成。唯一约束约束的是“键的取值不能重复”，而本项目要排除的是“时间段的重叠”，两者并不等价：同一房间在一天内可以有 multiple 互不重叠的预约，无法用一条唯一的索引表达。因此系统采用“应用层判定 + 行锁串行化”的组合：由判定式负责语义正确，由行锁负责并发正确，两者缺一不可。

当判定为冲突时，系统不只返回“时段已被占用”，而是同时返回冲突会议的主题、占用时段与预约人。这样做是因为单一结论无法指导用户下一步行动，而给出具体信息后，用户可以直接判断是改时间还是换会议室，减少反复尝试的次数。

5.4 并发与幂等设计

系统在并发正确性上采用三重保护：其一，创建预约时先对会议室数据行加悲观锁，使同一房间的并发创建请求串行化，后到者在锁释放后查冲突会被拒绝；其二，

以“用户 + 请求标识”建立数据库唯一约束实现幂等，前端重试不会产生重复预约，唯一键冲突时回查已有记录且不回滚事务；其三，预约数据携带版本号，取消与改期使用“期望状态 + 期望版本号”进行条件更新，未影响任何行即视为失败，防止并发覆盖。

并发正确性由三重保护共同承担，分别对应三类不同的并发问题，如图 5.5 所示。三者作用的层次并不相同：行锁作用于“同一房间的多个请求”，幂等键作用于“同一用户的重复请求”，乐观锁作用于“同一条记录的多处修改”，因此它们不是重复的保险，而是各自覆盖一类场景。



图 5.5 并发正确性的三重保护

行锁是悲观策略，在创建预约前对会议室数据行加锁，使同一房间的并发创建请求被迫排队。它的代价是并发度下降，但由于锁的范围仅限于单行、持有时间仅限于一次校验与写入，在本系统的使用规模下不会成为瓶颈。相较之下，若不加锁，两个请求可能同时通过冲突检测，产生双重占用，而这类错误在事后极难被用户察觉。

幂等键是数据库层面的去重：以“用户 + 请求标识”建立唯一约束，重复请求在写入时会命中唯一键而失败，系统据此识别出这是一次重复提交，转而返回首次的处理结果。这一保护之所以必要，是因为用户重复点击或客户端自动重试并不受服务端控制，只靠前端禁用按钮无法彻底避免。

乐观锁用于保护对已有预约的修改。预约记录带有版本号，修改时以“版本号 + 预约标识”作为更新条件，若版本已被其他操作改变，更新影响行数为零，系统据此提示用户重新加载后重试。相比直接加锁读改，这一方式在并发冲突较少的场景下开销更低，也不会因为忘记释放锁而长时间占用资源。

5.5 时间与预约窗口约束设计

可预约时段由三套相互独立的约束共同决定。第一套是“业务时钟”，系统以可配置的业务时间作为“当前时间”的基准，便于在演示与测试中灵活推进时间，而登录令牌仍采用物理时间以保持登录态稳定。第二套是“全局预约窗口”，以每天的起止分钟数限定可预约的时钟范围，例如仅允许 08:00 至 22:00 之间的时段。第三套是“提前天数”，由会议室分类规定可提前预约的最大天数，超出即拒绝。三者的作用范围不同、彼此正交，共同保证预约时间既符合管理规则，又落在合理范围内；校验失败时返回明确提示，便于用户调整。

除冲突之外，预约还受若干时间类约束的限制。这些约束的取值来源并不相同：一部分来自会议室分类（体现管理策略），一部分来自全局配置（体现组织统一规定），还有一部分来自系统当前时间。各约束的取值来源与不满足时的结果如表 5.2 所示。

表 5.2 预约相关的时间与容量约束

约束	取值来源	判定时机	不满足时的结果
开始时间须晚于当前时间	业务时钟	创建、改期	拒绝，提示预约时间已过
须落在全局可预约时段内	预约窗口配置	创建、改期	拒绝，提示不在可预约时段
须满足提前预约天数	会议室分类	创建、改期	拒绝，提示超出可提前预约天数
单次时长不超过上限	会议室分类	创建、改期	拒绝，提示超出单次最长时长
参会人数不超过容量	会议室	创建、改期	拒绝，提示超出会议室容量
时段不得重叠	已有预约记录	创建、改期	拒绝，并返回冲突的具体明细

系统对“当前时间”的处理采用可配置的业务时钟：业务判断读取业务时钟，而登录令牌的有效期仍使用物理时间。这样处理的目的是有两个：一是在演示与测试时可

以推进时间，以观察预约开始、缺席判定等依赖时间的逻辑，而不必真的等待；二是避免调整业务时钟后导致所有登录态失效。把两类时间的用途分开，是这一设计能够成立的前提。

各项约束的判定顺序按照“先易后难、先通用后具体”排列：先判断时间本身的合法性（是否已过、区间是否有效），再判断是否落在允许的窗口与提前天数内，最后判断容量与冲突。这样排列的好处是错误提示更有指向性——例如一个既超出提前天数又与已有预约冲突的请求，用户会先看到“超出可提前预约天数”，从而先解决时间选择问题，而不是被冲突提示误导去更换会议室。

5.6 审批与通知设计

审批动作由审批模块发起，经由“预约生命周期端口”委托给预约模块唯一的权威入口完成状态变更，并写入审批记录。通知则由预约模块在状态变化时通过“通知端口”交由会议模块完成：用户动作类通知每次送达，调度类通知使用确定性去重键以保证可重复执行而不重复发送；待审批类通知会广播给所有活跃管理员，新增管理员自动覆盖。通知写入采用“插入忽略 + 唯一键”实现天然去重。

审批侧的三种动作与预约状态迁移之间是一一对应的，并且全部经预约模块的统一入口完成，审批模块自身不直接更新预约状态。每个动作在改变状态的同时写入审批记录与操作日志，前者用于回答“这条预约经历过哪些审批”，后者用于回答“谁在什么时候做了什么管理动作”。三者的对应关系如表 5.3 所示。

表 5.3 审批动作、状态迁移与记录

审批动作	允许的前置状态	目标状态	同时写入的记录
审批通过	待审批	已确认	审批记录（审批人、动作、备注）与操作日志
驳回	待审批	已驳回	审批记录（含驳回理由）与操作日志，并通知预约归属人
强制取消	待审批、已确认	已取消	审批记录与操作日志（含来源地址），并通知预约归属人

通知由预约模块在状态变化时通过通知端口发起，由会议模块完成实际的存储与推送。为避免重复调度（例如定时任务重跑）造成重复消息，每条通知都带有一个由

业务事件生成的去重键，并与接收人组成唯一约束：同一事件对同一用户只会存在一条记录。对“预约创建成功”这类由用户动作直接触发的通知，去重键由调用方置空，每次动作都送达，符合用户的心理预期。

通知的接收人分为两类：与预约直接相关的个人（创建人、归属人、参会人）按用户标识定向发送；需要管理员处理的“有待审批预约”则广播给全部处于启用状态的管理员。后者采用按角色查询当前用户列表的方式实现，因此新增管理员会自动覆盖，无需在代码中登记接收人名单。

5.7 信用与预约资格设计

预约资格按“一票否决”顺序判定：用户不存在、账号被禁用、处于限制期或黑名单、信用分低于门槛，任一不满足即拒绝预约。信用规则约定默认分、预约门槛分、自动拉黑阈值与自动限制天数；低于自动拉黑阈值则系统自动限制，恢复达标且为系统加黑则自动解除，人工加黑须人工解除，并配套违规记录类型用于审计。

预约资格的判定采用一票否决的方式，四项条件依次判断，任一项不满足即拒绝预约，并给出对应的原因。把判定集中在一处，可以避免“新建时校验、改期时漏校验”的不一致。判定项与顺序如表 5.4 所示。

表 5.4 预约资格判定项与顺序

顺序	判定项	不满足时的结果
1	用户是否存在	拒绝，返回资源不存在
2	账号状态是否为启用	拒绝，提示账号当前不可用
3	是否处于限制期或黑名单	拒绝，提示处于限制期，并给出可预约的时间
4	信用分是否达到预约门槛	拒绝，提示信用分不足

信用分的变动分为系统自动与人工两类：系统在缺席判定等场景下自动扣减，人工则由管理员执行加分或扣分并填写原因。为避免两者混淆，系统在记录原因上加以区分，使管理员在查看信用明细时能够一眼分辨某次扣分是规则触发还是人工操作。当信用分低于自动限制阈值时，系统自动为用户设置一个限制期，期满且信用分恢复后自动解除，不需要人工干预。

5.8 数据库详细设计

系统数据以若干核心实体为中心：用户、会议室分类、会议室、预约、审批记录、操作日志、参会人与出勤、会议实际使用记录、通知、时间与窗口配置等。其中“分类”承载“是否需要审批、容量区间、单次最长时长、提前天数”等业务规则；“预约”是核心表，记录请求标识（幂等）、房间、起止时间、人数、状态与版本号（乐观锁）；“通知”通过去重键保证不重复。各表之间通过外键形成一致的数据关系，完整实体关系如图 5.6 所示。核心数据表及其作用如表 5.5 所示。

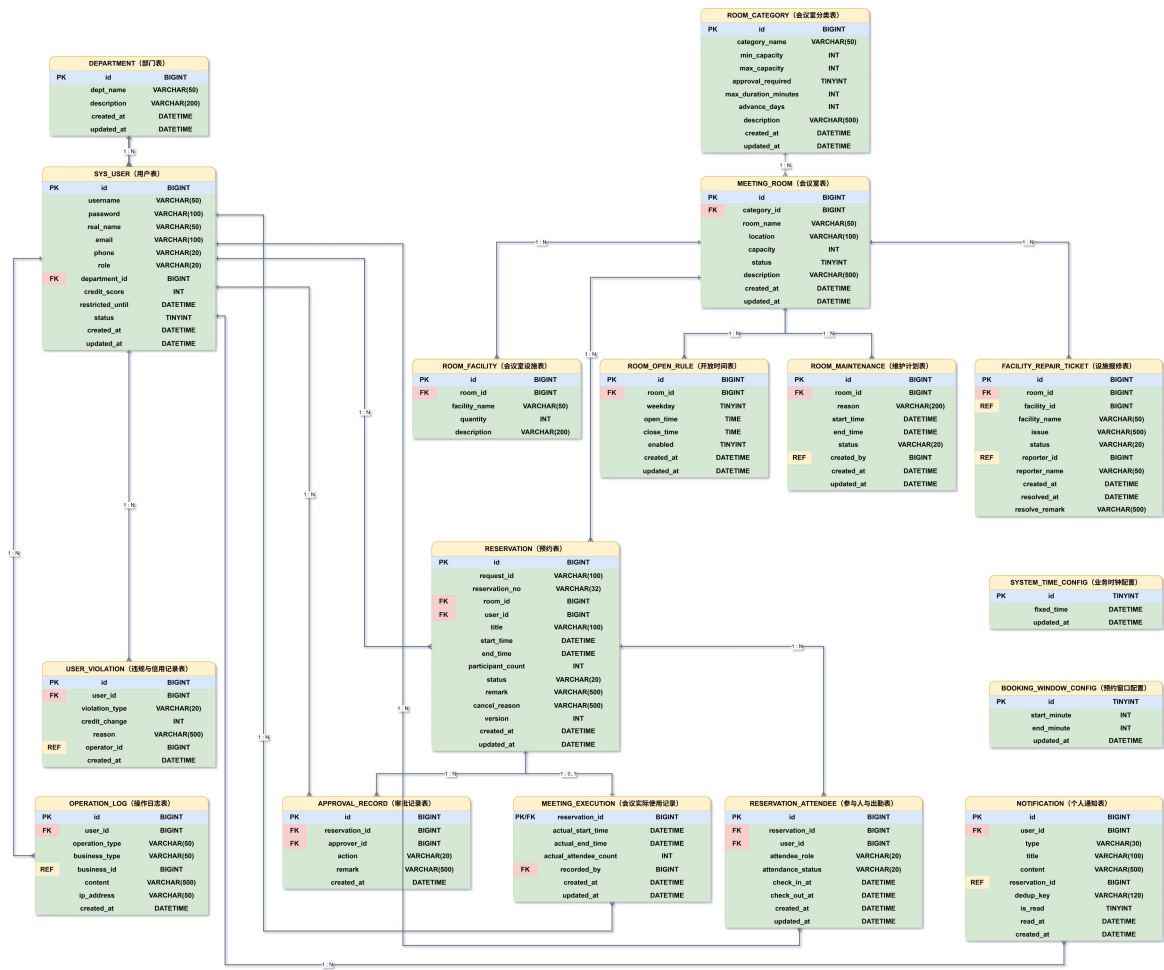


图 5.6 数据库完整实体关系图（17 张业务表）

表 5.5 核心数据表及其作用

表名	核心作用
sys_user	用户与资格三要素（状态/信用分/限制期）
room_category	分类规则：是否审批、容量区间、最长时长、提前天数

表名	核心作用
meeting_room	具体会议室：所属分类、名称、容量、状态
reservation	核心预约：幂等键、房间、起止时间、人数、状态、版本号
meeting_execution	会议级实际使用记录：实际起止时间、实际参会人数与登记人（与预约一对一）
approval_record	审批行为历史
operation_log	管理动作审计（含 IP）
reservation_attendee	参会人与出勤
notification	个人通知，含去重键与已读标记
system_time_config / booking_window_config	业务时钟与全局可预约时段

各核心实体之间的关系如图 5.7 所示。预约处于关系的中心：向上关联会议室与用户，向下派生参会人与出勤记录、审批记录与站内通知；会议室隶属于分类，分类承载审批与时间类规则；操作审计日志相对独立，通过业务对象类型与标识与其他实体关联。

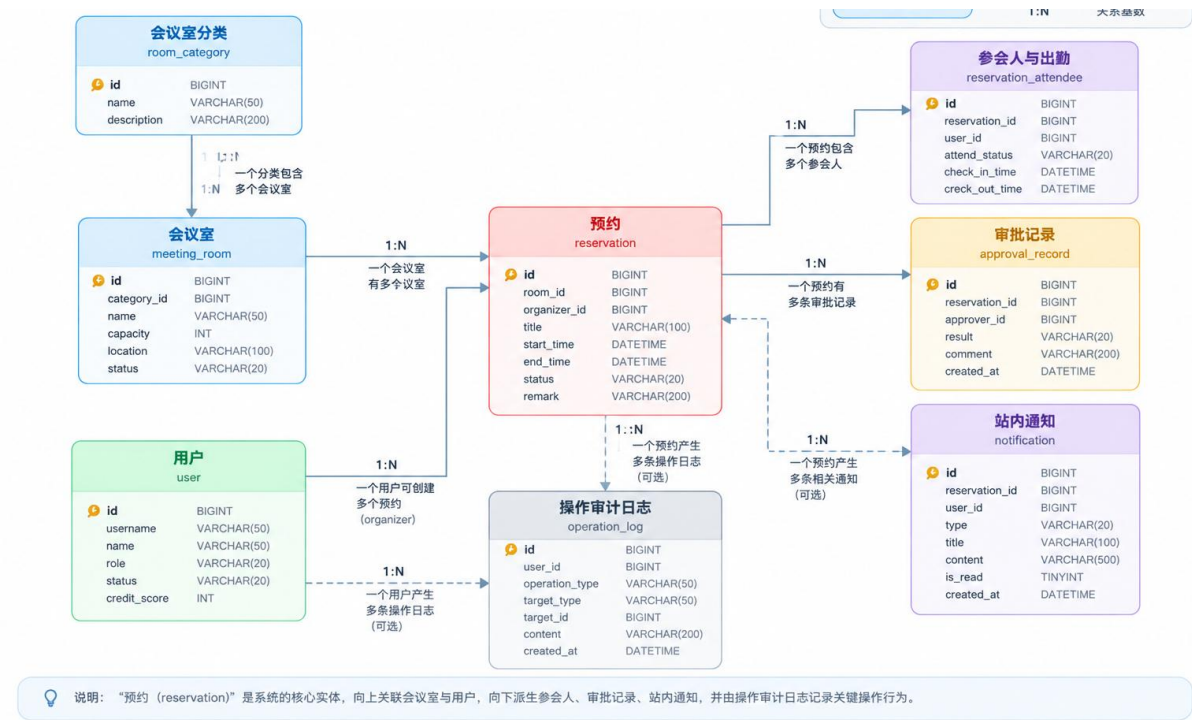


图 5.7 核心实体关系与一对多关联

在表结构之外，索引的设计同样围绕核心查询展开。系统并未追求为每个字段建索引，而是针对冲突检测、幂等去重、通知去重三类正确性相关的查询，以及列表与统计两类高频查询建立索引，如表 5.6 所示。

表 5.6 关键索引及其服务的查询

索引	所在表	主要服务的查询
idx_reservation_room_status_start	reservation	冲突检测：按房间、状态与开始时间定位可能重叠的预约
uk_reservation_user_request	reservation	幂等去重：同一用户的同一请求标识只允许存在一条
idx_reservation_user_start	reservation	我的预约：按用户与时间排序查询
idx_reservation_status_period	reservation	运营统计：按状态与时间段聚合预约情况
uk_notification_dedup	notification	通知去重：同一用户的同一业务事件只保留一条
idx_notification_user_read	notification	通知中心：按用户与已读状态分页查询
uk_attendee_reservation_user	reservation_attendee	参会人唯一性：同一预约中同一用户只出现一次
idx_approval_reservation	approval_record	审批历史：按预约查询审批记录
idx_operation_business_created	operation_log	操作审计：按业务对象与时间检索审计记录

其中三个唯一索引同时承担了业务规则的角色：预约编号的唯一索引保证编号不重复；幂等键的唯一索引把“避免重复提交”从应用层判断变成了数据库层面的硬约束；通知去重键则使通知的生成可以安全地重试。把这类规则放到数据库层实现，好处是不依赖调用方是否记得判断。

5.9 预约状态迁移汇总

下表汇总了预约状态的主要迁移路径及其触发条件。

表 5.7 预约状态迁移

当前状态	目标状态	触发条件
新建	待审批	分类要求审批
新建	已确认	分类免审
待审批	已确认	管理员审批通过
待审批	已驳回	管理员驳回
待审批	已取消	用户取消或管理员强制取消
已确认	已取消	用户取消或管理员强制取消
已驳回	不可迁移	终态
已取消	不可迁移	终态

表 5.8 给出的是状态机层面的合法迁移，但在实际执行一次迁移之前，还需要通过若干业务守卫。以取消预约为例，状态机允许“待审批或已确认迁移到已取消”，而业务上还要求该预约尚

未开始；以改期为例，除目标状态需要重算之外，还要重新执行资格、容量、时间窗口与冲突检测四项校验。这类守卫负责回答“这一次操作在当前时刻是否允许”，与状态机负责的“这种迁移是否合法”是两个不同层次的问题，因此系统把两者分开实现，避免把时间判断写进状态机。

把两层次分开还有一个实际收益：状态机是纯逻辑判断，不依赖数据库与系统时间，因此可以用一组输入输出直接验证迁移矩阵是否正确；而守卫依赖数据库与时间，通过接口测试验证。测试职责随之清晰，出现问题时也能快速判断是规则写错还是数据异常。

5.10 异常处理与错误码设计

系统采用统一异常处理机制：业务异常被映射为恰当的 HTTP 状态码，例如时间冲突返回冲突状态（409）、参数与业务规则校验失败返回请求错误（400）、越权访问返回禁止（403）；对于未预期的异常，由全局兜底返回服务器错误（500），避免异常堆栈直接暴露给前端。前端则根据错误类型给出清晰提示，提升系统可用性。

系统使用全局异常处理统一收敛各类异常，其映射规则如表 5.9 所示。映射的关键判断依据是“这个异常对使用者意味着什么”：业务规则导致的拒绝属于正常结果，返回可读原因并附带业务错误码；参数与请求体问题属于调用方式错误，返回具体字段或格式提示；只有未预期的异常才映射为服务器内部错误，并且不向前端暴露异常堆栈与数据库细节。

表 5.9 全局异常处理的映射规则

异常类型	映射结果	是否记录日志
业务异常	按业务错误码映射为 400 / 403 / 404 / 409，并返回可读提示	记录简要信息
参数校验失败	400，并附带出错字段与原因	不记录
请求体不可解析	400，提示请求体格式无效	不记录
无权限访问	403，提示无权限执行该操作	记录
其他未预期异常	500，对外仅提示服务器内部错误	记录完整堆栈
查询参数类型不匹配	400，提示“参数格式无效：参数名”	不记录

这种分层的另一个作用体现在排错效率上。当用户报告“提交失败”时，只需查看返回的状态码即可初步判断方向：400 与 409 说明请求本身或业务规则存在问题，

应当从操作或规则入手；403 说明是权限问题；500 才需要进入服务端日志排查。把这一判断前移到响应码，可以减少大量“先查日志再发现是业务规则”的往返。

5.11 关键设计权衡

在设计中做了若干权衡：其一，状态采用字符串而非数据库枚举，换取更好的可扩展性与排障便利；其二，为冲突检测建立了“房间 + 状态 + 开始时间”的索引，兼顾写入与查询性能；其三，通知采用“插入忽略 + 唯一键”的去重策略，使调度类任务可安全重复执行；其四，业务时钟与登录令牌分别采用业务时间与物理时间，兼顾演示灵活性与登录态稳定。

第一处权衡是状态值采用字符串而不是数据库枚举。字符串的可读性更好，查询与排障时不需要对照编码表；代价是失去了数据库层面的取值约束，因此系统通过迁移脚本为状态列补充了取值检查，把约束放在数据层，兼顾可读性与一致性。

第二处权衡是审批与免审的规则位置。规则挂在会议室分类上，而不是逐间会议室单独配置，好处是规则数量可控、便于统一调整，代价是同一分类下的房间无法拥有不同的审批策略。在“同一类房间管理策略一致”的现实前提下，这一取舍是合理的；若未来出现例外需求，可以在分类之下增加覆盖配置，而不必推翻现有结构。

第三处权衡是通知的投递范围。系统只实现站内通知，而没有接入邮件或短信。这样做的收益是不引入外部依赖与失败重试链路，代价是用户必须登录系统才能看到通知。考虑到系统的使用场景是组织内部且高频次使用，站内通知已能满足“状态变化可感知”的要求；若需要触达离线用户，可在现有通知表之上增加一个投递渠道层，而不改动业务侧的通知生成逻辑。

第四处权衡是把复杂查询交给可控 SQL，而不是在应用层拼装。冲突检测、可约查询与运营统计都用一条带条件的 SQL 完成，好处是判定过程可在数据库中直接验证，也便于利用索引；代价是 SQL 与表结构耦合较紧，表结构变更时需要一并检查相关语句。为此系统把所有涉及核心判定的语句集中在数据访问层，使这种影响范围是可见、可枚举的。

5.12 审批超时自动失效设计

待审批的预约存在一个天然的“过期点”：当时段已经结束，审批就失去了意义。此时通过会与“预约已开始，不能审批”冲突，驳回同样被拦截，强制取消也会被“预约已结束”挡住；如果不加处理，这类预约会长期堆积在待审批列表里，既让管理员的列表失真，也让预约人始终拿不到最终结果。这是一个典型的“状态机本身没有错，但缺少退出路径”的问题。

系统的处理方式是增加一个定时清扫任务：每分钟扫描一次“状态为待审批且结束时间早于当前业务时间”的预约，把它们迁移为已驳回，并向预约人推送一条驳回通知，理由写明超时。清扫只做迁移与通知两件事，涉及的业务判断全部复用预约模块已有的生命周期入口，不另写一套状态修改逻辑。

一个需要专门说明的取舍是：系统没有为“超时失效”新增状态，而是复用了已有的“已驳回”终态。这样做的好处是冲突统计口径、看板置灰展示、我的预约的状态文案都不需要任何改动——对冲突检测而言，“已驳回”与“超时失效”都不占用时段；对用户而言，两者都表示这条预约已经作废。代价是驳回原因的区分只能落在通知与备注中，而不是状态本身；在本场景下这一代价是可以接受的。另一处取舍是审计：审批记录与操作日志的操作人字段是必填的人工字段，系统定时动作不伪造操作人，因此这两张表不写入记录，超时失效的事实通过预约自身的状态与通知体现。

表 5.10 审批超时自动失效的设计要点

设计点	做法	理由
触发方式	定时任务每分钟执行一次，启动后延迟一段时间再首轮执行	与会议执行的定时任务错开启动时刻，避免重启后同一时刻两个任务同时抢行
扫描条件	状态为待审批，且结束时间早于当前业务时间	只有已经过期的待审批才失去审批价值，尚未开始的不应被清理
目标状态	迁移为已驳回，复用既有终态	不新增状态，冲突口径、置灰展示与前端文案均无需改动
通知	向预约人推送驳回通知，理由写明“审批超时”	结果必须让当事人知道，且原因要可解释
审计	不写审批记录与操作日志	两张表的人员字段是必填的人工操作字段，系统动作不伪造操作人

设计点	做法	理由
幂等与容错	迁移前复查状态；单条失败只记日志，不中断本轮	重复调度不产生副作用，一条异常不影响同批其他预约

5.13 周期性预约批量创建设计

例会类会议通常每周在同一时段重复占用同一间会议室，逐次手工创建既费时又容易漏掉某一周。因此系统在创建预约的入口上增加了“重复”选择：可取 2 至 8 周，缺省为 1（单次会议）。系统以用户首次选择的日期与时段为锚点，按每周同一时段展开相应场次，每场生成独立的预约记录。

批量创建最需要处理的是“部分成功”。如果逐场校验、逐场写入，就可能出现前几周成功、中间某一周因为冲突或超出提前天数而失败的情况，用户得到的是几场零散的预约，还需要自己判断剩下几周该怎么办。因此实现上采取“先整批校验、再整批写入”：所有场次全部通过校验后才开始落库，任何一场不满足规则都会导致整批拒绝、零落库，错误提示中给出冲突所在的具体场次与占用时段；落库过程由事务兜底，不会留下中间状态。

另一个关键口径是提前天数只约束首场。提前天数是针对“这一次预约意图”的限制，如果逐周校验，一个 4 周的周期性预约中靠后的周次必然超出可提前天数，功能将不可用。因此除提前天数外的其他规则（全局可预约时段、单次最长时长、容量与冲突检测）仍然逐周各自校验，因为它们反映的是那一周的实际可用情况，不能被首场的结果代表。

在标识与幂等上，首场沿用前端生成的原始请求标识，后续场次在标识后追加周次后缀，既保持了“同一请求重复提交返回首次结果”的幂等短路能力，又满足“用户+请求标识”的唯一约束。各场次之间没有主从关系：可以分别查看、改期或取消，取消其中一场不会影响其余各周。

表 5.11 周期性预约的规则与取舍

设计点	做法	理由
参数范围	2 至 8 周，缺省为 1（单次会议）	覆盖常见例会周期，同时限制单次请求的写入量
展开方式	以首场开始时刻为锚点，每场增加 7 天，时段完全一致	例会通常固定在同一星期与时段，语义直观

设计点	做法	理由
校验策略	先整批校验，全部通过后再逐场写入	避免出现“前几周成功、中间一周失败”的部分成功
提前天数	只约束首场，后续周次不再校验提前量	提前量针对同一次预约意图；逐周校验会使靠后的周次必然越界
其他规则	开放时段、单次最长时长、容量与冲突逐场各自校验	每一周面对的实际情况不同，不能被首场结果代表
幂等键	首场沿用原始请求标识，后续场次追加周次后缀	兼顾重试短路与唯一约束
失败提示	指出冲突所在的具体场次与占用时段	用户据此知道该调整哪一周

5.14 运营统计指标口径设计

运营看板的价值取决于口径是否说得清。如果把统计放在前端、由页面拿到明细后自己算，同一个指标在不同页面上就可能得出不同结果。因此系统把口径固定在服务端：先用数据库聚合查询求出基数（条数、分钟数、人次），再由服务层换算成比率，前端只负责展示。

本版在原有的“预约总量、取消率、会议室使用时长与预约次数、高峰时段”之外，补齐了验收点名的三项指标：各会议室使用率、日均会议时长与爽约率。使用率的分母取“统计天数 × 每日开放时长”，其中的每日开放时长来自全局可预约时段配置，因此调整开放时段后使用率会随之变化，不需要改代码；实际占用时长按预约时段与统计区间的交集计算，跨越统计区间边界的部分会被正确截断，不会因为一条预约横跨区间端点而把时长算多。

爽约率取“缺席人次 ÷ 应到人次”。分子来自出勤表中被判定为缺席（NO_SHOW）的记录，分母取同一范围内已确认会议的参会人行数；两者都限定在统计区间内，口径一致。把分母定义为“应到人次”而不是“预约场次”，是因为爽约的主体是参会人而不是会议——一场 20 人的会议与一场 2 人的会议，缺席一人对履行率的含义并不相同。

表 5.12 运营统计指标的计算口径

指标	计算口径	分母来源
预约总量与取消率	统计区间内的预约条数；取消数 ÷ 总量 × 100%	预约表中落在统计区间内的记录

指标	计算口径	分母来源
各会议室使用率	$\frac{\text{已确认预约的实际占用时长}}{(\text{统计天数} \times \text{每日开放时长})} \times 100\%$	全局可预约时段配置与统计区间整日数
日均会议时长	$\frac{\text{全部会议室已确认时长之和}}{\text{统计天数}}$	统计区间的整日数
爽约率	$\frac{\text{缺席人次}}{\text{应到人次}} \times 100\%$	统计区间内已确认会议的出勤记录行数

5.15 会议实际使用记录设计

预约表中的开始时间与结束时间表达的是“计划时间”：它同时是冲突检测、审批展示与运营统计的依据。但一场会议实际开了多久、真正到了多少人，与计划值往往并不一致——会议可能提前结束，也可能因为议程压缩而缩短。如果直接把实际时间写回预约表，就会污染既有口径：历史时段的偏差会被当成“计划变更”处理，事后也无法区分“当初就约错了”与“按计划约了、但实际没这么执行”。

因此系统把实际执行结果放在一张独立的一对一表 `meeting_execution` 中，其主键就是预约标识：一场预约最多只有一条记录，字段只保留实际开始时间、实际结束时间、实际参会人数、登记人与两个时间戳（首次登记、最后修正）。把主键本身设计成外键，一方面是“一对一”这一业务事实最直接的表达，另一方面也让“重复登记”天然退化为一次更新，不需要额外的唯一约束或应用层查重。

写入用一条“插入或更新”的语句完成：重复登记视为对原记录的修正而不是新增，接口因此是幂等的，网络重试或页面重复点击都不会留下第二条记录。记录同时保留首次登记时间与最后修正时间，用来区分“什么时候第一次确认”与“最后一次改了什么”，在需要回溯登记过程时有实际价值。

登记入口只对已确认且已经结束的会议开放，这一条件在服务端与前端各校验一次：服务端把它作为业务守卫，前端把它作为按钮的显示条件，二者使用同一个判断，避免出现“界面能点、接口拒绝”的错位。其余校验包括三项内容非空、实际结束时间必须晚于实际开始时间、实际参会人数不能为负数，以及实际结束时间不能晚于当前业务时间；最后一项按业务时钟而非物理时钟判断，使演示与验收可以通过推进业务时间来构造已经结束的场景。

权限上，只有预约组织者或管理员可以登记与查看实际使用记录。这一限制来自“实际执行结果属于事后确认”的定位：参会人只需要知道自己的出勤状态，不必也不应替组织者确认整场会议的实际情况。

需要说明的是，这一设计并不取代此前的出勤统计。逐人的签到与签退仍落在参会人表上，用于出勤明细与爽约率计算；实际使用记录补充的是会议级的整体口径。两者并存而不是互相覆盖，正是为了同时回答“谁到了”与“这场会实际怎么样”这两个不同的问题。设计要点汇总如表 5.13 所示。

表 5.13 会议实际使用记录的设计要点

设计点	处理方式	原因
存储位置	独立表 <code>meeting_execution</code> ，主键即预约标识	不改写预约的计划时间，冲突检测与统计口径不受影响
一对一约束	以预约标识作为主键	结构本身保证“一场预约最多一条记录”，无须额外唯一约束
幂等写入	单条“插入或更新”语句，重复登记即修正	重试或重复点击不会产生第二条记录
时间口径	实际起止时间只写入本表，不覆盖预约时间	计划与实际分离，历史可追溯
登记校验	非空、结束晚于开始、人数非负、结束不晚于业务时间	拒绝无意义或指向未来的执行结果
权限控制	仅预约组织者或管理员可登记与查看	实际执行结果属于事后确认
与出勤统计的关系	出勤逐人留痕，本表记录会议级口径	分别回答“谁到了”与“这场会实际如何”

6 系统实现

6.1 开发环境与工具

- 后端：JDK 21、Spring Boot 3、Maven、MySQL 8、MyBatis；
- 前端：Node.js、Vue3、Vite、TypeScript、Element Plus、Pinia；
- 协作：Git 版本控制，VSCode 一键启动前后端任务。

开发环境的版本需要在团队内保持一致，否则容易出现“本地可运行、他人机器报错”的问题。本项目统一使用的技术与工具版本如表 6.1 所示。

表 6.1 开发环境与工具清单

技术 / 工具	版本	用途
JDK	21	后端编译与运行
Maven	3.x	后端依赖管理与构建
Spring Boot	3.3.5	后端应用框架与依赖注入
MyBatis	3.x	数据访问与 SQL 映射
MySQL	8.0	关系型数据库
Node.js	18 及以上	前端依赖安装与开发服务器
Vue 3 + Vite	3.x	前端框架与构建工具
Git	—	版本控制与协作

6.2 后端分模块实现

后端以六大模块为单位组织实现，各模块的主要实现内容如下：

- 身份模块：实现了用户登录鉴权、信用分维护与预约资格判定，并维护部门与用户状态；
- 资源模块：实现了会议室与分类的维护、分类容量区间校验、按星期的开放规则与报修工单；
- 预约模块：实现了预约创建全流程、状态机、半开区间冲突检测与“行锁 + 幂等 + 乐观锁”三重并发保护，并提供预约查询与生命周期入口；
- 会议模块：实现了参会人与签到签退，以及通知的存储、去重与读取；
- 审批模块：实现了审批、驳回、强制取消，并落审批记录与操作日志，提供运营统计；

- 公共模块：实现了统一响应、错误码、全局异常映射、鉴权过滤器、业务时钟与预约窗口配置。

公共模块为其他五个模块提供基础设施。统一响应封装规定所有接口的返回结构；错误码枚举集中定义业务错误；全局异常处理器把异常映射为恰当的状态码与可读提示；鉴权过滤器负责解析令牌并写入安全上下文，安全配置负责确定哪些路径需要鉴权；业务时钟与预约窗口服务提供可配置的时间基准与全局可预约时段。这些能力被各业务模块共用，因此不放在任何业务模块内部。

身份模块实现了登录鉴权与用户治理两条线。登录服务校验账号密码后签发访问令牌，令牌中携带用户标识与角色；用户治理部分包括用户信息与状态的维护、部门管理、信用分的加分与扣分、违规记录的登记，以及预约资格的统一判定。资格判定被封装为一个独立服务，供预约创建与改期共用，避免两处各写一套判断。

资源模块围绕会议室台账展开。台账查询服务支持按位置、容量上下限、设备与名称关键字的组合筛选，并在筛选设备时采用存在性判断，避免同一会议室因登记多台设备而在结果中重复出现；可约查询服务按给定时段返回未被占用的会议室；管理服务负责会议室与分类的新增、修改与状态变更，并在删除时执行引用检查；维护与工单部分负责登记维护计划与报修工单。

预约模块承载系统的核心逻辑，由四个组件分工：预约服务负责创建与改期的完整校验流程与事务边界；状态机负责唯一的迁移矩阵与初始状态计算；查询服务供其他模块读取预约信息；生命周期服务是审批模块发起状态变更的唯一入口。把“写”集中在服务与状态机、把“读”集中在查询服务，使模块的对外能力边界清晰可辨。

会议模块负责会议执行与通知。执行服务维护参会人名单，处理签到、签退与出勤统计；调度器在会议临近开始时向参会人推送提醒，并在会议结束后将仍未签到的人员判定为缺席；通知服务负责通知的写入、去重与读取，并通过通知适配器实现预约模块声明的通知端口。通知的去重依赖数据库的唯一约束，因此重复调度不会产生重复消息。

审批模块负责管理侧的动作与统计。审批服务处理通过、驳回与强制取消，并在每次动作后写入审批记录与操作日志；统计部分按会议室与时段聚合使用时长、预约次数与高峰分布；审计部分提供按操作人、业务类型与时间范围检索操作日志的能

力。审批模块自身不修改预约状态，而是通过生命周期端口调用预约模块，保证管理动作与用户动作走同一套规则。

6.3 前端实现

前端为基于 Vue3 的单页应用，采用组件化与状态管理（Pinia）组织代码。其实现要点包括：以日历周视图直观展示各会议室的空闲与占用，支持在视图上直接选择时段发起预约；以“我的预约”页面查看与取消本人预约；以管理后台完成审批、资源配置与统计查看；以通知中心展示个人通知与已读状态。接口调用统一收敛在按业务域划分的 API 适配层，统一注入访问令牌与错误处理。系统状态完全以后端返回为准，前端不自行维护状态机，仅在改期时透传乐观锁版本号，从而避免前后端状态分叉。

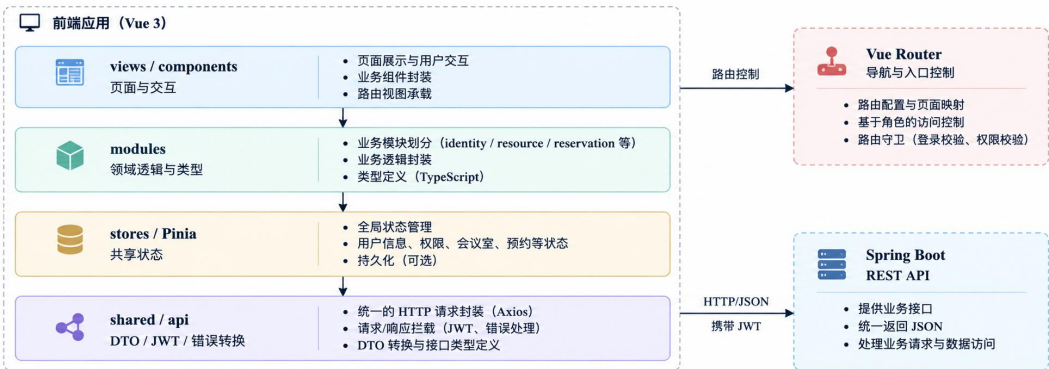


图 3-4 前端整体架构与模块关系图

图 6.1 前端整体架构与模块关系图

前端按“页面—模块—共享层”三级组织。页面层包含登录页、预约看板、我的预约、会议室台账与各管理页面；模块层按业务域拆分为预约、会议、审批等目录，每个目录内自带页面组件与接口适配器；共享层提供 HTTP 客户端、登录状态、类型定义与通用组件。这样组织的好处是：业务改动通常只落在一个模块目录内，不会波及无关页面。

请求封装是前端与后端约定的落点。HTTP 客户端在每次请求时自动注入访问令牌，并在响应为非成功状态时统一处理：可展示给用户的业务错误直接取后端返回的消息，鉴权失败则清除登录状态并跳转登录页。由于后端已经提供了可读的错误消

息，前端不再自行拼装提示文案，这使界面提示与接口实际结果保持一致——用户看到的失败原因与接口返回的原因始终是同一句话。

登录状态集中由状态管理维护，登录成功后写入令牌并保存当前用户与角色，应用启动时用令牌换取当前用户信息以恢复登录态；角色判断以服务端返回的角色为准，管理路由在进入前检查角色，不具备权限时给出提示并跳转回预约看板。前端的角色控制只用于改善交互体验，接口层的权限校验仍由后端负责。

本版在既有实现上补充了几处落点：预约模块新增审批超时清扫的服务与定时触发器，前者承载扫描与通知编排，后者只负责按固定间隔触发，两者分开是为了让业务逻辑可以在测试中直接驱动，不必依赖定时器；资源模块新增独立的只读查询数据访问接口，把台账筛选与“该时段被哪些会议室占用”的判定集中在一处，写路径保持原样；审批模块的统计查询补充了会议室使用明细、应到人次与缺席人次三个聚合，并在服务层结合全局开放时段换算为比率；公共模块补充了查询参数类型不匹配的异常映射，使日期、时间等参数格式写错时返回参数错误而不是兜底的系统错误。

6.4 数据库实现与迁移

数据库采用 MySQL 8 实现，表结构通过版本化脚本管理，按功能演进逐批升级：初始化脚本建立用户、分类、会议室、预约等基础表；随后的迁移脚本依次补充会议执行与通知相关表、预约状态约束、乐观锁版本列、系统时间配置与预约窗口配置等。采用“版本化迁移”的好处是结构变更可追溯、可重复执行，便于团队协作与部署重建。核心表结构及其主要字段见附录 A。

数据库表结构通过版本化脚本管理，按功能演进逐批升级，而不是维护一份不断被手工修改的完整脚本。基线脚本建立用户、部门、会议室分类、会议室、开放规则、报修工单、预约、审批记录与审计日志等基础表；后续迁移脚本依次补充各业务域在实现过程中新增的表、字段、约束与索引。迁移脚本的演进过程如表 6.2 所示。

表 6.2 数据库迁移脚本的演进

脚本	主要内容
schema.sql	基线表结构：用户与部门、会议室分类与会议室、开放规则与工单、预约、审批记录与审计日志

软件学院软件实践说明书

脚本	主要内容
V1.1	团队协作基线：预约请求标识、开放规则表的建立
V1.2	预约加固：用户与请求标识的唯一约束，支撑幂等去重
V1.3	身份治理：信用分、限制期与违规记录
V1.4	资源管理：维护计划与报修工单
V1.5	会议执行：参会人与出勤记录、站内通知
V1.6	审批运营：审批记录索引与操作审计索引
V1.7	预约状态的取值检查约束
V1.8	预约乐观锁版本号，支撑并发修改保护
V1.9	业务时钟配置，支撑可推进的时间基准
V1.10	全局可预约时段配置
V1.11	会议实际使用记录：与预约一对一的实际起止时间与实际参会人数

采用增量迁移而非修改基线的方式，是为了让任何一台开发机都能按固定顺序把库升到最新版本，也让“某个字段为什么存在”有据可查。迁移脚本按顺序执行即可完成初始化；对于不含库名限定的脚本，执行时需要显式指定目标库，这一点在脚本说明中已经注明。

本版前端补充了几个可复用的部件：一个“按页面可见性周期刷新”的组合式函数，供看板、审批页与我的预约共用，页面不可见时自动跳过、切回时立即刷新；一个描述预约时间推导态的公共模块，把“进行中 / 已结束”的判断与标签色集中定义，看板卡片与详情面板共同引用；一个维护周视图列宽的存储模块，按列的位置记录宽度并限制上下限；一个独立的会议室查询接口适配器，封装带筛选条件的台账查询与时段空闲查询；以及顶部的驳回通知横幅组件，登录后存在未读驳回通知时在页面顶部给出醒目提示。此外新增了“说明书”页面及其路由，把三步预约流程、看板颜色含义与会议室状态含义写成图文说明。

本版还补上了一份一体化建库脚本 `sql/init.sql`：它是表结构快照与种子数据的合并结果，一条命令即可把数据库建到最新结构（当前为 v1.11，共 17 张业务表）。合并过程中同时修正了快照脚本的两处缺陷——补上 V1.6 迁移增加而快照遗漏的 4 个运营查询索引，并补全 DROP 列表中漏掉的两张单例配置表，使脚本可以被重复执行。对已经存在业务数据的数据库，仍应使用 `migrations` 目录下的增量脚本逐版升级，而不是重跑一体化脚本。

6.5 安全与鉴权实现

系统采用基于令牌的鉴权方式：用户登录成功后由服务端签发令牌，前端在后续请求的请求头中携带；服务端通过过滤器解析令牌并写入安全上下文，实现“一次登录、全程可用”。在权限控制上，系统仅设管理员与普通用户两种角色，管理类接口通过方法级角色约束限定为管理员，普通用户无法访问。此外，令牌密钥设有最小长度校验，避免因密钥过短带来的安全风险。

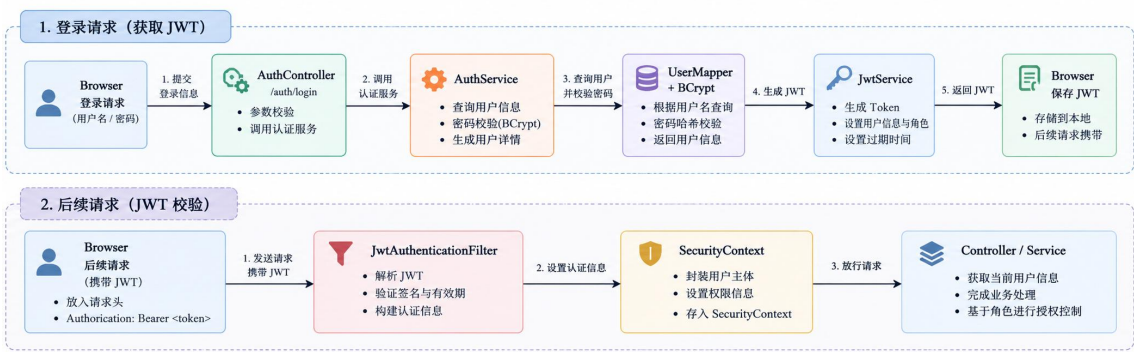


图 3-2 登录认证与后续请求处理流程图

图 6.2 登录认证与后续请求处理流程图

系统采用基于令牌的无状态鉴权。用户登录成功后，服务端签发访问令牌，令牌中携带用户标识与角色，并设置有效期；前端在后续请求的请求头中携带该令牌；服务端通过过滤器解析令牌并写入安全上下文，使后续处理能够直接获取当前用户。采用无状态方式的原因是前后端分离部署下不需要服务端会话，服务重启也不会影响已登录用户。

在权限控制上，系统采用“统一鉴权 + 方法级角色控制”的组合：过滤器负责判断请求是否携带有效令牌，方法级注解负责判断当前角色是否允许执行该操作。二者分工明确——前者回答“是否已登录”，后者回答“这个角色能否做这件事”。管理类接口只对管理员开放，普通用户即便手工构造请求也会被拒绝。

密码以加盐摘要的方式存储，数据库中不保存明文；错误响应不向前端返回异常堆栈、SQL 语句或表名，避免通过报错信息推测内部结构。对于越权访问，系统返回明确的“无权限执行该操作”，而不是返回“资源不存在”，使合法用户的误操作能够得到准确提示。

6.6 系统部署与启动

系统以单节点方式部署：先启动数据库，再启动后端服务，最后启动前端开发服务器。后端端口由 `SERVER_PORT` 配置（源码默认 8080，本机示例配置为 8081），前端开发服务器默认监听 5173；前端通过开发代理将接口请求转发至后端，避免跨域问题。为提升开发体验，项目提供了一键启动脚本，可同时拉起前后端。

系统以单节点方式部署，启动过程分为四步，每一步都有对应的检查点，便于在启动失败时快速定位问题，如表 6.3 所示。

表 6.3 启动步骤与检查点

步骤	操作	检查点
1	启动数据库服务	数据库端口可连接，目标库存在
2	按顺序执行初始化与迁移脚本	核心表与配置数据就绪
3	启动后端服务	后端端口处于监听状态，启动日志无异常
4	启动前端开发服务器	页面可访问，接口代理指向后端且请求返回正常

为降低启动阶段的排错成本，后端在启动时会对关键配置做前置校验：数据库连接不可用、访问令牌密钥长度不足等情况都会在启动阶段直接失败并给出明确提示，而不是等到第一次请求时才暴露问题。经过这一处理，绝大多数启动期问题都能在启动日志中直接定位。

6.7 关键页面与业务操作说明

本节依据课程设计任务书的功能要求，对系统关键页面及其对应业务操作进行说明。页面展示不以视觉效果为重点，而以“功能入口可用、真实接口联通、业务规则生效、操作结果可验证”为核心。为保证功能覆盖完整，展示内容同时包含普通用户端和管理员端，并覆盖会议室资源、预约、冲突检测、预约变更、会议执行、使用记录、统计、周期性会议、通知与参会人维护等功能。

6.7.1 功能展示路线

关键页面按照“身份进入—资源查询—预约创建—规则裁决—预约管理—会议执行—管理维护—统计与扩展”的顺序组织。该路线既覆盖任务书规定的必做功能，也覆盖系统已经实现的选做功能，避免仅展示孤立页面而无法证明完整业务闭环。

表 6.4

展示阶段	主要功能	重点验证内容	建议截图
身份与入口	登录、角色入口	身份认证、权限区分	登录页、用户端/管理端首页
资源查询	会议室台账、条件筛选、时段可用查询	位置、容量、设备、资源状态、可用性	会议室列表与筛选结果
预约创建	预约表单、容量校验、参数校验	真实接口提交、预约成功	预约填写与成功结果
核心裁决	时间冲突检测	包含、部分重叠、完全相同等冲突被阻止；首尾相接允许	典型冲突提示
预约管理	查看、改期、换会议室、取消	改期重新检测；已开始不可变更；取消释放时段	我的预约、改期、取消
会议执行	实际开始、实际结束、实际人数	计划与实际使用记录分离	执行登记、完成记录
资源管理	会议室新增、编辑、状态维护、删除保护	未来预约存在时禁止删除	管理端资源页面、删除保护

软件学院软件实践说明书

展示阶段	主要功能	重点验证内容	建议截图
异常反馈	参数错误、业务错误	不暴露异常堆栈，反馈明确	超容量/状态限制提示
统计分析	使用率、会议时长、爽约率等	SQL 聚合结果可查询和展示	统计页面
扩展功能	周期会议、参会人、会议通知	批量预约与附属信息维护	对应扩展页面

6.7.2 用户登录与角色功能入口

用户通过登录页面输入账号和密码完成身份认证。认证成功后，系统根据当前用户的角色加载相应功能入口。普通用户主要使用会议室查询、预约申请、个人预约管理、会议执行与记录等功能；管理员负责会议室资源维护、业务管理与统计查询。通过角色划分，系统将普通业务操作与资源管理操作进行隔离。

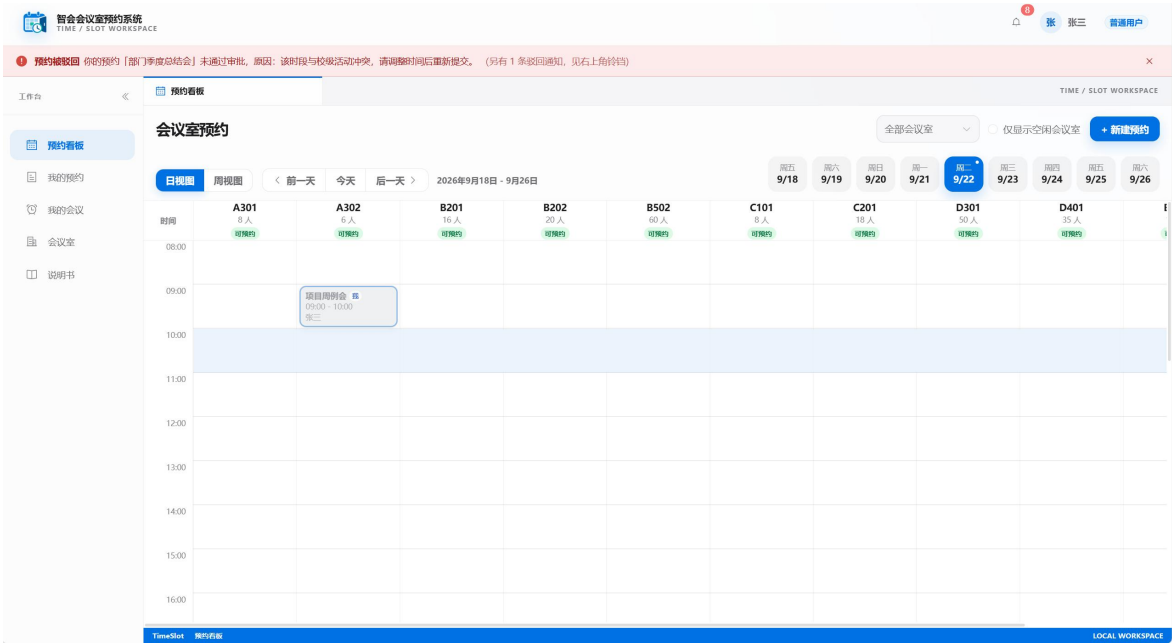


图 6.3 系统登录页面

软件学院软件实践说明书

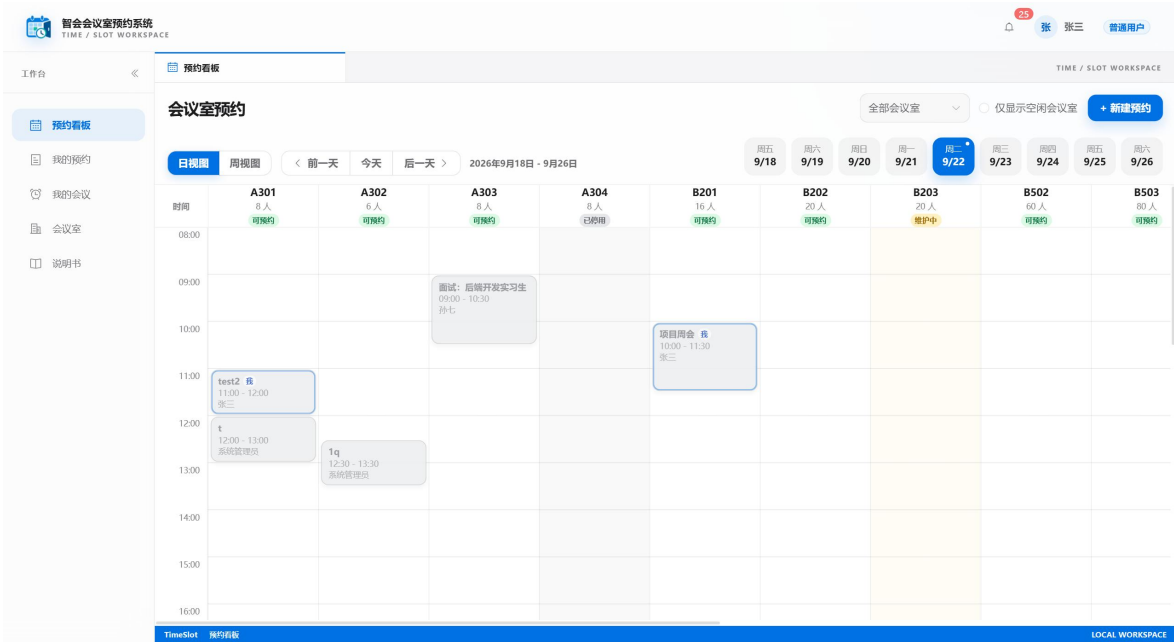


图 6.4 普通用户系统首页

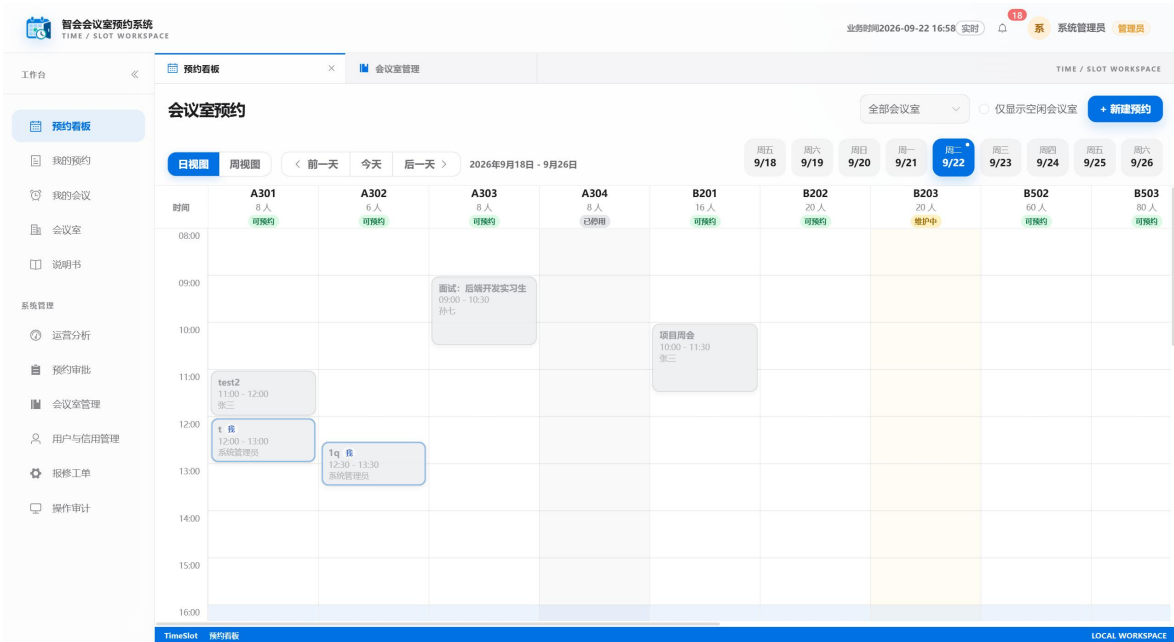


图 6.5 管理员系统首页

6.7.3 会议室台账与资源信息展示

会议室台账用于集中展示系统中的会议室基础资源，列表呈现会议室名称、所在位置、容量、设备配置与资源状态等信息。资源状态用于表示会议室当前是否可参与

预约，取值为 AVAILABLE（可用）、MAINTENANCE（维护中）与 DISABLED（停用）。该页面是预约业务的数据基础。

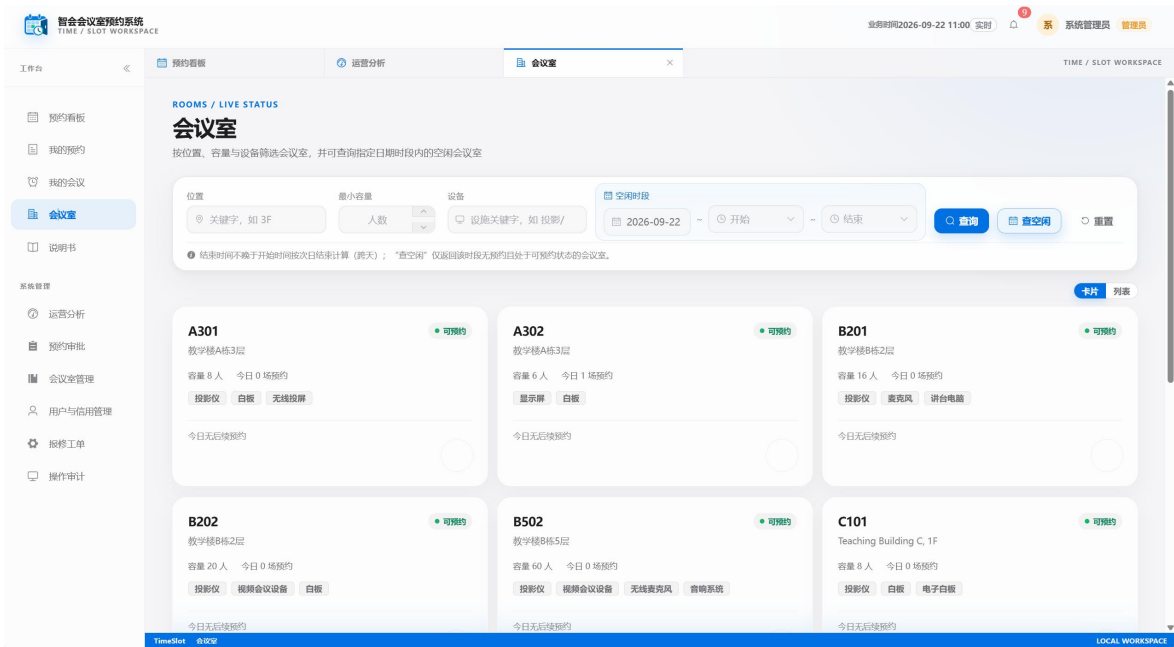


图 6.6 会议室台账页面

6.7.4 会议室条件筛选与指定时段可用查询

用户可以根据会议实际需求对会议室进行筛选，包括位置、最低容量以及投影、视频会议、白板等设备条件。对于给定日期和起止时间，系统进一步结合已有有效预约记录判断会议室是否处于可约状态，从而在预约提交前帮助用户排除明显不可用的资源。

软件学院软件实践说明书

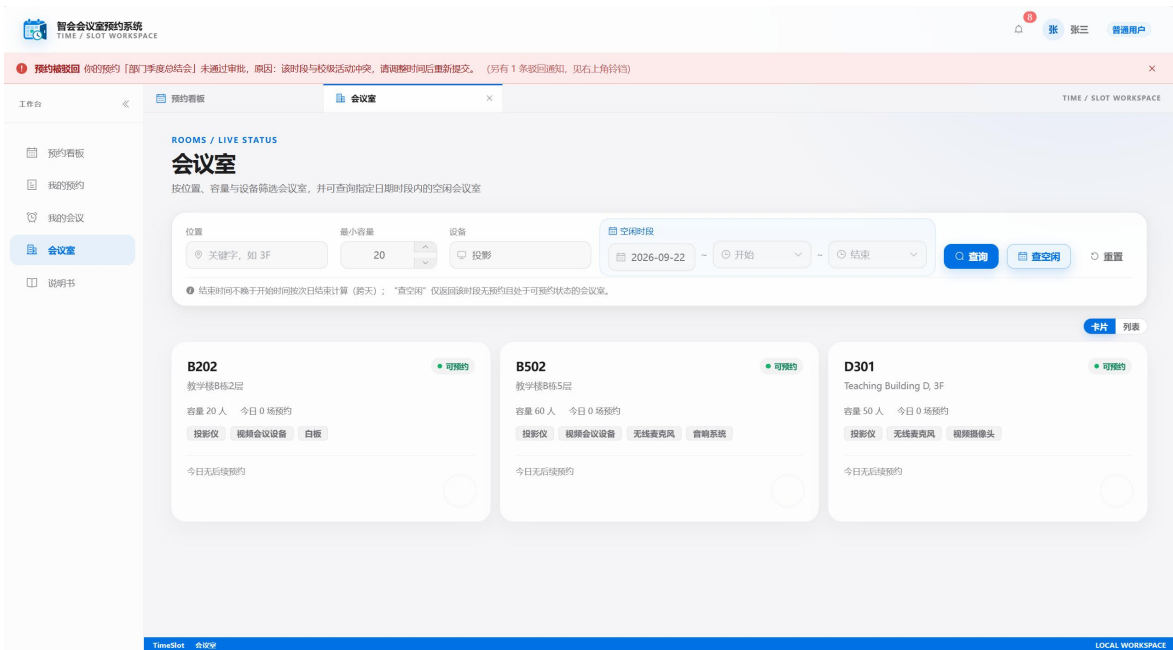


图 6.7 会议室条件筛选页面

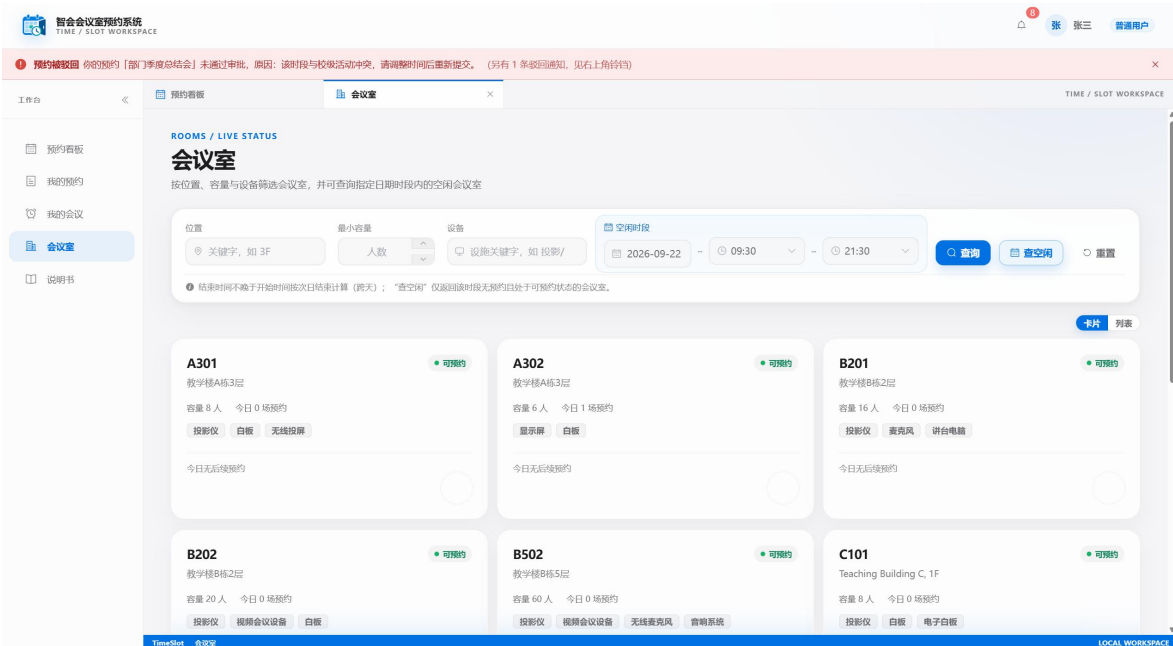


图 6.8 指定日期与时段的可用会议室查询结果

6.7.5 预约申请与参数校验

用户选定会议室后填写会议主题、预约日期、开始时间、结束时间和参会人数等信息。系统在提交阶段检查必要参数是否完整、结束时间是否晚于开始时间以及参会

软件学院软件实践说明书

人数是否超过会议室容量。基础校验通过后，后端继续执行时间冲突检测，全部规则通过后才创建预约记录。

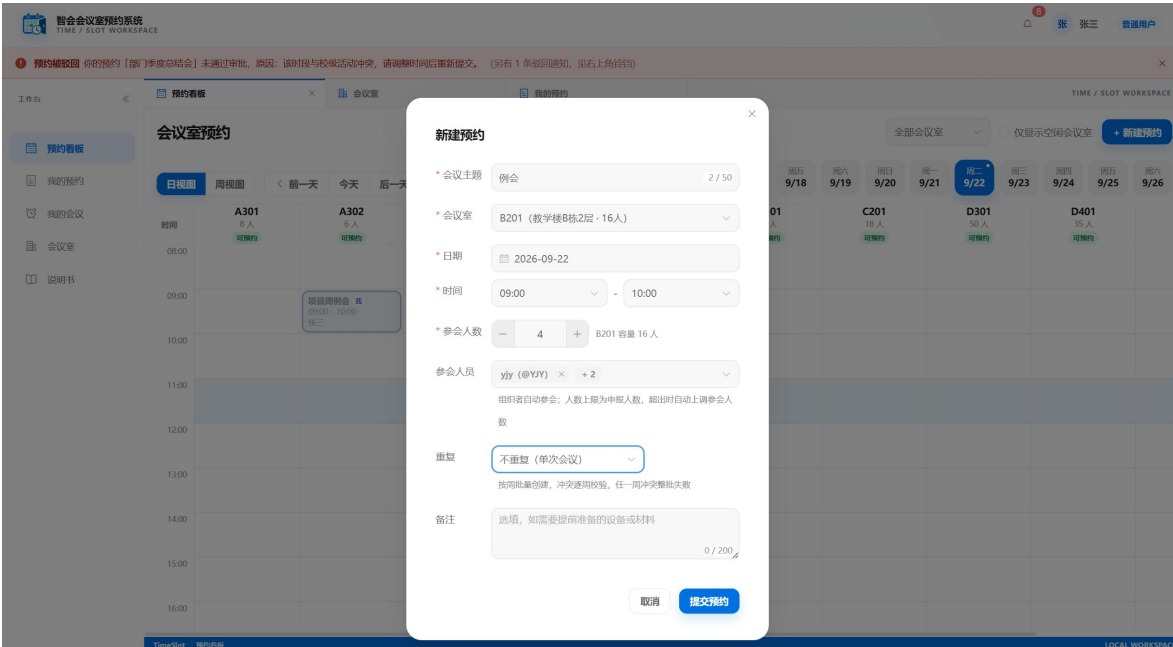


图 6.9 预约申请填写页面

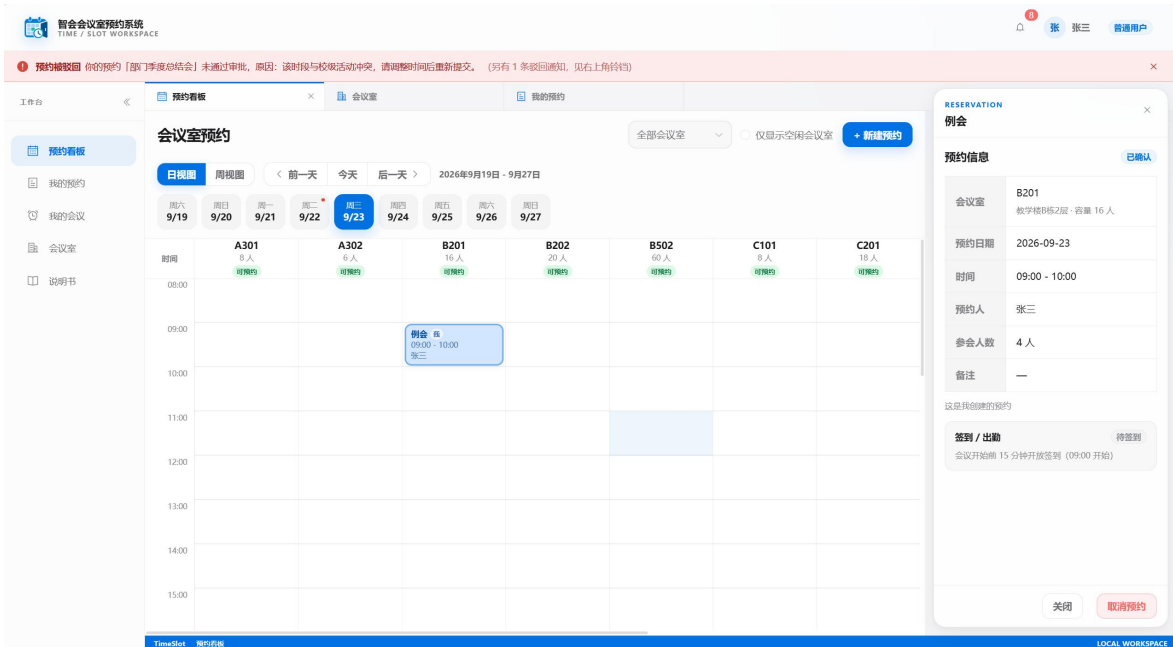


图 6.10 预约创建成功结果

软件学院软件实践说明书

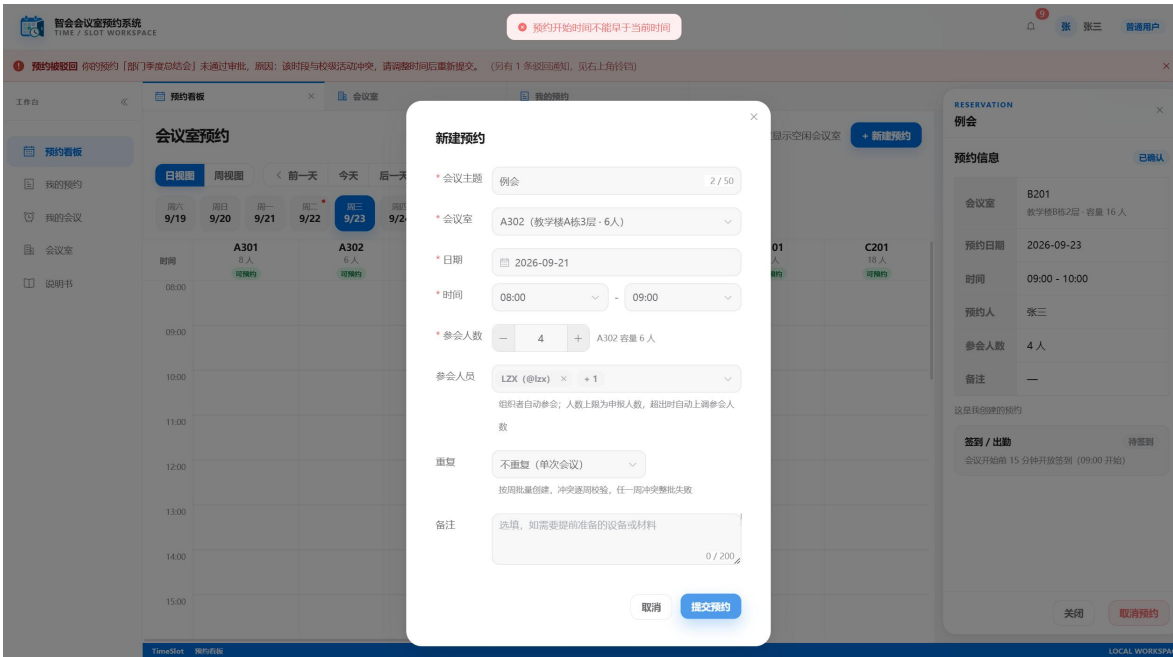


图 6.11 参数缺失或时间范围非法提示

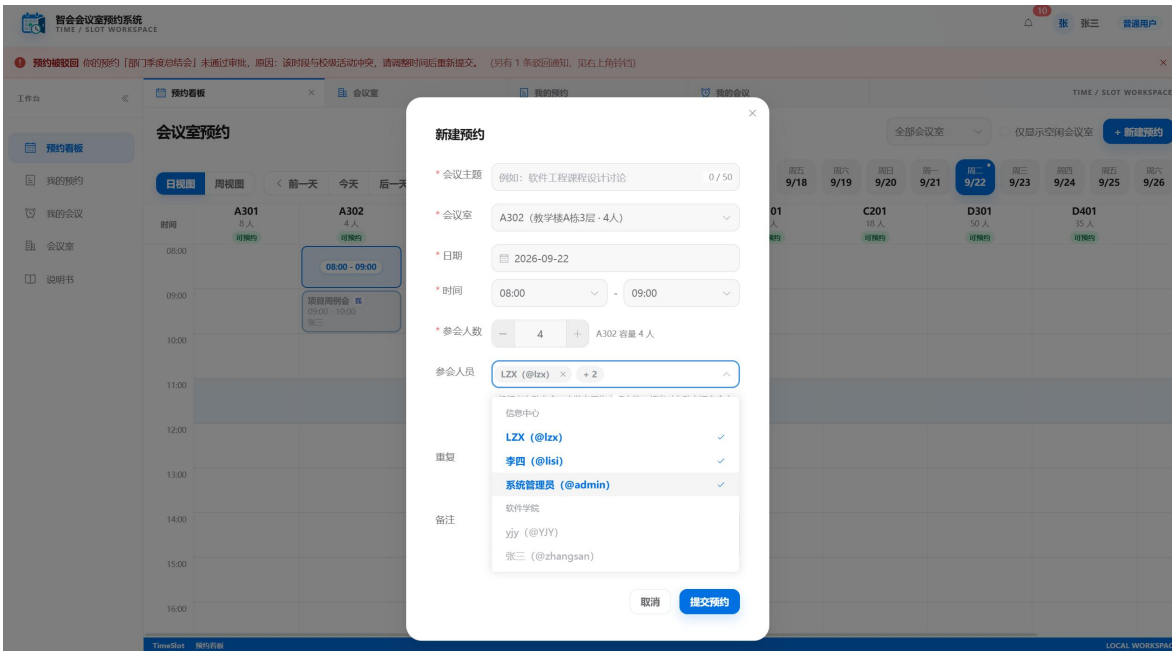


图 6.12 超过会议室容量的预约被阻止

6.7.6 时间冲突检测

时间冲突检测是本系统的核心业务规则。对于同一会议室，若已有有效预约区间 $[S1,E1)$ 与待提交区间 $[S2,E2)$ 满足 $S1 < E2$ 且 $S2 < E1$ ，则判定两个时间段存在重叠并拒绝预约。采用左闭右开的区间语义后，前一预约的结束时间等于后一预约的开始

软件学院软件实践说明书

时间时不构成冲突。冲突发生时，系统返回明确的业务提示，并说明冲突会议主题及其占用时段。

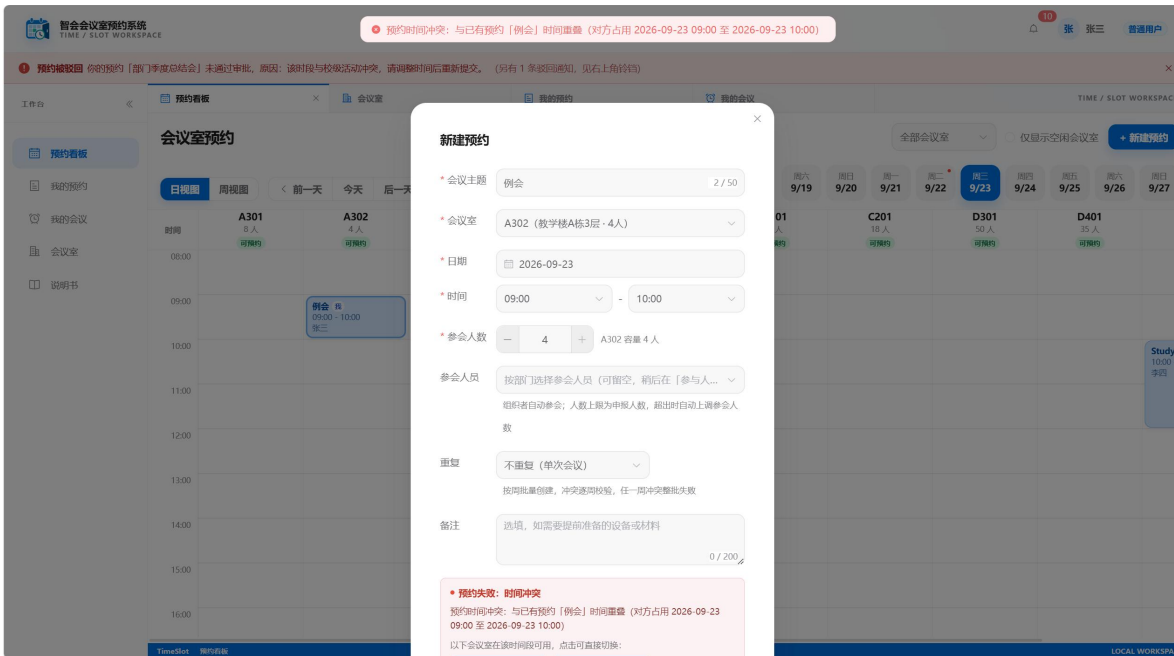


图 6.13 部分重叠预约被阻止

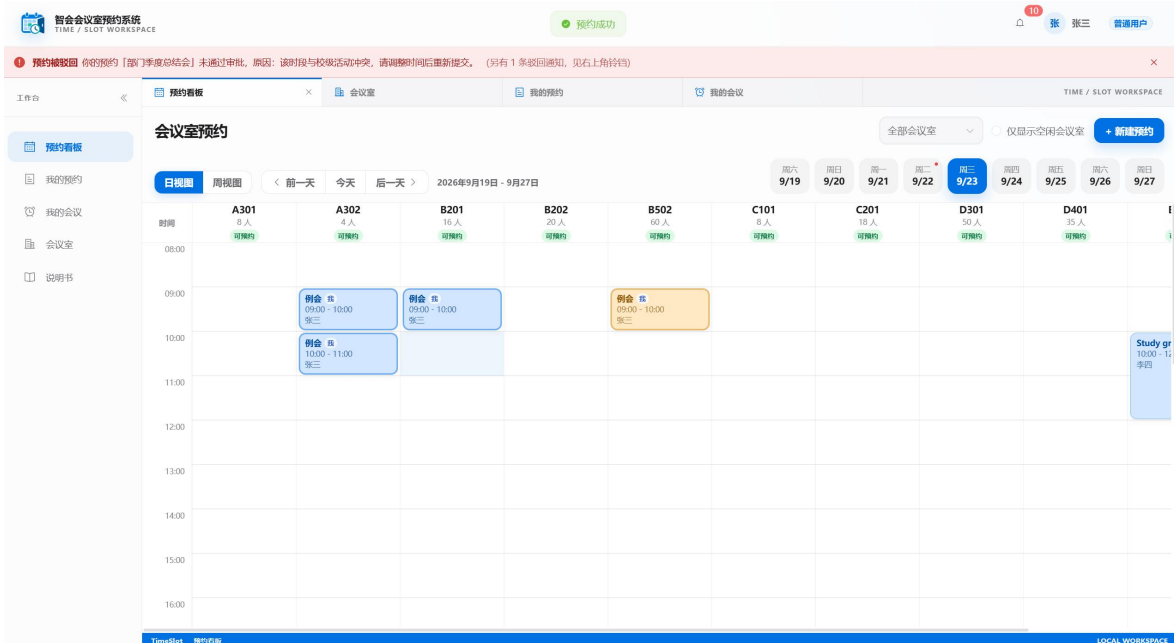


图 6.14 首尾相接预约成功

6.7.7 我的预约与预约详情

预约创建成功后，用户可在个人预约页面查看本人预约记录及其当前状态。列表与详情展示会议主题、会议室、预约起止时间、参会人数与预约状态，并提供与当前状态相匹配的后续操作入口。

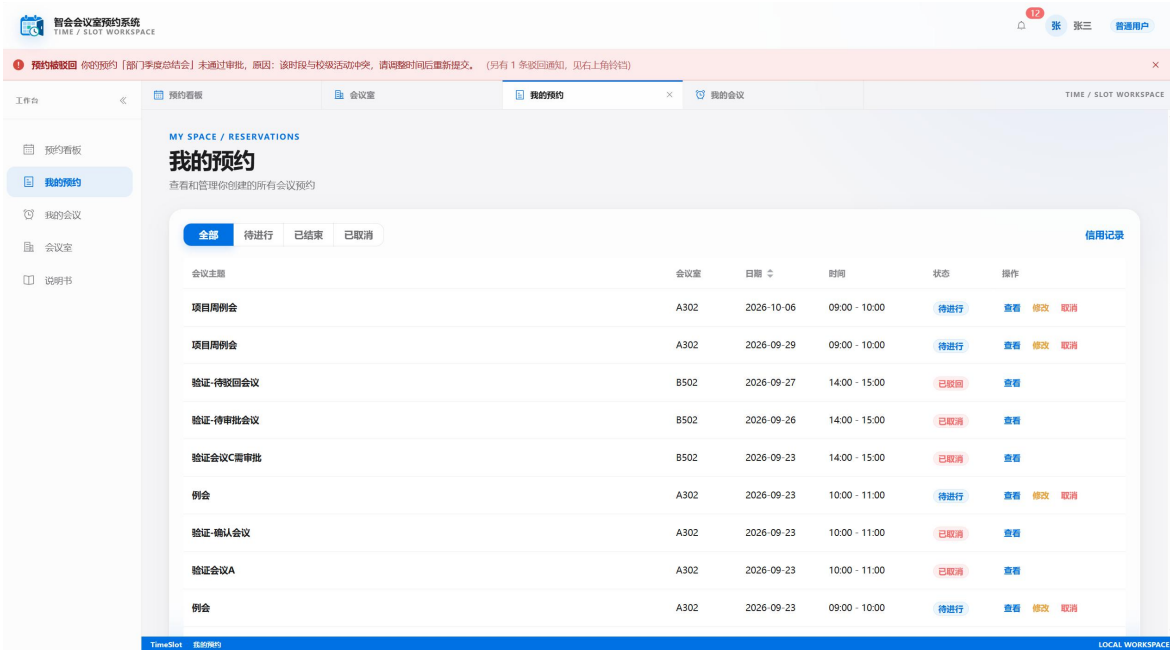


图 6.15 我的预约列表

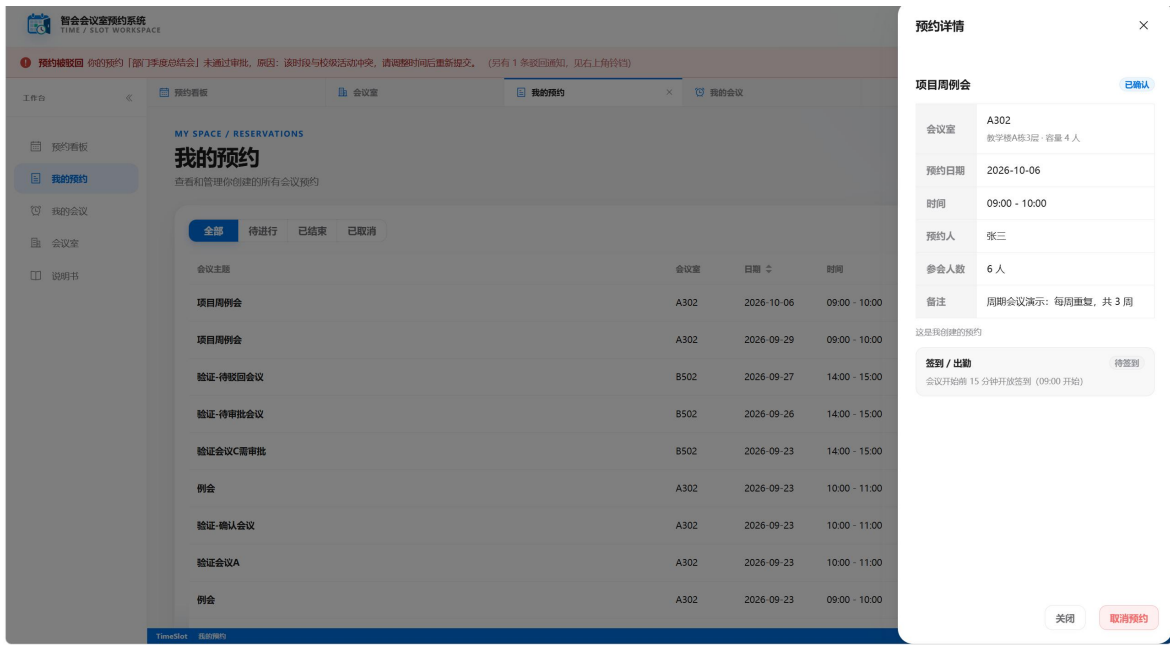


图 6.16 预约详情页面

6.7.8 预约改期与会议室变更

对于尚未开始的预约，预约人可以修改预约时段或更换会议室。修改后系统重新执行参数、容量与时间冲突检测，不因原预约已经存在而绕过业务规则；若新的时间或会议室不满足条件，则拒绝修改并保留原预约。

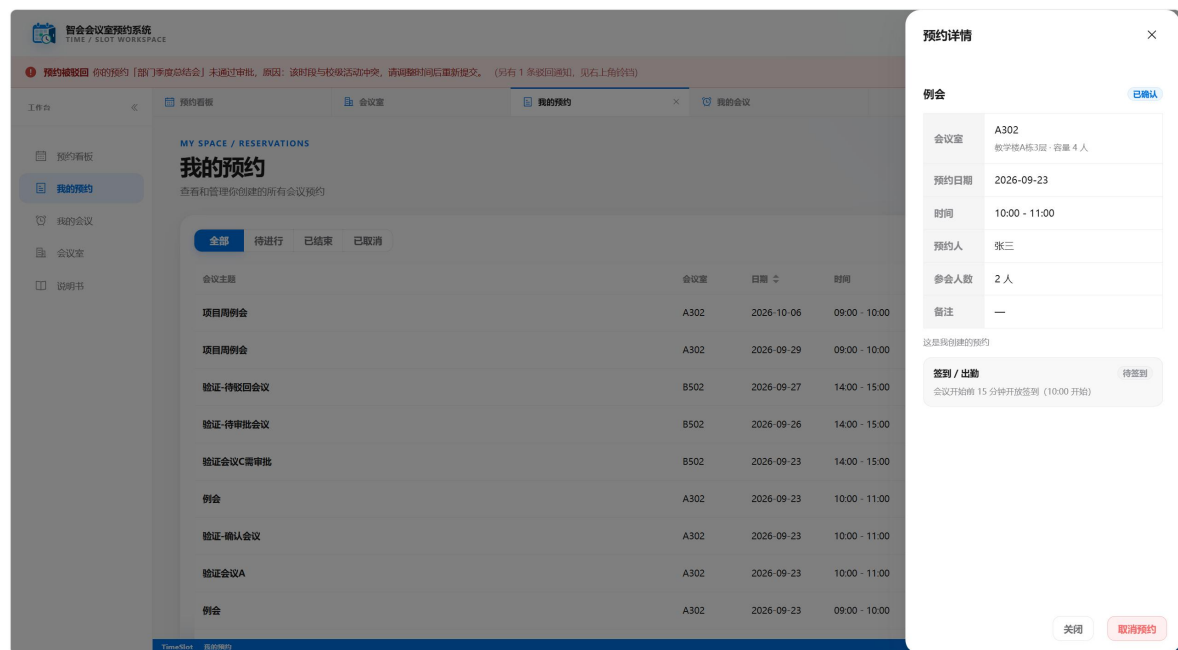


图 6.17 预约改期或更换会议室页面

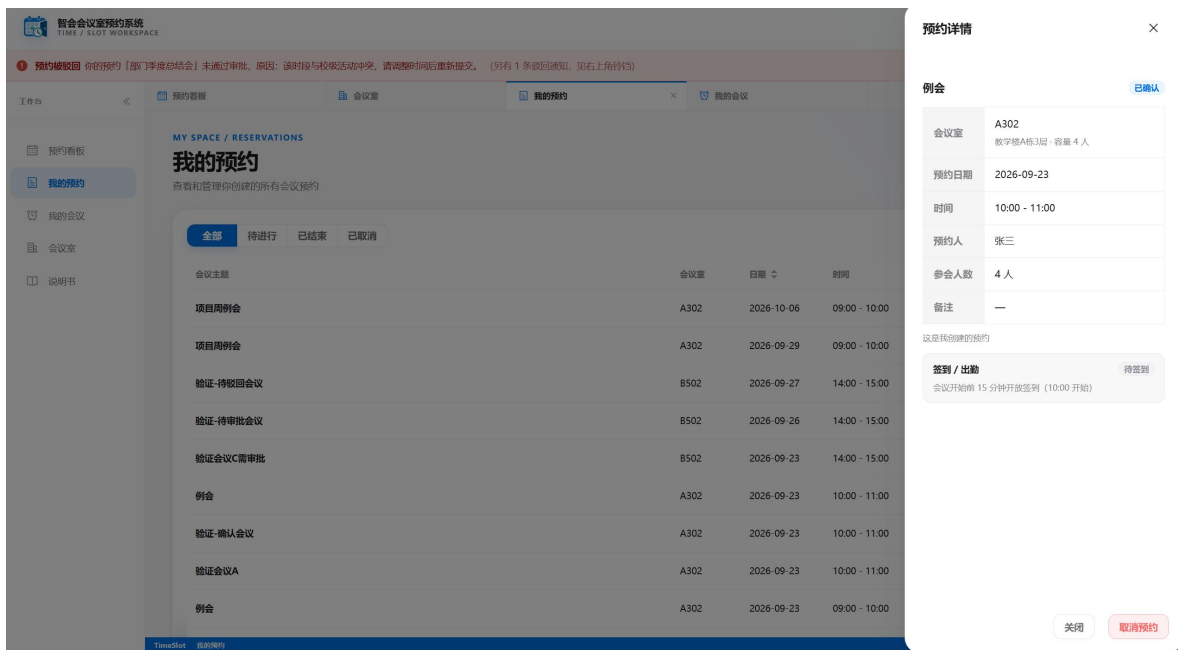


图 6.18 预约变更成功结果

软件学院软件实践说明书

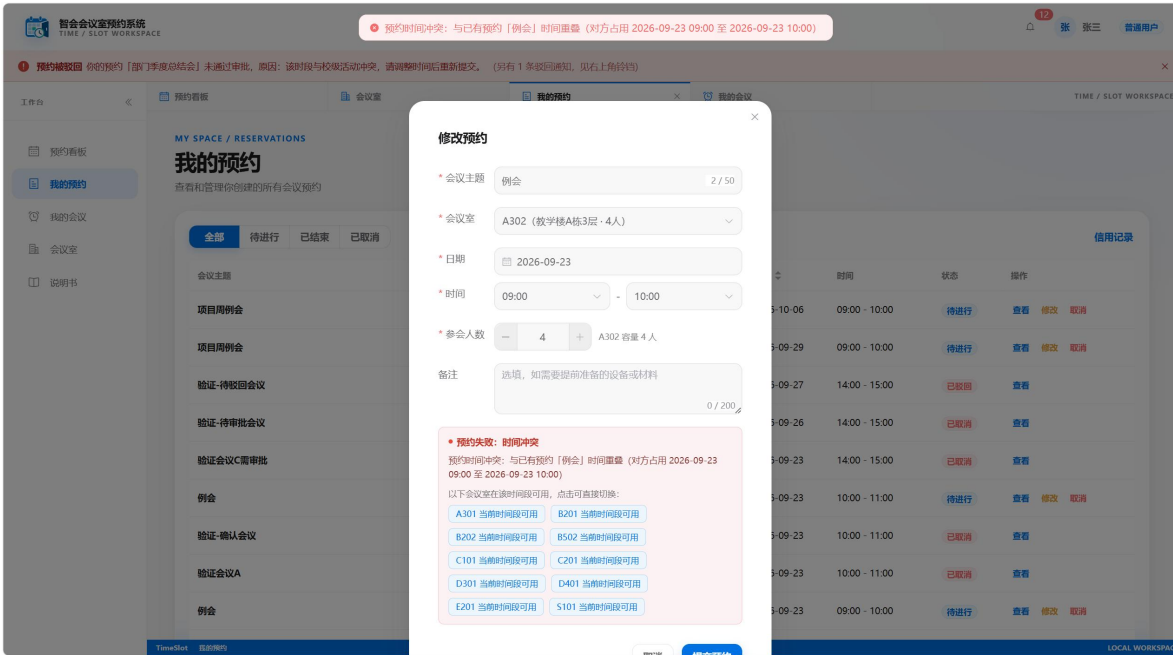


图 6.19 改期发生冲突时被阻止

6.7.9 预约取消与时段释放

尚未开始的预约允许由预约人取消。取消后该预约不再占用会议室对应时段，原时段随即重新进入可预约范围。已经开始的预约不能取消，以避免会议执行过程中的业务状态被破坏。

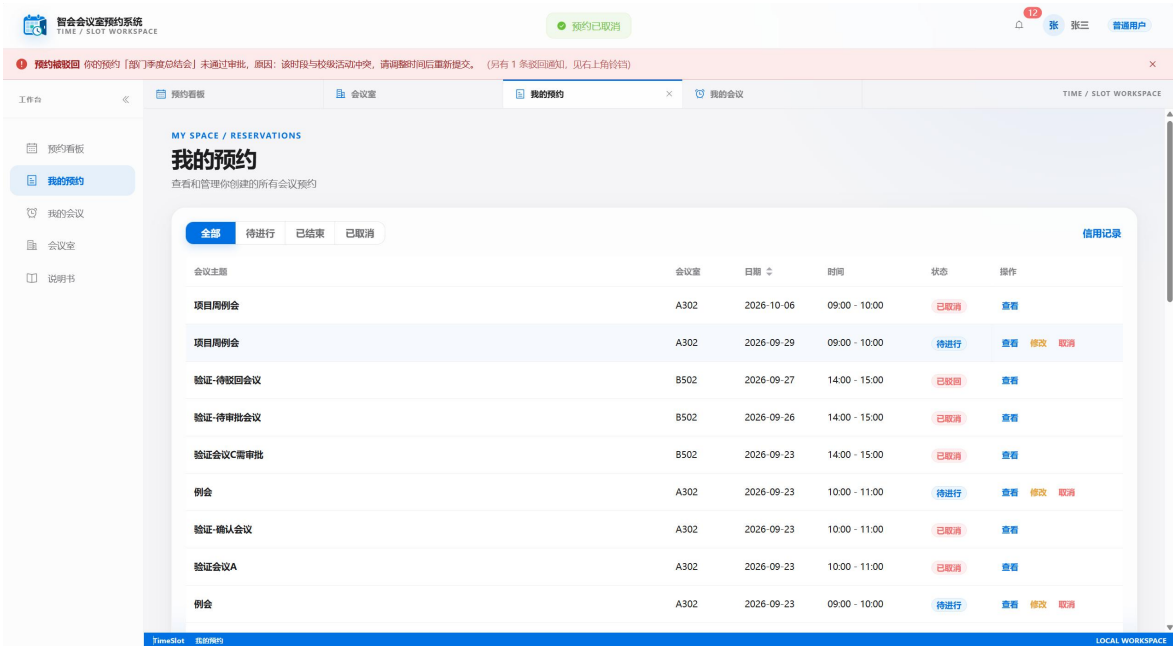


图 6.20 预约取消操作及取消后状态

软件学院软件实践说明书

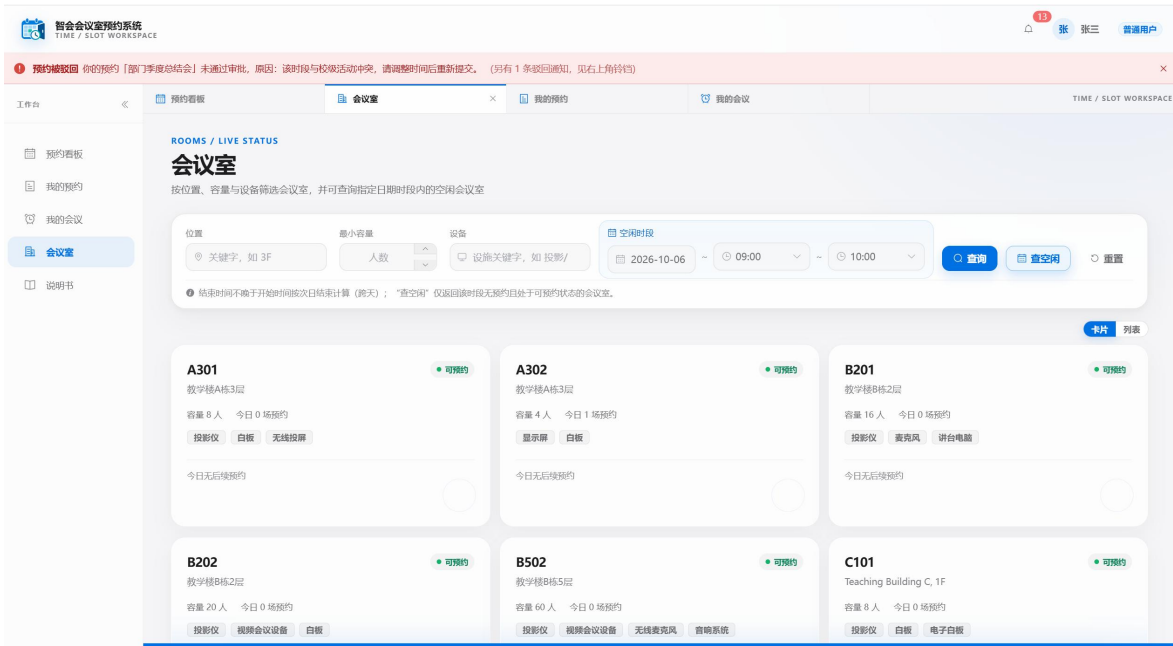


图 6.21 取消后原时段重新可约

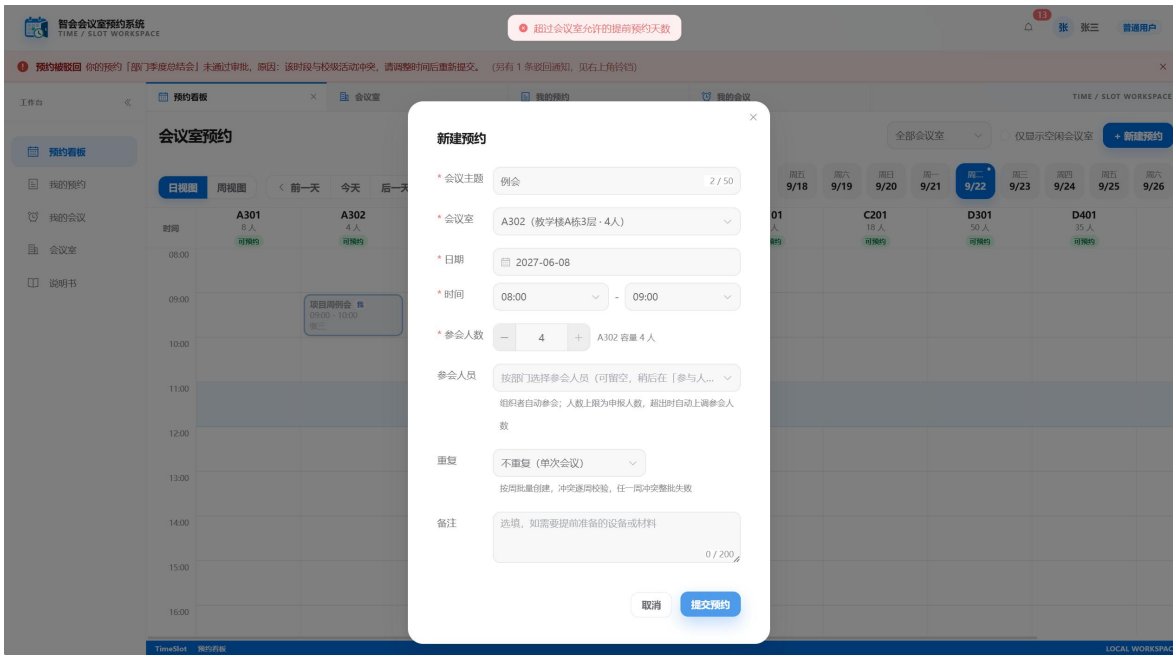


图 6.22 已开始预约禁止取消或修改

6.7.10 会议实际使用登记

对于进入执行阶段的会议，系统支持登记实际开始时间、实际结束时间以及实际参会人数。预约记录描述计划占用时间，使用记录描述实际执行情况，两者共同构成会议室使用数据。实际使用数据可以进一步支持会议时长、使用率和爽约情况统计。

软件学院软件实践说明书

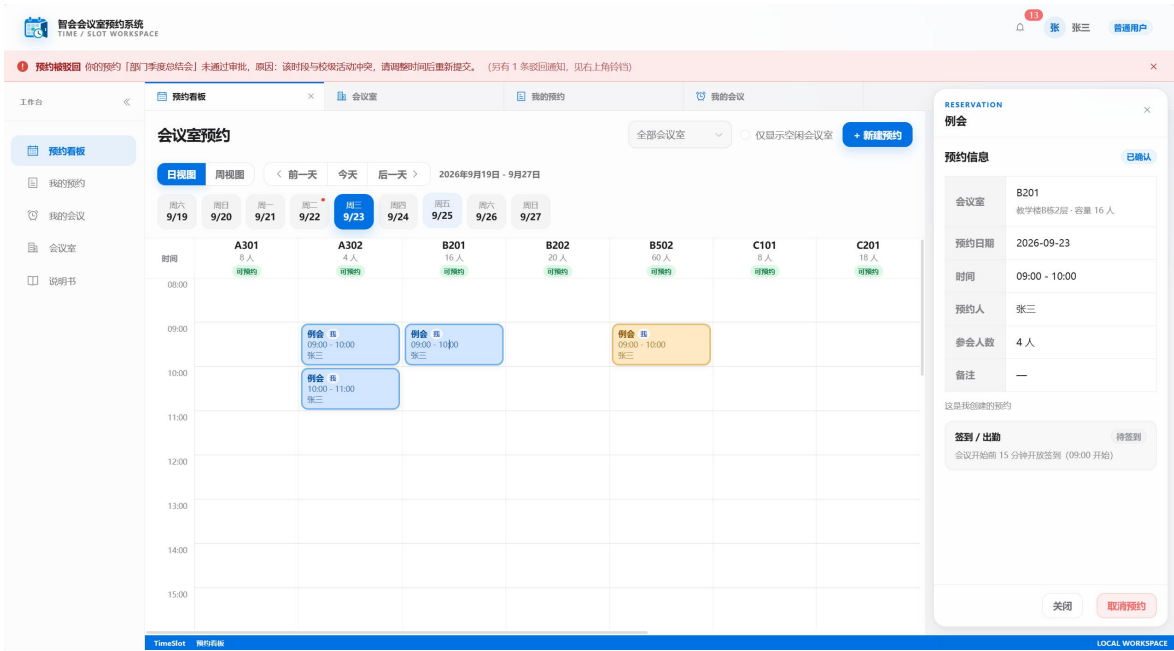


图 6.23 会议开始或使用登记页面

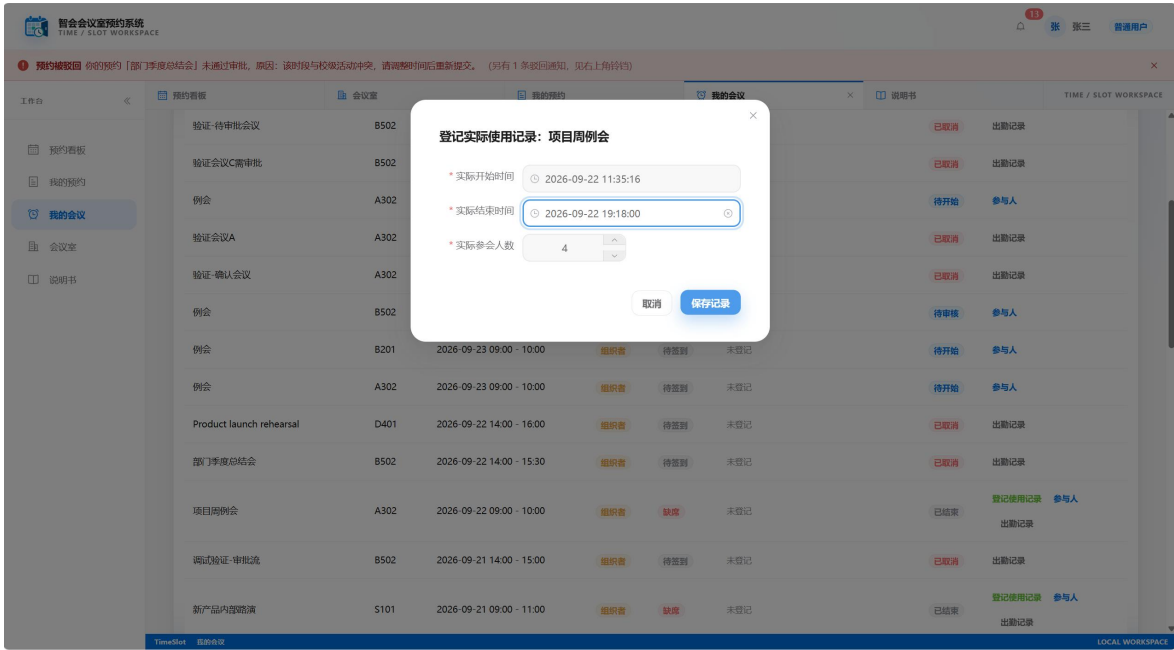


图 6.24 会议结束登记与完整使用记录

6.7.11 会议室资源新增、编辑与状态维护

管理员可对会议室资源进行新增与编辑，维护其名称、位置、容量与设备配置等基础信息，并根据实际情况调整资源状态。处于维护或停用状态的会议室不再作为可约资源参与新的预约流程。

软件学院软件实践说明书



图 6.25 新增会议室页面

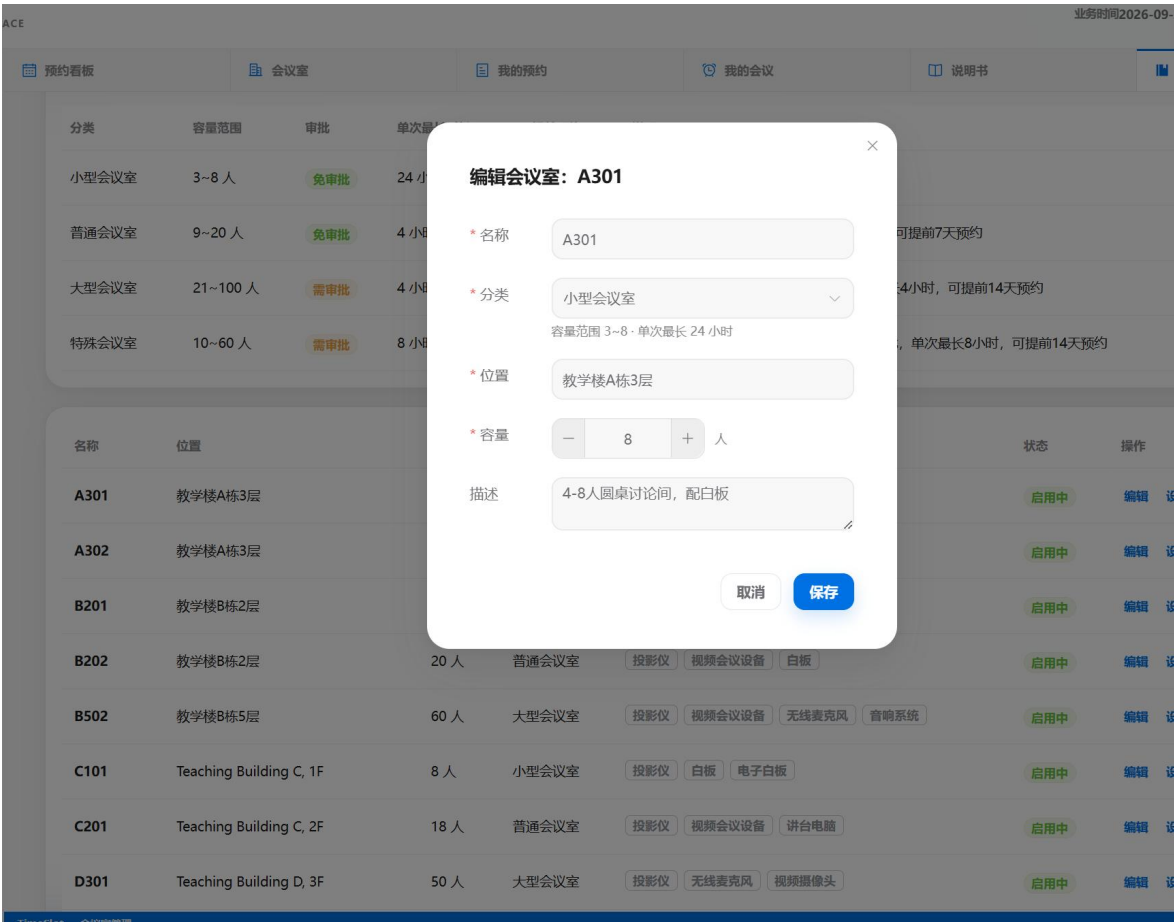


图 6.26 编辑会议室及资源状态页面

6.7.12 会议室删除保护

会议室删除操作受业务规则保护。当目标会议室仍存在未来有效预约或已存在历史预约记录时，系统拒绝删除并返回明确的业务错误提示，以防止已有预约失去关联资源；只有在不存在任何预约引用时，管理员才能执行资源删除操作。



图 6.27 存在未来预约时删除会议室被阻止

6.7.13 统一异常与业务反馈

系统对参数错误和业务规则错误进行统一处理。参数缺失、格式错误或时间范围非法等情况返回参数错误；预约冲突、超容量、状态不允许以及资源删除受限等情况返回业务错误。前端只展示面向用户的错误信息，不暴露后端异常堆栈，从而保持接口响应和页面反馈一致。

软件学院软件实践说明书



图 6.28 统一业务错误反馈示例

6.7.14 会议室使用统计

统计结果来源于系统真实业务数据，由数据库聚合查询得出，而非前端固定展示。

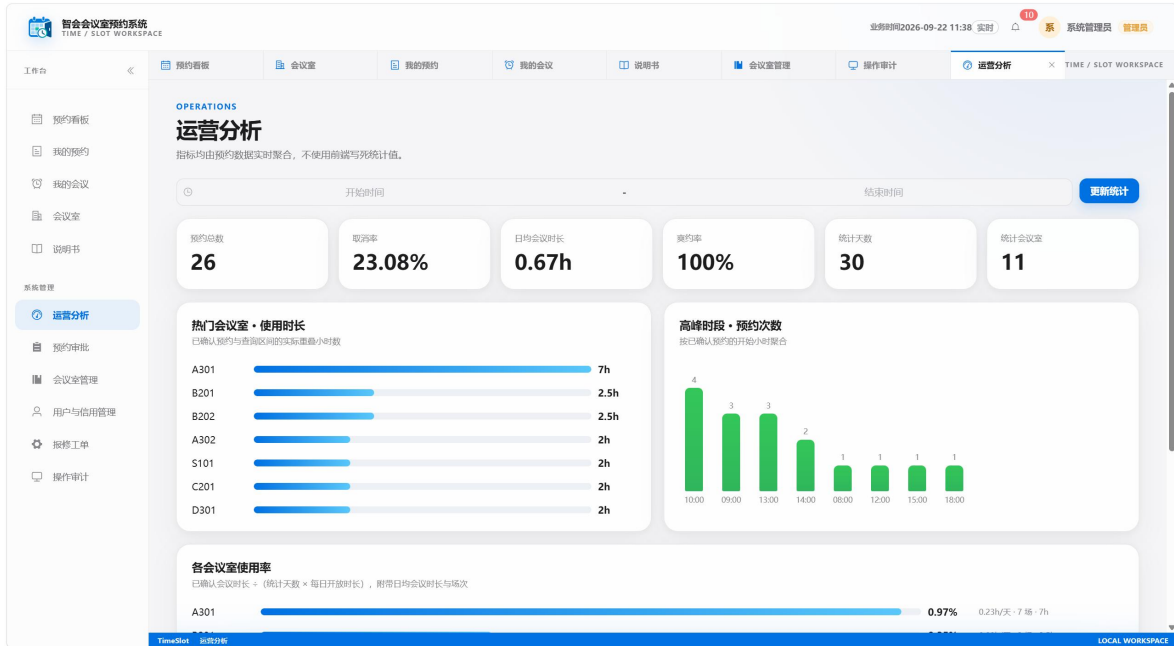


图 6.29 会议室使用统计页面

6.7.15 周期性会议批量预约

对于具有固定周期的会议，系统可通过周期性预约功能一次创建多次重复会议。用户设置会议室、会议主题、起止时间和重复规则后，系统按照规则生成对应预约。批量创建过程中仍需对各次预约执行会议室可用性和时间冲突校验，避免周期预约绕过普通预约约束。

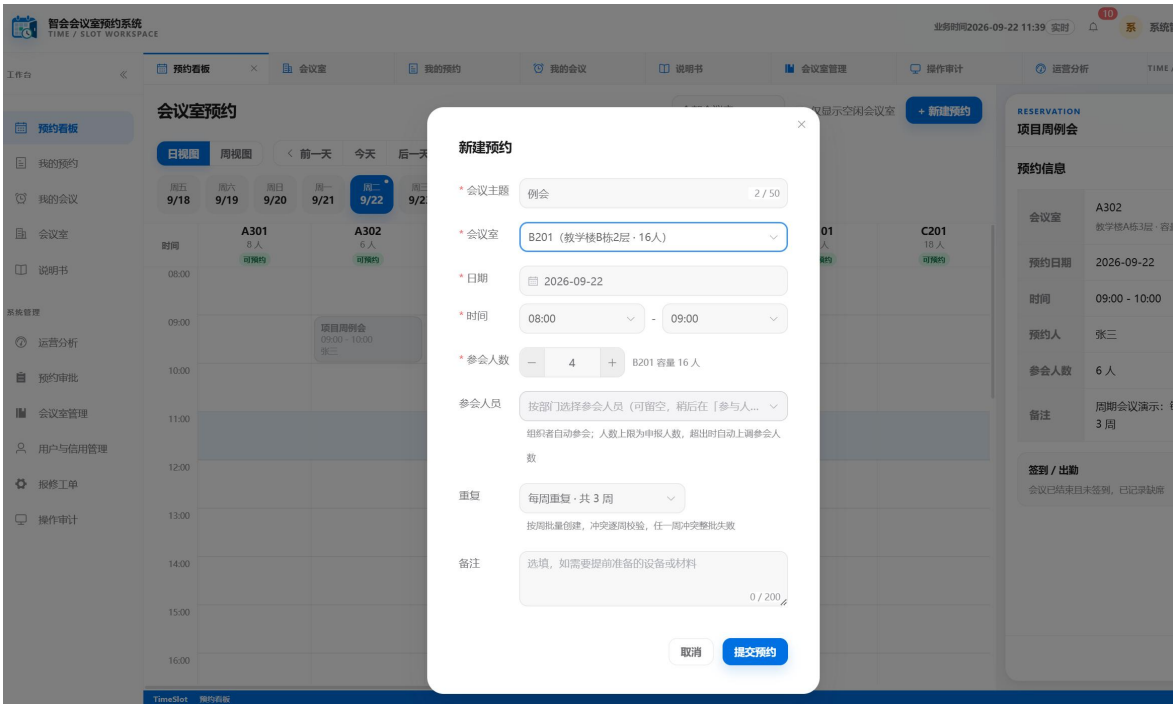


图 6.30 周期性会议创建页面

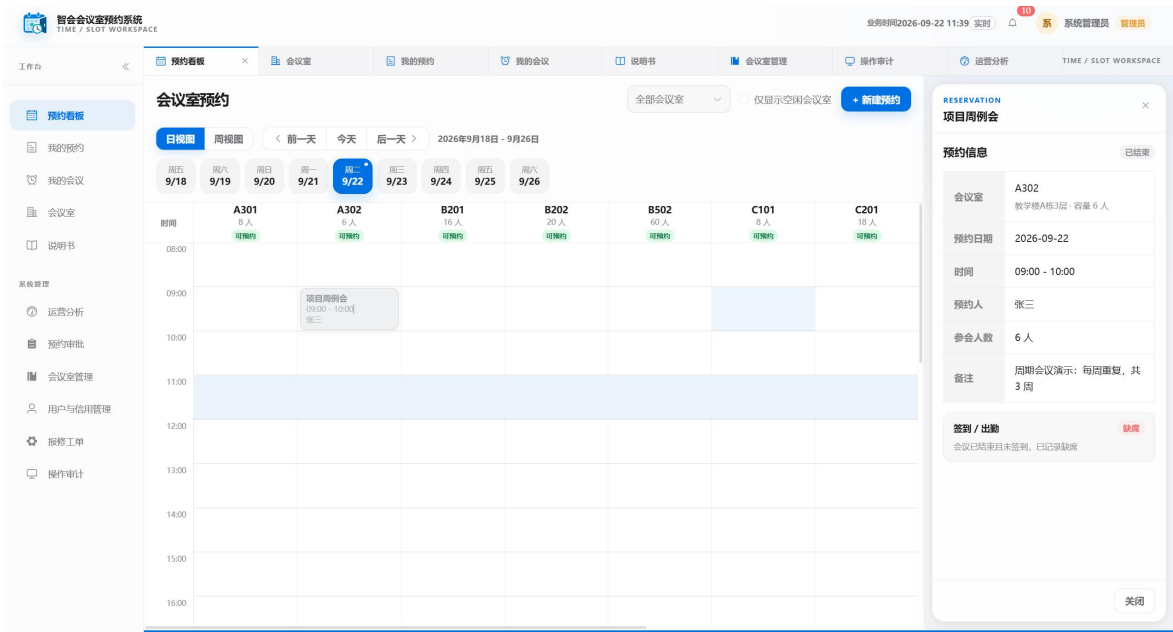


图 6.31 周期预约生成结果

6.7.16 参会人名单维护

系统支持在会议业务中维护参会人信息，使预约记录除会议主题和人数外，还能够关联具体参会人员。用户可在相应页面查看、增加或调整参会人名单，为会议通知和实际参会情况记录提供基础数据。

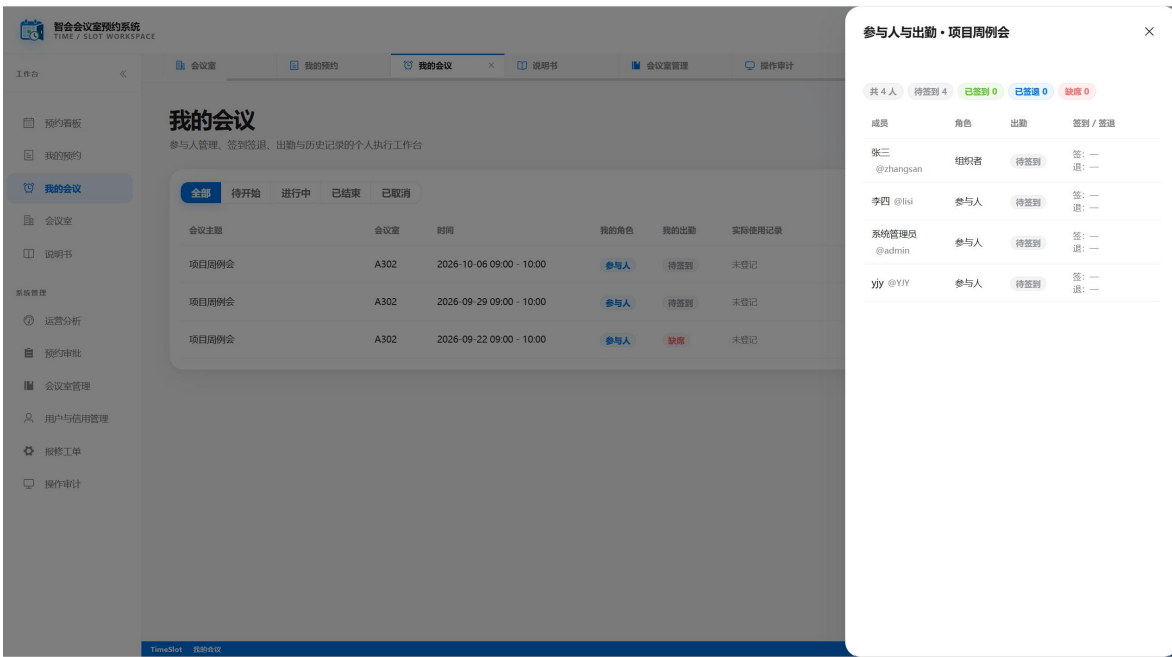


图 6.32 参会人名单维护页面

6.7.17 会议通知

系统围绕预约创建、变更与取消等业务事件生成会议通知，并通过通知中心向相关用户提供查看入口。通知列表展示通知内容、类型与已读状态，未读数量在导航栏实时提示；每条通知均关联到对应的会议或预约事件，便于用户回到业务上下文。

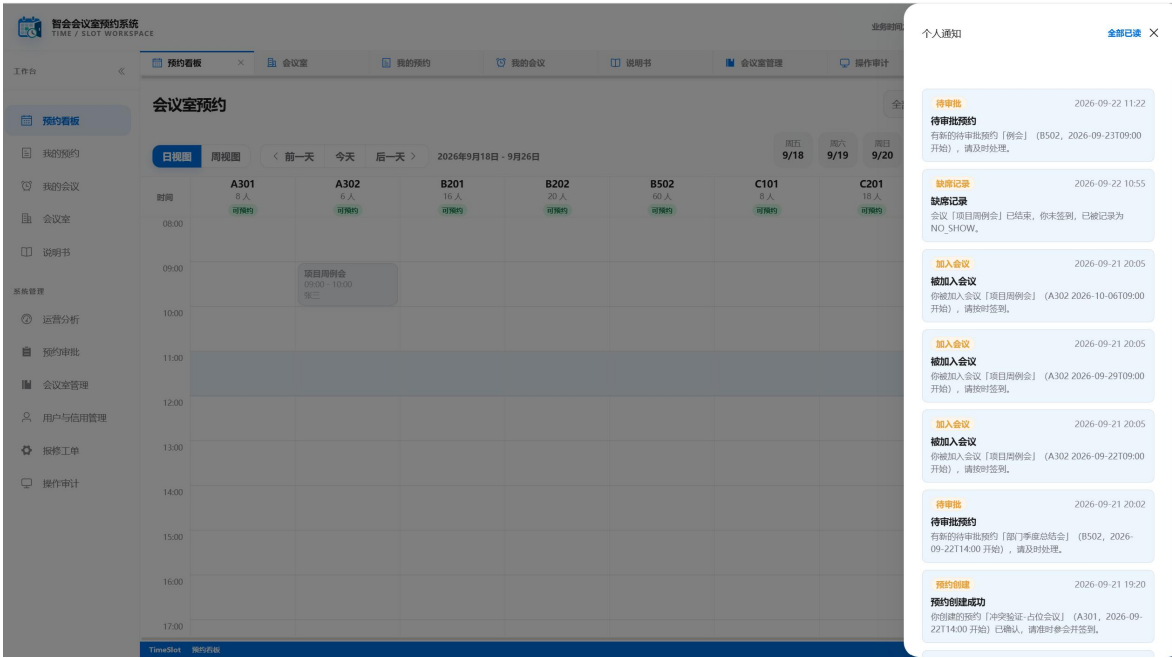


图 6.33 会议通知页面

6.7.18 关键页面业务闭环总结

通过上述页面，系统形成了从资源准备到预约执行和结果统计的完整业务闭环。普通用户侧的主要链路为：登录 → 查询会议室 → 条件筛选与时段可用查询 → 提交预约 → 参数与容量校验 → 时间冲突检测 → 查看预约 → 改期或更换会议室 → 取消预约 → 会议实际使用登记 → 查看相关记录与通知。管理员侧则通过会议室资源维护、状态管理、删除保护和统计查询保证系统基础资源与业务数据的一致性。

周期性会议、参会人名单和会议通知在基础预约流程之上扩展了批量预约和会议协作能力。系统各页面主要承担操作入口、数据展示和结果反馈职责，时间冲突、容量限制、状态限制、删除保护等关键业务规则由后端统一裁决，从而保证不同操作入口遵循一致的业务约束。

6.7.19 截图覆盖检查表

表 6.5 汇总了任务书要求与本章各节的对应关系，可用于核对关键功能的截图覆盖面。

表 6.5 截图覆盖检查表

任务书要求	6.7 对应内容	截图要求
会议室台账与列表	6.7.3	会议室台账
位置/容量/设备筛选	6.7.4	筛选条件与结果
指定日期和时段可用查询	6.7.4	时间条件与可用结果
预约申请	6.7.5	表单与成功结果
参数完整性与时间合法性	6.7.5	参数错误示例
参会人数容量限制	6.7.5	超容量被阻止
时间冲突检测	6.7.6	重叠冲突提示
首尾相接不冲突	6.7.6	连续预约成功
预约查看	6.7.7	列表/详情
改期及重新冲突检测	6.7.8	改期成功+冲突失败
更换会议室	6.7.8	会议室变更结果
取消预约与时段释放	6.7.9	取消+重新可约
已开始不可修改/取消	6.7.9	状态限制提示
实际开始/结束/参会人数	6.7.10	会议使用登记
会议室资源维护	6.7.11	新增/编辑/状态
未来预约存在时禁止删除	6.7.12	删除保护提示
统一异常反馈	6.7.13	无堆栈的业务提示
统计接口/页面（选做）	6.7.14	统计结果
周期性会议（选做）	6.7.15	周期规则+批量结果
参会人名单（选做）	6.7.16	名单维护
会议通知（选做）	6.7.17	通知记录

7 系统测试及结果

7.1 测试目标与环境

测试以验证系统功能正确性与并发正确性为核心目标：功能测试以接口为单位，验证预约创建、审批流转、冲突拒绝与各类异常；并发测试重点验证同一房间同一时段的双重占用防护与重复提交防护。测试在本地开发环境进行，后端服务与 MySQL 数据库以单节点方式运行，通过接口调用直接验证业务逻辑，覆盖正常路径、边界路径与异常路径。

测试在统一的开发环境中进行，被测对象为运行在本地的前后端服务与数据库，接口测试直接对后端接口发起请求，界面测试通过浏览器操作进行。测试环境的关键信息如表 7.1 所示。

表 7.1 测试环境

项目	内容
运行环境	单节点开发机，顺序启动数据库、后端与前端
后端	Spring Boot 应用，接口端口 8080
前端	开发服务器，页面端口 5173，接口经代理转发
数据库	MySQL 8，使用独立的开发库
测试账号	包含管理员与普通用户，用于验证角色边界
数据准备	覆盖免审批与需审批两类分类，以及多种容量与设备组合的会议室

7.2 功能测试

功能测试覆盖系统的主要业务路径，主要测试用例及预期结果如表 7.2 所示。

测试项	操作	预期结果
正常创建（免审）	对免审分类房间发起合规预约	状态直接为已确认，返回成功
正常创建（需审批）	对需审批分类房间发起合规预约	状态为待审批，通知管理员
时间冲突	对同一房间重叠时段再次预约	被拒绝并返回冲突提示
容量越界	将房间容量改至分类区间之外	被拒绝并返回容量区间提示
并发双占	两个请求同时抢同一时段	仅一个成功，另一个冲突拒绝
重复提交	同一请求标识重复提交	幂等，返回首次结果
审批流转	管理员审批通过/驳回	状态正确迁移并通知

资格不足	信用分低于门槛用户发起预约	被拒绝
------	---------------	-----

表 7.2 主要功能测试用例

功能测试用例按“正常流程、参数缺失、业务规则阻止、时间冲突、并发与幂等”五类设计，如表 7.2 所列。用例设计的原则是让每条业务规则至少有一条对应用例，使规则是否真正生效可以被直接证明，而不是依赖代码阅读来推断。

执行时逐条比对实际结果与预期结果。经实测，正常流程、参数校验与业务规则阻止三类用例全部符合预期；时间冲突类用例中，完全包含、部分重叠与完全相同三种情形均被正确拒绝，首尾相接的情形被正确放行，说明半开区间语义在实现中得到了准确体现；并发与幂等类用例中，并发抢占同一时段时仅有一个请求成功，相同请求标识的重复提交只生成一条记录。

7.3 接口测试

接口测试针对系统的关键接口，验证其“输入—处理—输出”是否符合约定。测试重点包括：统一响应结构（状态码、提示信息、数据体）是否一致；错误码是否按业务域正确区分，如参数校验失败、鉴权失败与资源冲突；受保护接口在缺少令牌或使用无效令牌时是否被正确拒绝；管理类接口在普通用户身份下是否返回禁止。接口测试可通过自动化脚本批量发起请求并比对响应，从而在联调阶段快速发现前后端约定不一致的问题。

接口测试关注接口在“输入—处理—输出”三个环节的表现是否符合约定，重点验证三点：统一响应结构是否在所有接口上保持一致；错误码是否按业务域正确区分；业务规则拒绝时是否返回可读且可操作的原因，而不是笼统的失败提示。测试要点与观察内容如表 7.3 所示。

表 7.3 接口测试要点

测试要点	观察内容
响应结构一致性	所有接口均返回状态码、提示信息与数据体三部分，成功与失败的结构一致
业务错误码区分	容量超限、时间冲突、状态不允许、删除被阻止等返回不同的业务错误码
状态码对应关系	参数问题返回 400、越权返回 403、冲突返回 409，均不带异常堆栈

测试要点	观察内容
提示可读性	失败提示包含可直接据以调整操作的信息，如冲突会议的主题与占用时段
鉴权行为	未携带令牌返回鉴权失败，普通用户访问管理接口被拒绝
幂等行为	相同请求标识重复提交返回首次处理结果，不产生第二条记录

7.4 并发与边界测试

针对并发与边界场景，设计了如下测试用例，见表 7.4。

表 7.4 并发与边界测试用例

测试项	方法与步骤	预期结果
相邻时段预约	先约 10:00-11:00，再约 11:00-12:00	均成功（半开区间不冲突）
部分重叠预约	先约 10:00-11:00，再约 10:30-11:30	第二次被拒绝
完全包含预约	先约 09:00-12:00，再约 10:00-11:00	第二次被拒绝
并发抢约	两个请求同时抢同一房间同一时段	仅一个成功，另一个冲突拒绝
重复提交	相同请求标识重复提交	幂等，仅生成一条记录
提前天数边界	预约时间超出分类允许的提前天数	被拒绝并给出提示
时长上限	预约时长超过分类单次最长时长	被拒绝并给出提示

并发与边界测试针对系统最容易出错的区域设计，用例见表 7.4。并发场景采用多请求同时发起的方式构造，重点观察是否出现同一时段被两条预约同时占用的情况；边界场景则围绕时间区间的相接与包含关系展开，逐一验证判定式在各种临界情形下的表现。

测试结果表明：在本轮并发验证范围内，同一时段的抢占请求经行锁串行进入临界区，后到者复查冲突后被正确拒绝，未观察到双重占用；相邻时段（首尾相接）的两次预约均成功，符合半开区间语义；超出提前天数与超出单次最长时长的请求均被拒绝并给出对应提示。

7.5 异常与安全测试

对异常与安全场景进行了如下验证：管理员接口在普通用户令牌下被拒绝（越权防护）；未携带或使用过期令牌的请求被拒绝（鉴权防护）；对同一分类使用重复名称被拒绝；对非法时间区间（结束时间早于开始时间）被拒绝。上述场景均返回明确的错误提示，未出现系统崩溃或信息泄露。

异常与安全测试覆盖越权访问、鉴权失效、重复数据与非法输入四类场景，测试项与预期结果如表 7.5 所示。这类测试的目的不只是确认功能可用，更是确认系统在异常输入下不会泄露内部信息、也不会写入不正确数据。

表 7.5 异常与安全测试项

测试项	操作	预期结果
越权访问	以普通用户令牌访问管理类接口	被拒绝并提示无权限执行该操作
鉴权失效	不携带令牌或使用过期令牌访问业务接口	被拒绝并要求重新登录
重复数据	创建与现有分类同名的分类	被拒绝并提示名称已存在
非法时间输入	提交结束时间早于开始时间的预约	返回参数错误并拒绝创建
异常信息脱敏	触发服务端异常后检查响应内容	仅提示服务器内部错误，不含堆栈与数据库信息

7.6 性能测试

性能测试关注的不是绝对吞吐量，而是系统在并发请求下是否仍能保持正确性与可接受的响应表现。测试方法是对预约创建等核心接口施加并发压力，观察响应时间分布、成功率与冲突拒绝率；同时观察冲突检测相关查询在不同数据量下的耗时变化，以判断索引是否被有效使用。观察指标与关注点如表 7.6 所示。

表 7.6 性能测试观察指标

观察指标	关注点
平均响应时间	核心接口在并发请求下的平均耗时是否保持在可接受范围
成功率与冲突拒绝率	并发抢占时段时，成功数是否恰为一条、其余是否均以冲突被拒绝
并发下正确性	高压请求后检查预约表，确认不存在同一房间同一时段的两条有效预约

观察指标	关注点
查询耗时	冲突检测与可约查询在数据量增长后的耗时变化，用于验证索引有效性

由于测试在单节点开发环境进行，测试机自身的资源也会影响结果，因此这里更看重“并发过程中是否出现错误结果”这一确定性结论，而不是把响应时间作为绝对指标。测试过程中未观察到并发导致的数据不一致；接口耗时主要受数据库查询影响，冲突检测与可约查询均命中了相应索引。

7.7 缺陷记录与修复

测试过程中发现并修复了两处缺陷，两者都属于“只在特定路径上才暴露”的问题，因此对测试用例的覆盖面提出了直接的改进要求。

第一处缺陷出现在管理员编辑会议室的场景：当修改会议室并调整其所属分类或容量时，接口返回服务器内部错误。经定位，原因是在查询分类信息时遗漏了容量相关的字段，导致容量值取到空值，随后在校验“容量是否落在分类区间之内”时对空值做了比较，抛出异常并被兜底处理为 500。修复方式是补齐该查询中容量字段的返回，使校验能够拿到真实取值。这一缺陷的启示是：容量校验虽然在创建会议室时已经验证通过，但编辑路径同样会走到这段校验，用例设计必须覆盖“同一规则的多个入口”。

第二处缺陷出现在按时段查询空闲会议室的场景：请求带时段参数时接口返回服务器内部错误。经定位，原因在数据访问层的语句写法上——该语句中使用的比较符号被转义成了字符实体形式，而这一段语句缺少相应的解析包装，导致数据库收到的是字面量而非比较运算，语句在解析阶段即报错。修复方式是改用原义的比较符号，并复核同类语句中是否存在相同写法。修复后实测：按时段查询空闲会议室返回 13 间可用房间（台账共 16 间，其余 3 间在该时段已被占用或不可预约），结果正确。

两处缺陷有一个共同点：都不是核心逻辑写错，而是“某条次要路径长期未被执行”。这提示在测试设计上不能只覆盖主流程，还应把管理端的编辑、按条件查询等路径纳入必测范围。修复完成后，相关用例一并补充到回归清单中。

7.8 新增能力补充测试

本版新增的会议室条件筛选与空闲查询、会议室删除保护、审批超时自动失效、周期性预约批量创建、会议实际使用记录登记以及运营统计三项口径，均按“一条规则至少对应用例”的原则补充了测试条件，并与既有条件合并为一份可逐条核对的清单，见交付文档《测试条件清单与执行结果》。该清单按时间冲突、正常流程、参数缺失与参数错误、业务规则阻止、条件筛选与空闲查询、并发与一致性、周期性预约与统计、会议实际使用记录八类组织，共 46 条，每条均标注预期结果与验证方式（对应的单元用例名或接口实测场景编号）。

除单元与切片测试外，本轮还以接口为单位完成了 15 个场景的实测，覆盖条件筛选与空闲查询的参数边界、周期性预约的成功与失败路径、会议室删除保护、会议实际使用记录以及幂等重放。下面按新增能力择要列出测试条件与实测结论。

表 7.7 本轮新增能力的测试条件与结果（摘要）

测试条件	预期结果	实际结果
按设施关键字与位置关键字筛选会议室台账	仅返回同时命中的会议室	通过：设备“投影”命中 7 间，位置“教学楼”命中 5 间
指定时段空闲查询，需剔除已有有效预约与非可预约状态的会议室	仅返回该时段空闲且处于可预约状态的会议室	通过：指定日期 13:00—16:00 返回 13 间，台账 16 间中其余 3 间被正确剔除
空闲查询的结束时间不晚于开始时间	按次日结束推导（跨天），与预约的时间约定一致	通过：22:00 至次日 02:00 判定正常
查询参数缺失、格式错误（如日期写成字母）、容量取值非法	返回 400 与可读提示，不落入兜底 500	通过：为此新增了参数类型不匹配的异常映射
删除存在未来预约、或存在历史记录会议室	返回业务错误并提示原因，建议改为“停用”	通过：未来预约与历史预约两级保护均按预期阻止删除
按周重复批量创建预约：任一一周冲突或超出提前天数；周数越界	全部通过则按周写入多场；任一一周不满足即整批拒绝、零落库	通过：越界返回参数错误，冲突时提示含冲突主题与占用时段
周期性请求的幂等重放（同一请求标识重复提交）	返回首次创建的预约，不新增场次	通过：重放后场次数量不变
待审批预约在时段结束后仍未审批	自动迁为已驳回，并向预约人推送驳回通知	通过：单条失败不影响同批其他预约
统计三项指标的口径（使用率、日均会议时长、爽约率）	按约定公式计算，可与明细逐条核对	通过：使用率 = 占用时长 ÷ (天数 × 每日开放时长)，并与明细一致
组织者登记已确认且已结束会议的实际开始/结束时间与实际参会人数	写入实际使用记录；重复提交修正同一条记录	通过：重复登记后记录条数不变，仅更新内容与修正时间

测试条件	预期结果	实际结果
实际结束时间不晚于开始时间、实际参会人数为负、结束时间晚于当前业务时间	返回参数或业务错误，不写入记录	通过：三类非法输入均被拒绝，且无落库副作用
非组织者的普通用户登记实际使用记录	返回禁止（403），不写入记录	通过：权限守卫按预期拦截，仅组织者与管理员可登记

并发与边界方面，交付脚本中提供了一键运行的并发演示：两个请求同时抢占同一间会议室的同一时段，观察是否只有一个成功、另一个被冲突拒绝，同时可用自定义时段复现场景。这类端到端并发验证目前需要单独运行，尚未纳入默认测试任务，已记入 8.3 节的改进方向。

本轮测试共发现并修复四处问题：一是新增只读查询的数据访问接口后，切片测试的上下文加载失败，原因是该测试未纳入数据访问层的依赖，已在测试配置中补齐；二是查询参数格式错误（如日期写成字母）原先落入兜底处理器返回 500，与“参数错误应返回参数错误提示”的要求不符，已新增参数类型不匹配的异常映射并返回 400；三是会议室删除被业务规则阻止时，错误码未登记 HTTP 状态，被兜底处理为 500，与“业务规则导致的拒绝不使用服务器内部错误”的设计原则不符，已将删除保护归入 409 冲突状态并补充状态断言；四是接口实测中携带中文的请求体在命令行工具下按本地代码页发送导致解析失败，属于测试环境问题而非代码缺陷，改用脚本化请求后复测通过。

会议实际使用记录共补充 3 条测试条件（对应 5 个单元用例），编号 T-44 至 T-46，已合并进交付文档《测试条件清单与执行结果》第 2.8 节。条件覆盖“登记成功与幂等修正”“三类非法输入被拒绝”“非组织者越权被拒绝”三条规则，与实现中的守卫一一对应。

7.9 测试结果分析

综合功能测试、接口测试、并发与边界测试以及异常与安全测试的结果，系统达到了预期的设计目标，具体体现在以下几个方面。

- 冲突判定的语义正确：完全包含、部分重叠与完全相同均被拒绝，首尾相接被放行，半开区间语义在各条目测中表现一致。

- 在本轮测试范围内，并发控制结果符合预期：行锁、幂等键与乐观锁分别覆盖并发抢占、重复提交和并发修改场景，未观察到双重占用或重复提交。
- 已纳入核心链路的业务规则在主要入口保持一致：创建、改期、审批与删除等操作按照统一规则进行校验，本轮测试未发现明显的入口漏校验问题。
- 错误处理分层清晰：业务拒绝返回可读原因与恰当状态码，未预期异常不泄露内部信息。
- 仍存在的待完善之处：并发抢占的端到端集成测试尚未纳入默认测试任务；会议室按星期的开放规则目前仅作为配置数据暴露，尚未强制参与创建校验；周期性预约目前只支持按周重复，尚不支持隔周、按月或跳过节假日。详见 3.3 节与 8.3 节。

8 结论

8.1 工作总结

本实践完成了会议室预约系统从需求分析、架构设计、核心业务逻辑实现到前后端联调与测试的完整过程。系统以“预约时段不被重复占用”为核心正确性目标，实现了预约创建全流程管控、基于半开区间的冲突检测、由会议室分类驱动的状态机，以及行锁、幂等键与乐观锁三重并发保护；此外还实现了审批与强制取消、参会人与出勤记录、站内通知、运营统计与操作审计等配套能力。

在工程层面，本实践完成了一次较完整的模块化拆分：六个业务模块之间通过接口协作，三对端口把最容易产生循环依赖的三处协作关系解耦，使模块依赖保持单向。测试环节发现并修复了两处缺陷，并把相关路径补充进回归范围。

在完成核心预约链路之后，本版的工作重心转向两处：一是把已经具备的能力补齐界面入口，二是让系统能够自行维护数据状态。前者包括为会议室条件筛选与指定时段空闲查询补充前端入口、为运营看板补齐使用率、日均会议时长与爽约率三项指标、为周期性会议提供按周批量创建能力；后者体现为待审批预约在时段结束后由定时任务自动作废并通知预约人。上述改动均未改变既有架构，而是沿模块边界与端口协作进行扩展，也从侧面印证了“规则集中、边界清晰”这一组织方式的收益。

8.2 系统特点与设计亮点

- 架构层面：通过端口与适配器约束跨模块协作，降低循环依赖风险，并保持主要依赖关系单向；
- 正确性层面：行锁、幂等键与乐观锁分别处理并发抢占、重复提交与并发修改，提高预约数据一致性；
- 体验层面：半开区间冲突模型符合会议室使用直觉；
- 可观测层面：统一异常映射正确状态码，审批与强制取消均留审计痕迹。

系统在四个方面形成了较为明确的特点。在架构层面，端口与适配器的使用不是形式化的分层包装，而是针对具体的循环依赖风险给出的解决方案，依赖方向可以通

过代码结构直接验证。在正确性层面，三重并发保护分别对应三类并发问题，而不是简单地叠加多种锁；冲突判定只依赖一个统一的判定式，所有入口共用，避免口径不一致。

在体验层面，半开区间的冲突模型符合会议室连续使用的直觉，用户不必理解“区间语义”就能正确使用；冲突提示直接给出冲突会议的主题与占用时段，把“失败原因”变成了“下一步怎么做”的指引。在可观测层面，审批与强制取消均留下审计记录，错误响应按语义分层，使管理动作可追溯、故障可快速归类。

- 交付层面：把已有能力补上界面入口，并让统计指标的口径可解释、可核对；
- 自维护层面：由定时任务为“待审批”提供一条退出路径，避免状态长期滞留。

8.3 不足与展望

- 按星期的会议室开放规则目前作为配置数据暴露，尚未强制参与创建校验，可后续接入；
- 通知目前以站内通知为主，可扩展邮件/短信等渠道；
- 运营看板可进一步增加按部门、按分类的统计维度；
- 可补充改期冲突重检与审批重算的端到端自动化测试。

受课程实践的时间与范围限制，系统仍存在若干不足。已识别的问题及可行的改进方向如表 8.1 所示。需要说明的是，“会议室筛选与空闲查询的前端入口”“周期性会议批量预约”“运营统计中的使用率与爽约率”“实际参会人数的独立登记”等此前列为待补充的能力，本版已经补齐，因此表 8.1 中的改进方向集中在规则完整性与测试覆盖，而不是功能缺口。

表 8.1 已识别的不足与改进方向

不足之处	影响	改进方向
并发抢占的端到端集成测试未纳入默认测试任务	并发抢占场景由门控集成测试覆盖，需显式开启环境变量才会执行，默认回归不包含该场景	把并发集成测试接入默认测试任务或 CI，使该场景随常规回归自动执行
按星期的开放规则未参与创建校验	开放规则可作为配置维护，但创建预约时尚未强制校验	把按星期的开放规则纳入创建与改期的时间校验

软件学院软件实践说明书

不足之处	影响	改进方向
周期性预约的重复规则较单一	目前支持按周重复 2 至 8 周，尚不支持隔周、按月或跳过节假日	扩展重复规则的表达，并支持对单场次单独调整
运营统计缺少组织维度	统计可按会议室与时段观察，尚未按部门或分类拆分	在现有聚合查询上增加部门与分类维度
通知渠道单一	仅提供站内通知，离线用户无法及时获知	在通知表之上增加投递渠道层，扩展邮件或短信

后续工作可从四个方向继续推进：一是增强预约规则的表达力，把按星期的开放规则、节假日例外与周期性会议纳入统一的时间校验；二是扩展通知的投递渠道，在现有通知生成逻辑之上增加邮件或短信通道，使通知能够触达未登录用户；三是完善运行观测手段，在审计日志之外补充接口耗时与错误码分布统计；四是扩大自动化测试覆盖，把并发抢占、改期冲突重检与审批状态重算等复杂场景纳入默认回归范围。

这些改进的共同前提是保持现有的模块边界与规则集中策略：只要规则仍然只有一处权威实现，增加功能就是扩展而非重构；反之，如果为了快速实现而把判断散落回去，后续每一次变更都会重新付出理解与验证的成本。这也是本次实践在工程方法上最主要的收获。

8.4 实践体会

本次软件实践完整贯通了从需求分析、架构设计到编码实现与测试验证的链路。最主要的收获在于认识到“正确性”是业务系统的核心：会议室预约的业务规则并不复杂，真正的困难集中在并发与边界——同一时段被重复承诺、相邻时段的判定、重复提交的幂等，都需要数据层与业务层协同设计，而无法仅依靠界面约束解决。其次，通过引入端口与适配器，体会到良好架构的价值不在于代码数量，而在于依赖关系清晰：把易变的部分收敛到接口之后，系统的可维护性与可测试性都会明显提升。最后，在排查“编辑会议室报错”的过程中，掌握了沿请求处理链逐层定位、由现象反推根因的方法，也进一步意识到空值与边界防御在编码中的重要性。

本次实践使我认识到，业务系统的复杂度往往不在于功能数量，而在于规则的“入口一致性”与“并发正确性”。会议室预约的表单只有几个字段，但同一条规则会在创建、改期、审批与删除等多个入口被使用；若每个入口各写一份判断，规则迟

早出现分歧。把规则集中到状态机与统一的校验入口，表面上增加了抽象层次，实际减少的是后续每一次变更的排查与验证成本。

另一处收获来自缺陷修复的过程。上述缺陷均非算法错误，而是特定路径长期未被覆盖，暴露的是测试用例设计的覆盖面问题。由此认识到，测试用例的价值不在于数量，而在于是否围绕“规则的入口”与“数据的边界”组织：一条规则有几个入口、一种判定有几个临界点，决定了用例应当覆盖到哪里。

参考文献

- [1] 张海藩, 牟永敏. 软件工程导论 (第 6 版) [M]. 北京: 清华大学出版社, 2013.
- [2] Craig Walls. Spring Boot 实战 [M]. 北京: 人民邮电出版社, 2016.
- [3] 尤雨溪 等. Vue.js 设计与实现 [M]. 北京: 人民邮电出版社, 2022.
- [4] 唐汉明 等. 深入浅出 MySQL (第 2 版) [M]. 北京: 人民邮电出版社, 2014.
- [5] Martin Fowler. 企业应用架构模式 [M]. 北京: 机械工业出版社, 2004.
- [6] 王珊, 萨师煊. 数据库系统概论 (第 5 版) [M]. 北京: 高等教育出版社, 2014.

致 谢

本软件实践说明书的完成，离不开指导教师的悉心指导与同学的帮助。在选题、需求分析到系统实现与测试的整个过程中，指导教师给予了耐心的方法指引与关键难点点拨，使我对软件工程的理论与方法有了更切实的体会。同时感谢团队成员在模块化划分、接口约定与联调过程中的协作配合。谨向所有给予支持与帮助的老师、同学致以诚挚的谢意。

附录说明

本附录作为《会议室预约与时间冲突检查系统》软件实践说明书的补充材料，用于集中给出正文中不宜大篇幅展开的工程性资料。内容依据当前代码、数据库结构快照、核心业务契约、接口实现与测试执行记录整理。

附录 A 主要数据库表结构

当前数据库采用 MySQL 8.x、InnoDB 与 utf8mb4，完整结构快照为 v1.11，共 17 张业务及配置表。预约状态等业务状态主要由应用层维护，数据库通过主键、唯一键、外键、索引和 CHECK 约束保证基础数据完整性。

表 A.1 数据库表总览

序号	表名	所属域	主要职责
1	department	identity	部门基础信息
2	sys_user	identity	用户、角色、部门、信用与限制状态
3	user_violation	identity	信用变动、违规与限制记录
4	room_category	resource	会议室分类及预约规则
5	meeting_room	resource	会议室资源主表
6	room_facility	resource	会议室设施
7	room_open_rule	resource	按星期配置的开放时间
8	room_maintenance	resource	会议室维护计划
9	facility_repair_ticket	resource	设施报修工单
10	reservation	reservation	预约事实与生命周期
11	approval_record	administration	审批行为历史
12	meeting_execution	meeting	会议实际使用结果
13	reservation_attendee	meeting	参会人与出勤记录
14	notification	meeting	站内通知
15	operation_log	administration	关键管理操作审计
16	system_time_config	common	业务测试时钟单例配置
17	booking_window_config	common	全局可预约时间窗口单例配置

A.1 department

表 A.2 department 表结构

字段名	数据类型	允许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
dept_name	VARCHAR(50)	否	UNIQUE	部门名称
description	VARCHAR(200)	是	—	部门说明

软件学院软件实践说明书

字段名	数据类型	允许空	键/约束	说明
created_at	DATETIME	否	DEFAULT CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	否	自动更新时间	更新时间

A.2 sys_user

表 A.3 sys_user 表结构

字段名	数据类型	允许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
username	VARCHAR(50)	否	UNIQUE	登录用户名
password	VARCHAR(100)	否	—	BCrypt 密码摘要
real_name	VARCHAR(50)	否	—	真实姓名
email	VARCHAR(100)	是	—	邮箱
phone	VARCHAR(20)	是	—	手机号
role	VARCHAR(20)	否	默认 USER	角色: USER/ADMIN
department_id	BIGINT	是	FK→department.id	所属部门
credit_score	INT	否	默认 100	信用分
restricted_until	DATETIME	是	—	预约限制截止时间
status	TINYINT	否	默认 1	账号状态: 1 正常/0 禁用
created_at	DATETIME	否	默认当前时间	创建时间
updated_at	DATETIME	否	自动更新时间	更新时间

A.3 user_violation

表 A.4 user_violation 表结构

字段名	数据类型	允许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
user_id	BIGINT	否	FK→sys_user.id	用户 ID
violation_type	VARCHAR(20)	否	—	信用/限制/账号治理类型
credit_change	INT	否	默认 0	信用分变化量
reason	VARCHAR(50)	否	—	原因
operator_id	BIGINT	是	—	操作人: NULL 表示系统

字段名	数据类型	允 许空	键/约束	说明
created_at	DATETIME	否	默认当前时间	记录时间

A.4 room_category

表 A.5 room_category 表结构

字段名	数据类型	允 许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
category_name	VARCHAR(50)	否	UNIQUE	分类名称
min_capacity	INT	否	—	分类容量下限
max_capacity	INT	否	—	分类容量上限
approval_required	TINYINT	否	默认 0	是否需要审批
max_duration_minutes	INT	否	默认 1440	单次最大预约时长
advance_days	INT	否	默认 7	最大提前预约天数
description	VARCHAR(500)	是	—	分类说明
created_at	DATETIME	否	默认当前时间	创建时间
updated_at	DATETIME	否	自动更新时间	更新时间

A.5 meeting_room

表 A.6 meeting_room 表结构

字段名	数据类型	允 许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
category_id	BIGINT	否	FK→room_category.id	所属分类
room_name	VARCHAR(50)	否	UNIQUE	会议室名称
location	VARCHAR(100)	是	—	位置
capacity	INT	否	—	实际容量
status	TINYINT	否	默认 1	资源状态映射值
description	VARCHAR(500)	是	—	说明
created_at	DATETIME	否	默认当前时间	创建时间
updated_at	DATETIME	否	自动更新时间	更新时间

A.6 room_facility

表 A.7 room_facility 表结构

字段名	数据类型	允 许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
room_id	BIGINT	否	FK→meeting_room.id	会议室 ID
facility_name	VARCHAR(50)	否	UNIQUE(room_id,name)	设施名称
quantity	INT	否	默认 1	设施数量
description	VARCHAR(200)	是	—	设施说明

A.7 room_open_rule

表 A.8 room_open_rule 表结构

字段名	数据类型	允 许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
room_id	BIGINT	否	FK→meeting_room.id	会议室 ID
weekday	TINYINT	否	CHECK 1~7	星期：1 周一...7 周日
open_time	TIME	否	—	开放时间（含）
close_time	TIME	否	CHECK close>open	关闭时间（不含）
enabled	TINYINT	否	默认 1	是否启用
created_at	DATETIME	否	默认当前时间	创建时间
updated_at	DATETIME	否	自动更新时间	更新时间

A.8 room_maintenance

表 A.9 room_maintenance 表结构

字段名	数据类型	允 许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
room_id	BIGINT	否	FK→meeting_room.id	会议室 ID
reason	VARCHAR(200)	否	—	维护原因
start_time	DATETIME	否	—	维护开始
end_time	DATETIME	否	CHECK end>start	维护结束
status	VARCHAR(20)	否	默认 PLANNED	PLANNED/FINISHED

软件学院软件实践说明书

字段名	数据类型	允 许空	键/约束	说明
created_by	BIGINT	否	—	创建管理员 ID
created_at	DATETIME	否	默认当前时间	创建时间
updated_at	DATETIME	否	自动更新时间	更新时间

A.9 facility_repair_ticket

表 A.10 facility_repair_ticket 表结构

字段名	数据类型	允 许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
room_id	BIGINT	否	FK→meeting_room.id	会议室 ID
facility_id	BIGINT	是	—	设施 ID, 可空表示整室
facility_name	VARCHAR(50)	否	—	报修时设施名称快照
issue	VARCHAR(500)	否	—	故障描述
status	VARCHAR(20)	否	默认 OPEN	OPEN/RESOLVED
reporter_id	BIGINT	否	—	报修人 ID
reporter_name	VARCHAR(50)	否	—	报修人姓名
created_at	DATETIME	否	默认当前时间	报修时间
resolved_at	DATETIME	是	—	解决时间
resolve_remark	VARCHAR(500)	是	—	处理说明

A.10 reservation

表 A.11 reservation 表结构

字段名	数据类型	允 许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
request_id	VARCHAR(100)	否	UNIQUE(user_id,request_id)	创建幂等键
reservation_no	VARCHAR(32)	否	UNIQUE	预约业务单号
room_id	BIGINT	否	FK→meeting_room.id	会议室 ID
user_id	BIGINT	否	FK→sys_user.id	预约人 ID
title	VARCHAR(100)	否	—	会议主题

软件学院软件实践说明书

字段名	数据类型	允 许空	键/约束	说明
start_time	DATETIME	否	—	计划开始时间
end_time	DATETIME	否	CHECK end>start	计划结束时间
participant_count	INT	否	默认 1	参与人数
status	VARCHAR(20)	否	CHECK 四状态	PENDING/CONFIRMED/REJECTED/CANCELLED
remark	VARCHAR(500)	是	—	预约备注
cancel_reason	VARCHAR(500)	是	—	取消原因
version	INT	否	默认 0	乐观锁版本
created_at	DATETIME	否	默认当前时间	创建时间
updated_at	DATETIME	否	自动更新时间	更新时间

A.11 approval_record

表 A.12 approval_record 表结构

字段名	数据类型	允 许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
reservation_id	BIGINT	否	FK→reservation.id	预约 ID
approver_id	BIGINT	否	FK→sys_user.id	审批人 ID
action	VARCHAR(20)	否	—	APPROVE/REJECT
remark	VARCHAR(500)	是	—	审批意见
created_at	DATETIME	否	默认当前时间	审批时间

A.12 meeting_execution

表 A.13 meeting_execution 表结构

字段名	数据类型	允 许空	键/约束	说明
reservation_id	BIGINT	否	PK, FK→reservation.id	预约 ID，一对一
actual_start_time	DATETIME	否	—	实际开始时间
actual_end_time	DATETIME	否	CHECK end>start	实际结束时间
actual_attendee_count	INT	否	CHECK >=0	实际参会人数
recorded_by	BIGINT	否	FK→sys_user.id	登记人
created_at	DATETIME	否	默认当前时间	首次登记时间

软件学院软件实践说明书

字段名	数据类型	允 许空	键/约束	说明
updated_at	DATETIME	否	自动更新时间	最后修正时间

A.13 reservation_attendee

表 A.14 reservation_attendee 表结构

字段名	数据类型	允 许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
reservation_id	BIGINT	否	FK→reservation.id	预约 ID
user_id	BIGINT	否	FK→sys_user.id	参会用户 ID
attendee_role	VARCHAR(20)	否	默认 ATTENDEE	ORGANIZER/ATTENDEE
attendance_status	VARCHAR(20)	否	默认 EXPECTED	EXPECTED/CHECKED_IN/CHECKED_OUT/NO_SHOW
check_in_at	DATETIME	是	—	签到时间
check_out_at	DATETIME	是	—	签退时间
created_at	DATETIME	否	默认当前时间	加入时间
updated_at	DATETIME	否	自动更新时间	更新时间

A.14 notification

表 A.15 notification 表结构

字段名	数据类型	允 许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
user_id	BIGINT	否	FK→sys_user.id	接收人
type	VARCHAR(30)	否	—	通知类型
title	VARCHAR(100)	否	—	通知标题
content	VARCHAR(500)	否	—	通知内容
reservation_id	BIGINT	是	—	关联预约
dedup_key	VARCHAR(120)	否	UNIQUE(user_id,dedup_key)	业务去重键
is_read	TINYINT	否	默认 0	是否已读
read_at	DATETIME	是	—	已读时间
created_at	DATETIME	否	默认当前时间	创建时间

A.15 operation_log

表 A.16 operation_log 表结构

字段名	数据类型	允 许空	键/约束	说明
id	BIGINT	否	PK, AUTO_INCREMENT	主键 ID
user_id	BIGINT	否	FK→sys_user.id	操作人
operation_type	VARCHAR(50)	否	—	操作类型
business_type	VARCHAR(50)	否	—	业务类型
business_id	BIGINT	否	—	业务 ID
content	VARCHAR(500)	否	—	操作内容
ip_address	VARCHAR(50)	是	—	操作 IP
created_at	DATETIME	否	默认当前时间	操作时间

A.16 system_time_config

表 A.17 system_time_config 表结构

字段名	数据类型	允 许空	键/约束	说明
id	TINYINT	否	PK,CHECK id=1	单例主键
fixed_time	DATETIME	是	NULL=实时	固定业务时间
updated_at	DATETIME	否	自动更新时间	更新时间

A.17 booking_window_config

表 A.18 booking_window_config 表结构

字段名	数据类型	允 许空	键/约束	说明
id	TINYINT	否	PK,CHECK id=1	单例主键
start_minute	INT	否	CHECK 范围	相对 00:00 的可预约开始分钟
end_minute	INT	否	CHECK <=2880	可预约结束分钟，可跨次日
updated_at	DATETIME	否	自动更新时间	更新时间

附录 B REST API 接口清单

系统所有真实业务请求统一使用 /api 前缀，采用 JSON 传输；登录接口公开，其余接口按 USER/ADMIN 角色与业务所有权进行授权。下表依据当前后端 Controller 映射整理，共 67 个 HTTP Mapping。

权限说明：公开表示无需登录；USER/ADMIN 表示任意已登录用户；ADMIN 表示管理员。部分 USER/ADMIN 写操作还会在 Service 层校验预约所有权、组织者身份或业务状态。

表 B.1 REST API 接口清单

模块	Method	Path	权限	说明
管理/审批	GET	/api/admin/reservations	ADMIN	预约治理列表，可按状态筛选
管理/审批	GET	/api/admin/reservations/{id}	ADMIN	管理视角预约详情
管理/审批	POST	/api/admin/reservations/{id}/approve	ADMIN	审批通过 PENDING→CONFIRMED
管理/审批	POST	/api/admin/reservations/{id}/reject	ADMIN	审批驳回 PENDING→REJECTED
管理/审批	POST	/api/admin/reservations/{id}/force-cancel	ADMIN	管理员强制取消
管理/审计	GET	/api/admin/audit-logs	ADMIN	关键操作审计查询
管理/统计	GET	/api/admin/statistics	ADMIN	运营统计看板
管理/审计	GET	/api/admin/audit-logs/export	ADMIN	导出审计日志 CSV
公共配置	GET	/api/booking-window	USER/ADMIN	读取全局预约窗口
公共配置	PUT	/api/admin/booking-window	ADMIN	设置全局预约窗口
公共配置	GET	/api/system-time	USER/ADMIN	读取业务时间
公共配置	PUT	/api/admin/system-time	ADMIN	设置固定业务时间
公共配置	DELETE	/api/admin/system-time	ADMIN	恢复实时业务时间
身份	POST	/api/auth/login	公开	用户名密码登录并签发 JWT
身份	GET	/api/admin/departments	ADMIN	部门列表
身份	POST	/api/admin/departments	ADMIN	新增部门
身份	PUT	/api/admin/departments/{id}	ADMIN	修改部门
身份	GET	/api/admin/users	ADMIN	用户列表与筛选
身份	GET	/api/admin/users/{id}	ADMIN	用户详情
身份	GET	/api/admin/users/{id}/qualification	ADMIN	指定用户预约资格
身份	GET	/api/admin/users/{id}/violations	ADMIN	指定用户信用/违规记录
身份	PUT	/api/admin/users/{id}/credit	ADMIN	调整信用分
身份	PUT	/api/admin/users/{id}/restriction	ADMIN	设置/解除预约限制
身份	POST	/api/admin/users	ADMIN	新建用户
身份	PUT	/api/admin/users/{id}	ADMIN	修改用户资料与角色
身份	PUT	/api/admin/users/{id}/status	ADMIN	启用/停用账号
身份	PUT	/api/admin/users/{id}/password	ADMIN	管理员重置密码
身份	GET	/api/users/me	USER/ADMIN	当前登录用户
身份	GET	/api/users/me/qualification	USER/ADMIN	本人预约资格
身份	GET	/api/users/me/violations	USER/ADMIN	本人信用/违规记录
会议执行	GET	/api/meetings/{reservationId}/attendees	USER/ADMIN	参会人列表
会议执行	POST	/api/meetings/{reservationId}/attendees	USER/ADMIN	添加参会人；Service 校验组织者
会议执行	DELETE	/api/meetings/{reservationId}/attendees/{userId}	USER/ADMIN	移除参会人
会议执行	POST	/api/meetings/{reservationId}/check-in	USER/ADMIN	签到
会议执行	POST	/api/meetings/{reservationId}/check-out	USER/ADMIN	签退
会议执行	GET	/api/meetings/{reservationId}/attendance	USER/ADMIN	出勤情况
会议执行	GET	/api/meetings/{reservationId}/execution	USER/ADMIN	查询实际使用记录
会议执行	PUT	/api/meetings/{reservationId}/execution	USER/ADMIN	登记/修正实际使用记录
会议执行	GET	/api/meetings/my	USER/ADMIN	我的会议
通知	GET	/api/notifications	USER/ADMIN	通知列表，可筛未读
通知	GET	/api/notifications/unread-count	USER/ADMIN	未读通知数

软件学院软件实践说明书

模块	Method	Path	权限	说明
通知	POST	/api/notifications/{id}/read	USER/ADMIN	单条标记已读
通知	POST	/api/notifications/read-all	USER/ADMIN	全部标记已读
预约	GET	/api/reservations/calendar	USER/ADMIN	预约日历查询
预约	GET	/api/reservations/my	USER/ADMIN	我的预约
预约	POST	/api/reservations	USER/ADMIN	创建预约；支持幂等与周期预约
预约	GET	/api/reservations/{id}	USER/ADMIN	预约详情
预约	PUT	/api/reservations/{id}	USER/ADMIN	预约改期/修改
预约	POST	/api/reservations/{id}/cancel	USER/ADMIN	预约人取消未开始预约
资源	GET	/api/admin/room-categories	ADMIN	会议室分类列表
资源	POST	/api/admin/room-categories	ADMIN	新增会议室分类
资源	PUT	/api/admin/room-categories/{categoryId}	ADMIN	修改分类规则
资源	GET	/api/admin/repair-tickets	ADMIN	报修工单列表
资源	POST	/api/admin/repair-tickets/{ticketId}/resolve	ADMIN	处理报修工单
资源	POST	/api/admin/rooms	ADMIN	新增会议室
资源	PUT	/api/admin/rooms/{roomId}	ADMIN	修改会议室
资源	POST	/api/admin/rooms/{roomId}/status	ADMIN	切换资源状态
资源	DELETE	/api/admin/rooms/{roomId}	ADMIN	删除会议室；受预约引用保护
资源	PUT	/api/admin/rooms/{roomId}/open-rules	ADMIN	全量替换开放规则
资源	PUT	/api/admin/rooms/{roomId}/facilities	ADMIN	全量替换设施
资源	POST	/api/admin/rooms/{roomId}/maintenance	ADMIN	创建维护计划
资源	GET	/api/admin/rooms/{roomId}/maintenance	ADMIN	维护计划列表
资源	POST	/api/admin/rooms/{roomId}/maintenance/{planId}/finish	ADMIN	完成维护计划
资源	GET	/api/rooms	USER/ADMIN	会议室列表；位置/最小容量/设施筛选
资源	GET	/api/rooms/available	USER/ADMIN	指定日期和时段查询可用会议室
资源	GET	/api/rooms/{roomId}	USER/ADMIN	会议室详情
资源	POST	/api/rooms/{roomId}/repair-tickets	USER/ADMIN	提交设施/整室报修

附录 C 核心业务规则与状态转换

本附录集中列出系统中影响预约结果和资源一致性的核心规则。持久化状态与展示状态严格区分，关键业务裁决由后端完成，前端仅负责交互与展示。

C.1 预约持久化状态

表 C.1 预约持久化状态

状态	含义	占用时间资源	主要规则
PENDING 审批	特殊/受控分类预约待	是	可审批、驳回；未开始时可取消
CONFIRMED	预约正式成立	是	未开始可修改/取消；会议执行阶段可签到等
REJECTED	审批被拒绝	否	历史终态，不参与冲突
CANCELLED	有效预约被取消	否	历史终态，不参与冲突

C.2 展示状态推导

表 C.2 展示状态推导规则

条件	展示状态	说明
CONFIRMED 且 now < start_time	UPCOMING	即将开始
CONFIRMED 且 start_time ≤ now < end_time	IN_USE	进行中
CONFIRMED 且 now ≥ end_time	COMPLETED	已完成
PENDING / REJECTED / CANCELLED	保持原状态	直接展示持久化状态

C.3 时间冲突与资源占用规则

表 C.3 核心预约规则

规则项	约束说明
时间区间	统一采用半开区间 [start_time, end_time)，相邻时段可首尾衔接。
冲突公式	newStart < oldEnd AND newEnd > oldStart。
有效占用状态	仅 PENDING 与 CONFIRMED 参与冲突检测；REJECTED、CANCELLED 不占用时段。
并发临界区	创建预约时先锁定目标 meeting_room 行，再在锁内完成规则校验、冲突查询和写入。
创建幂等	requestId 标识一次逻辑提交；同一用户重复提交同一 requestId 返回既有业务结果。
修改并发	reservation.version 作为乐观锁版本，过期写入必须 fail-closed，要求刷新后重试。
容量规则	participant_count 不得超过 meeting_room.capacity。
资源状态	仅 AVAILABLE 会议室可接受新预约；MAINTENANCE、DISABLED 拒绝新预约。

规则项	约束说明
预约窗口	创建和改期受全局 booking_window_config 约束，可配置跨次日窗口。
分类规则	room_category 控制 approval_required、max_duration_minutes 与 advance_days。
周期预约	repeatWeeks 取 1~8；按周展开；批次中任一场规则或冲突失败则整批拒绝；提前天数只约束首场。

C.4 审批与取消状态转换

表 C.4 预约状态转换

当前状态	操作	目标状态	主要条件
PENDING	管理员通过	CONFIRMED	必须在会议开始前
PENDING	管理员驳回	REJECTED	驳回意见必填
PENDING	预约人取消	CANCELLED	仅本人且会议未开始
CONFIRMED	预约人取消	CANCELLED	仅本人且会议未开始
PENDING/CONFIRMED	管理员强制取消	CANCELLED	允许会议进行中强制取消；会议结束后拒绝

C.5 其他业务状态

表 C.5 其他业务状态转换

对象	状态转换	说明
会议室	AVAILABLE → MAINTENANCE / DISABLED	管理员切换；非 AVAILABLE 不接受新预约
维护计划	PLANNED → FINISHED	维护计划完成登记
报修工单	OPEN → RESOLVED	管理员处理；并发重复处理拒绝
参会人	EXPECTED → CHECKED_IN → CHECKED_OUT	正常签到签退链路
参会人	EXPECTED → NO_SHOW	预约结束后由执行逻辑判定，要求幂等
通知	未读 → 已读	支持单条已读和全部已读

附录 D 错误码与异常响应说明

后端采用统一 ApiResponse 结构和 ErrorCode 枚举表达业务结果，由全局异常处理器完成参数异常、认证异常、业务异常和未预期异常的统一映射。

表 D.1 统一错误码

错误码	HTTP	典型场景
SUCCESS	200/201	请求成功；创建类接口可返回 201
VALIDATION_ERROR	400	参数缺失、格式错误、时间/窗口/规则校验失败
UNAUTHORIZED	401	未登录、JWT 无效或认证失败
FORBIDDEN	403	角色不足、资源所有权不足或预约资格不足
RESOURCE_NOT_FOUND	404	用户、会议室、预约等资源不存在
ROOM_UNAVAILABLE	409	会议室维护、停用或当前不可预约
ROOM_CAPACITY_EXCEEDED	400	参与人数超过会议室容量
ROOM_DELETE_BLOCKED	409	会议室存在预约引用，禁止物理删除
RESERVATION_TIME_CONFLICT	409	同一会议室存在重叠的有效预约
RESERVATION_INVALID_STATE	409	当前预约状态/版本不允许执行目标操作
DUPLICATE_REQUEST	409	幂等键与既有请求语义冲突
INTERNAL_ERROR	500	未预期服务器异常

D.1 统一响应示例

成功响应：{"code":"SUCCESS","message":"success","data":{...}}

失败响应：{"code":"RESERVATION_TIME_CONFLICT","message":"目标时段已被占用","data":null}

错误信息用于帮助前端和用户定位问题，但系统不向客户端暴露服务器堆栈、SQL 细节或内部实现信息。前端依据错误码和 HTTP 状态决定是否保留表单、刷新局部数据或提示重新登录。

附录 E 测试用例与执行记录

测试覆盖时间冲突、正常流程、参数错误、业务规则阻止、资源筛选、并发一致性、周期预约、统计以及会议实际使用记录等场景。以下清单依据 2026-09-21 的实际测试记录整理。

表 E.1 测试条件清单

编号	类别	测试条件	预期结果	结果
T-01	时间冲突	两时段完全相同	判定冲突	通过
T-02	时间冲突	新时段被已有预约完全包含	判定冲突	通过
T-03	时间冲突	新时段完全包含已有预约	判定冲突	通过
T-04	时间冲突	前段部分重叠	判定冲突	通过
T-05	时间冲突	后段部分重叠	判定冲突	通过
T-06	时间冲突	首尾相接	不冲突	通过
T-07	时间冲突	两时段相离	不冲突	通过
T-08	时间冲突	业务链路命中冲突	409 且不落库	通过
T-09	时间冲突	改期冲突排除自身	仅检查其他有效预约	通过
T-10	时间规则	跨天预约仍在窗口内	允许	通过
T-11	时间规则	跨天预约超出窗口	400 拒绝	通过
T-12	正常流程	免审批分类正常提交	直接 CONFIRMED	通过
T-13	正常流程	需审批分类正常提交	进入 PENDING	通过
T-14	正常流程	修改未开始预约	重跑规则并按新分类重算状态	通过
T-15	正常流程	取消未开始预约	转 CANCELLED 并释放时段	通过
T-16	幂等	相同 requestId 重复提交	返回既有预约	通过
T-17	参数错误	主题/时间/人数缺失或非法	400 VALIDATION_ERROR	通过

软件学院软件实践说明书

编号	类别	测试条件	预期结果	结果
18 T-	参数错误	结束时间≤开始时间	400 拒绝	通过
19 T-	参数错误	空闲查询缺 date	400 缺少必需参数	通过
20 T-	参数错误	date 格式错误	400 参数格式无效	通过
21 T-	参数错误	minCapacity=0	400 拒绝	通过
22 T-	参数错误	空闲查询日期/时间为空	400 拒绝	通过
23 T-	业务规则	参会人数超过容量	400 ROOM_CAPACITY_EXCEEDED	通过
24 T-	业务规则	已开始预约改期	409 拒绝	通过
25 T-	业务规则	已开始预约取消	409 拒绝	通过
26 T-	业务规则	维护/停用会议室预约	409 ROOM_UNAVAILABLE	通过
27 T-	业务规则	禁用/限制/信用不足账号预约	403 拒绝	通过
28 T-	删除保护	存在未来预约的会议室删除	409 ROOM_DELETE_BLOCKED	通过
29 T-	删除保护	仅历史预约的会议室删除	同样阻止	通过
30 T-	资源查询	设施+最小容量筛选	仅返回同时满足项	通过
31 T-	资源查询	位置关键字筛选	仅返回位置命中项	通过
32 T-	资源查询	筛选条件为空白	视为未提供	通过
33 T-	空闲查询	剔除占用和非可预约会议室	仅返回空闲可预约项	通过
34 T-	空闲查询	结束时间≤开始时间	按次日结束推导	通过
35 T-	并发一致性	并发抢占同房同一时段	仅一单成功，其余冲突	通过
36 T-	并发一致性	过期版本/expected-state 不命中	fail-closed	通过
37 T-	周期预约	repeatWeeks=3	按周展开 3 场	通过
38 T-	周期预约	任一周发生冲突	整批拒绝、零落库	通过
39 T-	周期预约	提前量口径	只约束首场，其余规则逐周校验	通过
40 T-	周期预约	repeatWeeks 越界	400 拒绝	通过

编号	类别	测试条件	预期结果	结果
41	T- 周期预约	相同 requestId 重放	不新增场次	通过
42	T- 生命周期	已开始预约不能取消独立断言	409 拒绝	通过
43	T- 统计	使用率/日均时长/爽约率	按定义正确聚合	通过
44	T- 实际使用	组织者登记已确认且已结束会议	保存并支持幂等修正	通过
45	T- 实际使用	非法实际时间/人数	拒绝且不写入	通过
46	T- 实际使用	非组织者普通用户登记	403 拒绝	通过

E.1 自动化与集成测试执行汇总

表 E.2 测试执行汇总

测试层级	执行方式	执行数	失败	错误	跳过	结论
后端默认测试	mvn test	179	0	0	0	BUILD SUCCESS
并发集成测试	ReservationConcurrencyIT (TIMESLOT_IT_DB=true)	5	0	0	0	BUILD SUCCESS
前端类型检查	vue-tsc --noEmit	—	—	—	—	通过
接口级实测	15 个 curl 场景	15 场景	0	0	0	全部符合预期

并发集成测试连接本地 MySQL，并在测试后清理写入数据；测试记录显示执行后 reservation 表种子行数保持不变。

附录 F 系统运行与部署说明

本系统采用前后端分离的模块化单体架构。前端使用 Vue 3 + TypeScript + Vite，后端使用 Spring Boot 3.3.5 + Java 21 + MyBatis，数据库使用 MySQL 8.x。以下步骤用于本地开发、演示与课程验收环境。

F.1 环境要求

表 F.1 运行环境

组件	版本要求	用途
JDK	Java 21	后端编译与运行
Maven	3.9+	后端依赖与测试
Node.js	20+	前端开发环境
npm	与 Node.js 配套	前端依赖管理
MySQL	8.x	业务数据持久化
操作系统	Windows 10/11 或等价环境	仓库提供 Windows 一键启动脚本

F.2 数据库初始化

从零初始化推荐执行 `sql/init.sql`。该脚本会重建 17 张表并写入演示数据，因此不得直接用于需要保留数据的数据库。若需分步执行，可先执行 `sql/schema.sql`，再执行 `sql/data.sql`；已有数据库按 `sql/migrations/` 中 V1_1 至 V1_11 的迁移顺序升级。

推荐命令：`mysql --default-character-set=utf8mb4 -u<user> -p -e "source sql/init.sql"`

Windows 下不建议使用 `Get-Content ... | mysql` 管道执行含中文注释的 SQL，以避免本地代码页重编码导致脚本解析错误。

F.3 本地环境配置

仓库根目录使用 `.env.local` 保存本机数据库账号、JWT 密钥、后端端口和前端代理目标。该文件不应提交到 Git。JWT_SECRET 应使用至少 32 个字符的本地密钥。

```
DB_USERNAME=root
DB_PASSWORD=<本地数据库密码>
JWT_SECRET=<至少 32 字符的本地密钥>
```

```
SERVER_PORT=8081
VITE_API_PROXY_TARGET=http://localhost:8081
VITE_USE MOCK=false
```

F.4 启动方式

Windows 开发环境可在仓库根目录执行 `start-dev.cmd`，一键读取 `.env.local` 并分别启动后端和前端。也可手动启动。

后端：`cd backend → mvn spring-boot:run`

前端：`cd frontend → npm install → npm run dev`

默认前端开发地址：`http://localhost:5173`；后端端口由 `SERVER_PORT` 决定。

F.5 验证命令

前端类型检查：`npm run type-check`

前端构建：`npm run build`

后端测试：`mvn test`

并发验证：`python scripts/concurrency-test.py --base-url http://localhost:8081/api --token <JWT> --room-id <ID>`

并发脚本默认针对同一会议室发送 100 个重叠预约请求，并在请求完成后再次查询有效预约，检查是否存在重叠结果。运行前需要准备有效 JWT、可预约会议室以及与测试时间匹配的业务时钟。

F.6 部署与安全注意事项

表 F.2 部署与安全注意事项

事项	要求
密钥管理	不提交 <code>.env.local</code> 、真实数据库密码或 JWT 密钥。
数据库	生产/共享环境不得使用会 DROP 表的 <code>init.sql</code> ；应按迁移脚本升级。
认证	除登录接口外，业务接口需要 Bearer JWT；管理员接口要求 ADMIN 角色。
时间	业务规则使用统一业务 Clock；JWT 过期仍基于物理墙上时间。
前端	页面通过 <code>shared/api</code> 适配层访问后端，不在组件中复制后端业务裁决。
日志	操作审计记录关键治理行为，不以审计表替代完整服务器运行日志。